

Real-Time Retail Feedback Intelligence

Final Capstone Submission | ChicStyle GenAI-Powered Review Intelligence

Executive Summary

This project develops a real-time retail feedback intelligence system that transforms unstructured customer reviews into structured, actionable insights. By automatically classifying feedback, summarizing key issues, identifying sentiment and urgency, and drafting personalized responses, the solution helps customer support, merchandising, and operations teams respond faster during high-volume review periods.

The analysis shows that structured reasoning improves output quality and consistency. The strongest proposed production design combines retrieval-based context with structured reasoning, enabling the system to use similar historical reviews and approved responses as guidance while still producing review-specific outputs.

- Convert large volumes of review text into standardized business signals.
- Identify product, fit, quality, price, delivery, and other customer issues faster.
- Support consistent customer communication through structured personalized messages.
- Prepare the system for future retrieval-augmented deployment using historical response data.

Problem & Solution Summary

ChicStyle receives large volumes of customer reviews, especially during promotional and seasonal sales. Manual review monitoring is slow, inconsistent, and difficult to scale. Traditional keyword-based methods also miss nuance such as mixed sentiment, subtle fit issues, and context-specific quality concerns.

The proposed solution uses a GenAI-based review intelligence pipeline to classify each review, summarize the main customer experience, determine sentiment and urgency, and

generate a concise response message. This creates a structured feedback layer that can support dashboards, escalation workflows, and customer service operations.

Model Comparison & Insights

When no reference is provided, evaluation compares predictions against internally derived expectations based on the evaluation framework. This makes reference-free evaluation useful for pairwise comparison between versions of the same technique. For absolute benchmarking across techniques, a labeled ground truth dataset is more appropriate, though maintaining such datasets at large scale may require historical response data.

Across prompting approaches, more structured techniques produce more reliable and consistent outputs. The overall performance ranking observed from the analysis is:

RAG + Structured Reasoning > Structured Reasoning > Few-Shot > Zero-Shot

Structured reasoning improves consistency by breaking the task into stages: identifying the main issue, selecting the correct category, determining sentiment, summarizing the review, and generating a response. Retrieval further improves the proposed production design by grounding outputs in similar historical examples or approved responses.

Final Solution Proposal

The final proposed solution is a structured GenAI review intelligence system with a future retrieval-enhanced layer. The current notebook demonstrates the core classification, summarization, sentiment, urgency, and response-generation workflow. RAG is included as a production enhancement proposal, not as a completed implementation inside the notebook.

- Input layer: ingest customer reviews and product metadata such as rating, recommendation, division, department, and class.
- Reasoning layer: apply structured prompting to identify the primary issue, sentiment, urgency, and customer-facing response.
- Output layer: produce Category, Summary, Sentiment, Urgency, and PersonalizedMessage fields for downstream reporting and workflow use.
- RAG enhancement: retrieve similar historical reviews and approved responses to improve consistency, reduce hallucination, and align responses with business policy.

- Human oversight: review high-urgency, low-confidence, or customer-sensitive cases before customer-facing deployment.

Implementation Plan

Phase 1: Pilot Deployment

- Deploy the structured prompting workflow on a representative review sample.
- Validate outputs for category, sentiment, urgency, summary quality, and response usefulness.
- Create dashboard-ready fields for business review.

Phase 2: Dashboard & Monitoring

- Track review trends by category, sentiment, urgency, and product group.
- Monitor repeated issues such as sizing, fabric quality, delivery delays, and price concerns.
- Add quality checks for output consistency and escalation rules.

Phase 3: RAG Enhancement

- Build a vector index of historical reviews, labeled outcomes, and approved response examples.
- Retrieve top similar cases as reference guidance for structured generation.
- Use retrieved context to improve consistency and brand-safe response quality.

Phase 4: Production Integration

- Integrate the system with CRM, support, and analytics workflows.
- Route high-urgency or sensitive reviews to human review.
- Continuously improve the retrieval index and prompts based on feedback.

Estimated Benefits vs Costs

Estimated Benefits

- 30–50% reduction in manual review triage effort by automating classification, summarization, and response drafting.
- 2–3x faster identification of negative sentiment and urgent product issues during peak review periods.

- More consistent response quality across customer support teams.
- Earlier detection of recurring product issues that can inform merchandising and operations decisions.

Basis for Benefit Estimates

- Assumes manual review triage typically requires several minutes per customer review.
- Assumes high-volume retail review periods where automation can reduce repetitive reading and labeling work.
- Assumes human review remains in place for high-risk, high-urgency, or customer-sensitive responses.
- Future RAG and CRM integration may further improve consistency and response quality.

Development Costs

- One-time pipeline development and testing: approximately \$3,000–\$8,000 for a pilot implementation.
- Dashboard and reporting setup: approximately \$1,000–\$3,000 depending on integration depth.
- RAG knowledge-base preparation and vector index setup: approximately \$2,000–\$6,000 depending on data quality and volume.

Operational Costs

- LLM API usage: approximately \$200–\$1,000 per month depending on review volume, model choice, and prompt size.
- Cloud storage and compute: approximately \$100–\$300 per month for pilot to early-scale usage.
- Ongoing monitoring and prompt maintenance: approximately \$100–\$500 per month depending on governance requirements.

Estimated per-review processing cost is approximately \$0.002–\$0.01 for lightweight classification and generation workflows, depending on prompt length, model selection, and whether retrieval context is included.

Risks & Mitigation

- Risk: incorrect classification or summary. Mitigation: evaluation against labeled samples and human review for high-risk cases.
- Risk: hallucinated or overconfident responses. Mitigation: retrieval grounding, approved response templates, and escalation rules.
- Risk: inconsistent tone across responses. Mitigation: structured prompt rules and brand-approved message guidelines.
- Risk: cost growth at scale. Mitigation: batching, model selection, prompt optimization, and selective use of RAG for cases that need grounding.

Conclusion

The project demonstrates that GenAI can convert unstructured retail reviews into useful business intelligence. A structured reasoning pipeline provides a strong foundation for classification, summarization, sentiment analysis, urgency detection, and response drafting. For production, adding RAG as a future enhancement will improve grounding, consistency, and traceability while enabling scalable customer feedback operations.



Real-Time Retail Feedback Intelligence

Business Context

ChicStyle — a growing fashion retail platform — experiences massive spikes in customer activity during festive seasons and holiday sales. As people buy clothing and accessories for celebrations, the volume of incoming reviews increases drastically. These reviews pour in every hour, ranging from positive praise to urgent complaints about fit, delivery delays, product defects, or sizing issues. During such high-pressure periods, even a slight delay in reading or responding to customer feedback can have serious consequences. If the retail team fails to take quick action, customers may feel ignored during an emotionally significant time (festive purchases), leading to frustration, spoiled

shopping experiences, and ultimately reduced trust in the brand. This not only results in immediate business loss but also impacts repeat purchases and long-term loyalty. To avoid customer dissatisfaction and protect brand reputation during peak sales periods, ChicStyle needs a system that can process thousands of reviews instantly, accurately, and with business context.

Problem Statement

Retailers need a smart feedback analysis system that can automatically process large volumes of real-time customer reviews, accurately detect sentiment (positive, negative, neutral), identify which product or service the feedback is about, and highlight urgent or high-impact issues. Traditional NLP models often struggle with this because they depend on fixed rules and limited training data. They can't easily understand complex or mixed feedback. For example, in the review "The fit is great but the color was not as per the product image," older systems may give only one sentiment, either positive or negative, missing that the review talks positively about the fit but negatively about the color. Generative AI models are much better at this. They can understand the full meaning of a sentence, separate opinions about different products, and catch mixed emotions. This helps retailers get more accurate insights and respond to customer problems faster.

Objective

Build a Generative AI feedback system that uses prompt engineering (Zero-Shot, Few-Shot, and Chain-of-Thought prompting) to:

- Analyze and categorize sentiment in real time.
- Detect which product or service each feedback refers to.
- Summarize insights by product category and urgency level.
- Automatically send short personalized messages to customers based on sentiment—thanking them for positive feedback, acknowledging neutral comments, and apologizing for negative ones while informing them that a team member will reach out soon.
- Generate short, actionable reports for retail teams. This intelligent system will help retailers take quick action on customer issues, improve product quality faster, and enhance customer satisfaction—turning massive unstructured feedback into meaningful, real-time business intelligence.

Dataset Used for the Notebook

This analysis uses the "Women's E-Commerce Clothing Reviews" dataset. The key columns for the analysis are:

- Clothing.ID: A unique ID for each piece of clothing.
- Age: The age of the reviewer (Positive Integer).
- Title: The title of the review (String).
- Review.Text: The main body of the customer's review text (String).
- Rating: The product score given by the customer, from 1 (Worst) to 5 (Best) (Positive Ordinal Integer).
- Recommended.IND: A binary variable indicating if the customer recommends the product (1 for recommended, 0 for not recommended).
- Positive.Feedback.Count: The number of other customers who found the review helpful(Positive Integer).
- Division.Name: The high-level division of the product (Categorical).
- Department.Name: The specific department of the product (Categorical).
- Class.Name: The clothing item's class name (Categorical)

Installing and Importing Necessary Libraries

First, let's set up the environment by installing the required Python libraries.

```
In [1]: #!/pip uninstall -y langchain langchain-community langchain-openai
# Install the required libraries for the project
!pip install -q ratelimit ipywidgets
!jupyter nbextension enable --py widgetsnbextension
!pip install -q openai==1.66.3 \
    tqdm==4.67.0 \
    tiktoken==0.9.0 \
    langchain==0.3.20 \
    langchain-community==0.3.19 \
    langchain-chroma==0.2.2 \
    langchain-openai==0.3.9 \
    chromadb==0.6.3
```

Enabling notebook extension jupyter-js-widgets/extension...
 - Validating: OK

```
In [2]: # Import the required libraries for the project
# import libraries for data manipulation
import numpy as np
import pandas as pd
from tqdm import tqdm
# OpenAI-compatible client
from openai import OpenAI
# import libraries for data visualization
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from wordcloud import WordCloud, STOPWORDS
import time
from ratelimit import limits, sleep_and_retry
from os import path
import csv, re, json
from scipy import stats
```

```

from typing import Any, Dict, Optional
from dataclasses import dataclass
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
from IPython.display import clear_output
# disable warning for fonts not loaded
plt.rcParams['font.family'] = 'sans-serif'
plt.rcParams['text.usetex'] = False
warnings.filterwarnings("ignore", category=UserWarning, message=".*Glyph 14*")
plt.rcParams['axes.unicode_minus'] = False

```

Data Loading

Loading and Understanding the Data

```

In [3]: from google.colab import drive

# Mount gdrive and set project root
drive.mount('/content/drive')
project_root = '/content/drive/MyDrive/AAIDSCourses/projects'
project_folder = 'GenAIReviewCategorization'

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```

In [4]: # Read the data. Use semicolon as a delimiter
df = pd.read_csv(path.join(project_root, 'GenAIReviewCategorization', 'RetailFeedback.csv'), delimiter=';')
# Drop the first column by index. The first column is a sequential number not a name
df = df.drop(df.columns[0], axis=1)

```

Data Overview

```

In [6]: # rename columns
df = df.rename(columns={'Clothing.ID': 'ClothingID', 'Review.Text': 'Review', 'Division.Name': 'Division', 'Department.Name': 'Department', 'Class.Name': 'Class'})
df.info()

```



```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23486 entries, 0 to 23485
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ClothingID            23486 non-null  int64
1   Age                   23486 non-null  int64
2   Title                 19676 non-null  object
3   Review                22641 non-null  object
4   Rating                23486 non-null  int64
5   Recommended           23486 non-null  int64
6   PositiveFeedbackCount 23486 non-null  int64
7   Division              23472 non-null  object
8   Department            23472 non-null  object
9   ClassName             23472 non-null  object
dtypes: int64(5), object(5)
memory usage: 1.8+ MB

```

Sanity checks

```

In [7]: # manual check sample values
display(df.head().style.hide(axis="index"))
display(df.tail().style.hide(axis="index"))

```

ClothingID	Age	Title	Review	Rating	Recommended	PositiveFeedba
767	33	nan	Absolutely wonderful - silky and sexy and comfortable	4	1	
1080	34	nan	Love this dress! it's sooo pretty. i happened to find it in a store, and i'm glad i did bc i never would have ordered it online bc it's petite. i bought a petite and am 5'8". i love the length on me- hits just a little below the knee. would definitely be a true midi on someone who is truly petite.	5	1	
1077	60	Some major design flaws	I had such high hopes for this dress and really wanted it to work for me. i initially ordered the petite small (my usual size) but i found this to be outrageously small. so small in fact that i could not zip it up! i reordered it in petite medium, which was just ok. overall, the top half was comfortable and fit nicely, but the	3	0	

ClothingID	Age	Title	Review	Rating	Recommended	PositiveFeedba
			bottom half had a very tight under layer and several somewhat cheap (net) over layers. imo, a major design flaw was the net over layer sewn directly into the zipper - it c			
1049	50	My favorite buy!	I love, love, love this jumpsuit. it's fun, flirty, and fabulous! every time i wear it, i get nothing but great compliments!	5		1
847	47	Flattering shirt	This shirt is very flattering to all due to the adjustable front tie. it is the perfect length to wear with leggings and it is sleeveless so it pairs well with any cardigan. love this shirt!!!	5		1

ClothingID	Age	Title	Review	Rating	Recommended	PositiveFeedba
1104	34	Great dress for many occasions	I was very happy to snag this dress at such a great price! it's very easy to slip on and has a very flattering cut and color combo.	5	1	
862	48	Wish it was made of cotton	It reminds me of maternity clothes. soft, stretchy, shiny material. cut is flattering and drapes nicely. i only found one button to close front... looked awkward. nice long sleeves. not for me but maybe for others. just ok.	3	1	
1104	31	Cute, but see through	This fit well, but the top was very see through. this never would have worked for me. i'm glad i was able to try it on in the store and didn't order it online. with different fabric, it would have been great.	3	0	
1084	28	Very cute dress, perfect for summer parties and we	I bought this dress for a wedding i have this summer, and it's so cute. unfortunately the fit isn't perfect. the	3	1	

ClothingID	Age	Title	Review	Rating	Recommended	PositiveFeedba
			medium fits my waist perfectly, but was way too long and too big in the bust and shoulders. if i wanted to spend the money, i could get it tailored, but i just felt like it might not be worth it. side note - this dress was delivered to me with a nordstrom tag on it and i found it much cheaper there after looking!			
1104	52	Please make more like this one!	This dress in a lovely platinum is feminine and fits perfectly, easy to wear and comfy, too! highly recommend!	5		1

```
In [8]: # Check for missing values
df.isnull().sum()
```

Out[8]:

	0
ClothingID	0
Age	0
Title	3810
Review	845
Rating	0
Recommended	0
PositiveFeedbackCount	0
Division	14
Department	14
ClassName	14

dtype: int64

Data Cleaning and Preprocessing

Think about it: The Review Text column is the most critical feature for our Generative AI model. What should be done with rows where this text is missing?

Two options:

- Ignore reviews missing text if only small percentage without text. If a customer does not leave a review, it is likely the customer does not expect a post-sale response. Will follow this option in the milestone submission. In the final submission, plan to add specific instruction to treat no-comment reviews.
- Impute a default value, i.e. "No comment" as input to LLM. This can be done without modifying dataframe itself. Rather can enforce the rule in LLM processing logic.

```
In [9]: # explore rows where Division/Department/Class is null
df_missing_categorical_info= df[df['Division'].isnull() | df['Department'].i
display(df_missing_categorical_info.info())
print("\nSample no-category records:\n")
df_missing_categorical_info.head().style.hide(axis="index")
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 14 entries, 9444 to 23011
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ClothingID                            14 non-null     int64
1   Age                                    14 non-null     int64
2   Title                                  13 non-null     object
3   Review                                13 non-null     object
4   Rating                                14 non-null     int64
5   Recommended                            14 non-null     int64
6   PositiveFeedbackCount                 14 non-null     int64
7   Division                              0 non-null      object
8   Department                            0 non-null      object
9   ClassName                             0 non-null      object
dtypes: int64(5), object(5)
memory usage: 1.2+ KB
None
Sample no-category records:
```

Out[9]:

ClothingID	Age	Title	Review	Rating	Recommended	PositiveFeedba
------------	-----	-------	--------	--------	-------------	----------------

72	25	My favorite socks!!!	I never write reviews, but these socks are so worth it! they are incredibly soft, stretchy, and hug your feet just tight enough where you don't even realize you're wearing socks, but they aren't too tight. i wash them straight away after wearing so i can wear them again. you won't regret it. true to color!	5		1
----	----	----------------------	---	---	--	---

492	23	So soft!	I just love this hoodie! it is so soft and comfortable. i am usually an extra small and that size fit me great. i wish it didn't have the brand on the back of it but i can get over that. this is a very cozy piece that i will wear many times.	5		1
-----	----	----------	---	---	--	---

492	49	Wardrobe staple	Love this hoodie. so soft and goes with everything. highly recommend for comfort	5		1
-----	----	-----------------	--	---	--	---

ClothingID	Age	Title	Review	Rating	Recommended	PositiveFeedbackCount
			and style. great company.			
492	48	nan	nan	5	1	
152	36	Warm and cozy	Just what i was looking for. soft, cozy and warm.	5	1	

```
In [10]: ids_missing_categorical_info = df_missing_categorical_info.ClothingID.unique
print(f"Total reviews with with missing categorical info:{len(df[df.ClothingID.isin(ids_missing_categorical_info)])}")
print("ClothingIDs with missing categorical info:", ids_missing_categorical_info)
```

Total reviews with with missing categorical info:14

ClothingIDs with missing categorical info: [72 492 152 184 772 665 136]

Observation:

- ClothingIDs in [72 492 152 184 772 665 136] are missing categorical info in 14 associated reviews.
- Other ClothingIDs not in the above list do not have missing categorical info in their associated reviews.

```
In [11]: # explore rows where review is missing
df_no_review = df[df['Review'].isnull()]
print("\nSample no-review records:\n")
display(df_no_review.head().style.hide(axis="index"))
print("\nStats for no-review records:\n")
display(df_no_review.describe(include = 'all').T)
print("\nStats for not-recommended and no-review records:\n")
df_no_review_not_recommended = df_no_review[df_no_review['Recommended'] == 0]
display(df_no_review_not_recommended.describe(include = 'all').T)
```

Sample no-review records:

ClothingID	Age	Title	Review	Rating	Recommended	PositiveFeedbackCount
861	23	nan	nan	5	1	0
1081	31	nan	nan	5	1	0
1133	50	nan	nan	5	1	0
861	35	nan	nan	4	1	0
1126	35	nan	nan	5	1	0

Stats for no-review records:

	count	unique	top	freq	mean	std	m
ClothingID	845.0	NaN	NaN	NaN	885.6	226.957375	0
Age	845.0	NaN	NaN	NaN	41.005917	10.708048	20
Title	1	1	Such a beautiful dress!	1	NaN	NaN	NaN
Review	0	0	NaN	NaN	NaN	NaN	NaN
Rating	845.0	NaN	NaN	NaN	4.530178	0.880597	1
Recommended	845.0	NaN	NaN	NaN	0.915976	0.277588	0
PositiveFeedbackCount	845.0	NaN	NaN	NaN	0.0	0.0	0
Division	844	3	General	485	NaN	NaN	NaN
Department	844	6	Tops	420	NaN	NaN	NaN
ClassName	844	19	Knits	217	NaN	NaN	NaN

Stats for not-recommended and no-review records:

	count	unique	top	freq	mean	std	n
ClothingID	71.0	NaN	NaN	NaN	895.098592	216.515928	60
Age	71.0	NaN	NaN	NaN	42.71831	11.15499	24
Title	0	0	NaN	NaN	NaN	NaN	NaN
Review	0	0	NaN	NaN	NaN	NaN	NaN
Rating	71.0	NaN	NaN	NaN	2.323944	1.038882	0
Recommended	71.0	NaN	NaN	NaN	0.0	0.0	0
PositiveFeedbackCount	71.0	NaN	NaN	NaN	0.0	0.0	0
Division	71	3	General	44	NaN	NaN	NaN
Department	71	6	Tops	33	NaN	NaN	NaN
ClassName	71	15	Dresses	15	NaN	NaN	NaN

Observation:

- There are 845 records without review, less than 5% of the total records.
- For all no-review records, Positive Feedback Count is zero. This is expected as empty review does not help any other customer.
- Out of 845 records without review, only 1 record has title.
- Out of 845 records without review, 71 records do not recommend the purchased items, 0.3%(71/23486) of the total records. Rest of the no-review records all recommended the purchased item. Hence we recommend to drop the no-review record as most of them, $\geq 91.5\%$ $(845-71)/845$, do

recommend the purchased items. Also given that it is difficult to follow up on no-review feedbacks, it is recommended to exclude no-review record in the scope of this project. In production scenario, can add strict and specific processing rules in the prompt for such reviews. Related example rules will be added in final submission.

```
In [12]: # drop the no-review records for further explore, use a copy to avoid warnin
df_has_review = df.dropna(subset=['Review']).copy()
# set ClothingID as type string
df_has_review['ClothingID'] = df_has_review['ClothingID'].astype(str)
# set Recommended as type bool
df_has_review['Recommended'] = df_has_review['Recommended'].astype(bool)
print("\nStats for records with review:\n")
display(df_has_review.info())
print("\nSample records with review:\n")
df_has_review.head().style.hide(axis="index")
```

Stats for records with review:

```
<class 'pandas.core.frame.DataFrame'>
Index: 22641 entries, 0 to 23485
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ClothingID                           22641 non-null  object
1   Age                                   22641 non-null  int64
2   Title                                 19675 non-null  object
3   Review                               22641 non-null  object
4   Rating                               22641 non-null  int64
5   Recommended                           22641 non-null  bool
6   PositiveFeedbackCount                 22641 non-null  int64
7   Division                              22628 non-null  object
8   Department                            22628 non-null  object
9   ClassName                             22628 non-null  object
dtypes: bool(1), int64(3), object(6)
memory usage: 1.7+ MB
None
Sample records with review:
```

Out[12]:

ClothingID	Age	Title	Review	Rating	Recommended	PositiveFeedb
------------	-----	-------	--------	--------	-------------	---------------

767	33	nan	Absolutely wonderful - silky and sexy and comfortable	4	True	
-----	----	-----	---	---	------	--

1080	34	nan	Love this dress! it's sooo pretty. i happened to find it in a store, and i'm glad i did bc i never would have ordered it online bc it's petite. i bought a petite and am 5'8". i love the length on me- hits just a little below the knee. would definitely be a true midi on someone who is truly petite.	5	True	
------	----	-----	--	---	------	--

1077	60	Some major design flaws	I had such high hopes for this dress and really wanted it to work for me. i initially ordered the petite small (my usual size) but i found this to be outrageously small. so small in fact that i could not zip it up! i reordered it in petite medium, which was just ok. overall, the top half was comfortable and fit nicely, but the	3	False	
------	----	-------------------------	--	---	-------	--

ClothingID	Age	Title	Review	Rating	Recommended	PositiveFeedback
			bottom half had a very tight under layer and several somewhat cheap (net) over layers. imo, a major design flaw was the net over layer sewn directly into the zipper - it c			
1049	50	My favorite buy!	I love, love, love this jumpsuit. it's fun, flirty, and fabulous! every time i wear it, i get nothing but great compliments!	5	True	
847	47	Flattering shirt	This shirt is very flattering to all due to the adjustable front tie. it is the perfect length to wear with leggings and it is sleeveless so it pairs well with any cardigan. love this shirt!!!	5	True	

Exploratory Data Analysis

We will answer the following questions with EDA

1. What is the summary statistics of the numerical data? What can you infer about the distribution of Age, Rating, and Positive Feedback Count?
2. How many unique values are there in the categorical columns like Division Name, Department Name, and Class Name?

3. What is the overall distribution of product Rating? Is the dataset skewed towards positive or negative reviews?
4. Which Department Name receives the highest average rating, and which receives the lowest? What might this indicate?
5. What are the most common words found in highly-rated reviews (4-5 stars) versus poorly-rated reviews (1-2 stars)? What initial hypotheses can you form about the key drivers of customer satisfaction and dissatisfaction?

```
In [13]: def histogram_boxplot_ax(feature, ax_box, ax_hist, bins="auto"):
    """ Boxplot and histogram combined
    feature: 1-d feature array
    figsize: size of fig (default (15, 10))
    bins: number of bins (default "auto")
    """
    # Creating the subplots
    # Boxplot will be created and the mean value of the column will be indicated
    sns.boxplot(x=feature, ax=ax_box, showmeans=True, color='red')
    sns.histplot(x=feature, kde=False, ax=ax_hist, bins=bins)

    # Add mean and median to the histogram
    ax_hist.axvline(np.mean(feature), color='g', linestyle='--')
    ax_hist.axvline(np.median(feature), color='black', linestyle='--')

def plot_tiled_combo(df, columns, nrows, ncols, bins="auto"):
    from matplotlib.gridspec import GridSpec, GridSpecFromSubplotSpec

    fig = plt.figure(figsize=(3*ncols, 2*nrows))
    outer = GridSpec(nrows, ncols, figure=fig)

    for idx, col in enumerate(columns):
        r = idx // ncols
        c = idx % ncols

        # Create inner grid (box + hist) inside each tile
        inner = GridSpecFromSubplotSpec(
            2, 1,
            subplot_spec=outer[r, c],
            height_ratios=[1, 3]
        )

        ax_box = fig.add_subplot(inner[0])
        ax_hist = fig.add_subplot(inner[1], sharex=ax_box)
        ax_hist.set_title('')
        histogram_boxplot_ax(df[col], ax_box, ax_hist, bins)
        ax_box.set_title(col)
        ax_box.set_xlabel('')

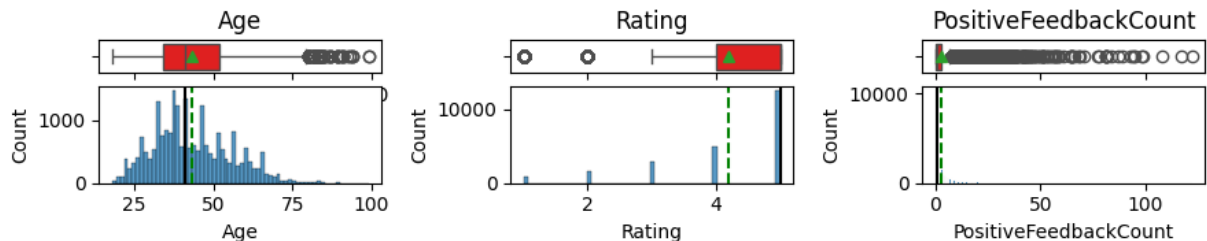
    plt.tight_layout()
    plt.show()
```

```
In [14]: df_has_review.describe().T
```

```
Out[14]:
```

	count	mean	std	min	25%	50%	75%
Age	22641.0	43.280376	12.326980	18.0	34.0	41.0	52.0
Rating	22641.0	4.183561	1.115762	1.0	4.0	5.0	5.0
PositiveFeedbackCount	22641.0	2.630582	5.786164	0.0	0.0	1.0	3.0

```
In [15]: plot_tiled_combo(df_has_review,columns=["Age", "Rating", "PositiveFeedbackCo
```



Observation

- Age has a rather symmetrical distribution, a wide range of values from 18-99 and slightly skewed to the right.
- Median value of Rating is 5. In fact $\geq 75\%$ of the rating values are 5. The distribution is concentrated in value 5, but skewed to the left, low ratings.
- Median of PositiveFeedbackCount is 1, with max of 122, heavily skewed to the right.

```
In [16]: # unique values of Division, Department, Class
print("\nUnique divisions:\n")
display(df_has_review.Division.dropna().unique())
print("\nUnique departments:\n")
display(df_has_review.Department.dropna().unique())
print("\nUnique classes:\n")
display(df_has_review.ClassName.dropna().unique())
```

Unique divisions:

```
array(['Initmates', 'General', 'General Petite'], dtype=object)
```

Unique departments:

```
array(['Intimate', 'Dresses', 'Bottoms', 'Tops', 'Jackets', 'Trend'],
      dtype=object)
```

Unique classes:

```
array(['Intimates', 'Dresses', 'Pants', 'Blouses', 'Knits', 'Outerwear',
      'Lounge', 'Sweaters', 'Skirts', 'Fine gauge', 'Sleep', 'Jackets',
      'Swim', 'Trend', 'Jeans', 'Legwear', 'Shorts', 'Layering',
      'Casual bottoms', 'Chemises'], dtype=object)
```

Observation

How many unique values are there in the categorical columns like Division Name, Department Name, and Class Name?

- 3 unique division names: 'Initmates', 'General', 'General Petite'
- 6 unique department names: 'Intimate', 'Dresses', 'Bottoms', 'Tops', 'Jackets', 'Trend'
- 20 unique classe names: 'Intimates', 'Dresses', 'Pants', 'Blouses', 'Knits', 'Outerwear', 'Lounge', 'Sweaters', 'Skirts', 'Fine gauge', 'Sleep', 'Jackets', 'Swim', 'Trend', 'Jeans', 'Legwear', 'Shorts', 'Layering', 'Casual bottoms', 'Chemises'

```
In [17]: display(df_has_review.groupby('Rating').size())
# rating By Division, Department, ClassName
fig, axes = plt.subplots(1, 3, figsize=(12, 3))
# ----- 1. Department -----
sns.violinplot(x="Department", y="Rating", data=df_has_review, ax=axes[0])
dept_order = ['Intimate', 'Dresses', 'Bottoms', 'Tops', 'Jackets', 'Trend']
grouped_dept = df_has_review.groupby('Department')['Rating']
for i, dept in enumerate(dept_order):
    if dept in grouped_dept.mean():
        axes[0].scatter(i, grouped_dept.mean()[dept], marker='D')
axes[0].set_title("Rating by Department")
axes[0].tick_params(axis='x', rotation=45, labelsz=7)

# ----- 2. Division -----
sns.violinplot(x="Division", y="Rating", data=df_has_review, ax=axes[1])
grouped_div = df_has_review.groupby('Division')['Rating']
div_order = df_has_review['Division'].unique()
for i, div in enumerate(div_order):
    if div in grouped_div.mean():
        axes[1].scatter(i, grouped_div.mean()[div], marker='D')
axes[1].set_title("Rating by Division")
axes[1].tick_params(axis='x', rotation=45, labelsz=7)

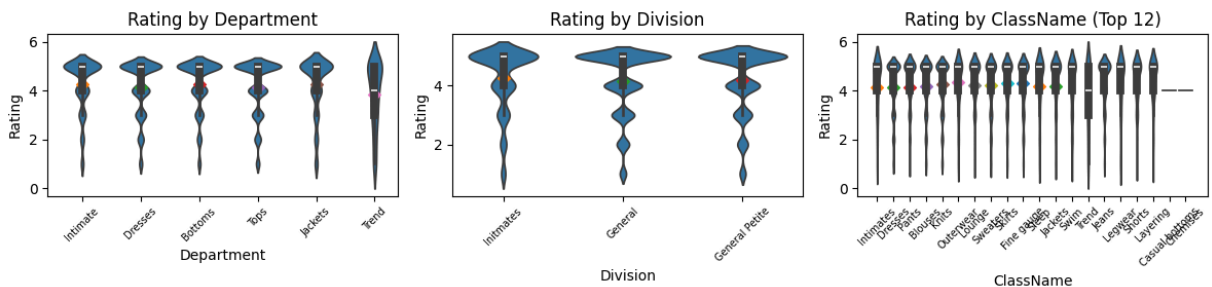
# ----- 3. ClassName -----
sns.violinplot(x="ClassName", y="Rating", data=df_has_review, ax=axes[2])
# Optional: limit to top N classes for readability
top_classes = df_has_review['ClassName'].value_counts().head(12).index
df_top_class = df_has_review[df_has_review['ClassName'].isin(top_classes)]
grouped_class = df_top_class.groupby('ClassName')['Rating']
for i, cls in enumerate(top_classes):
    if cls in grouped_class.mean():
        axes[2].scatter(i, grouped_class.mean()[cls], marker='D')
axes[2].set_title("Rating by ClassName (Top 12)")
axes[2].tick_params(axis='x', rotation=45, labelsz=7)

plt.tight_layout()
plt.show()
```


0

Rating

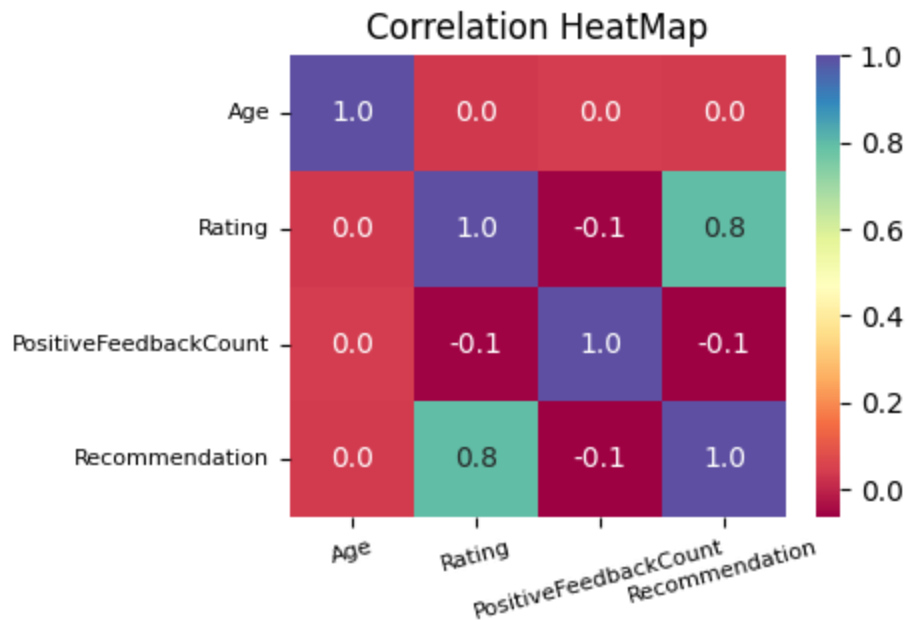
1	821
2	1549
3	2823
4	4908
5	12540

dtype: int64**Observation**

Which Department Name receives the highest average rating, and which receives the lowest? What might this indicate?

- Intimate, Bottoms, Jackets tend to receive high average rating whereas Dresses, Tops and Trend tend to receive low average rating.
- The above observation is reasonable since buyers of Intimate, Bottoms, Jackets pay more attention to functionalities than to appearance while people focus more on appearance for Dresses, Tops and Trend, with Trend being the most appearance-focused department. This observation is also confirmed by mean/median comparison across class names.
- There is no significant difference in rating across three divisions.

```
In [18]: # Convert Recommended to integer column Recommendation
df_has_review['Recommendation'] = df_has_review.Recommended.astype(int)
num_var = ['Age', 'Rating', 'PositiveFeedbackCount', 'Recommendation']
corr = df_has_review[num_var].corr()
plt.figure(figsize=(4, 3))
ax = sns.heatmap(corr, annot = True, cmap = 'Spectral', fmt = ".1f", xticklab
ax.set_title('Correlation HeatMap')
ax.tick_params(axis='x', labelsz=8, rotation=15)
ax.tick_params(axis='y', labelsz=8)
```



Observation:

- There is strong correlation between Rating and Recommendation, a 0.8 positive correlation.
- There is weak negative correlation of -0.1 for the pair of Recommendation & PositiveFeedbackCount and the pair of Rating and PositiveFeedbackCount. This can be interpreted as customers tend to pay more attention to low-rating and no-recommendation reviews and consider them to be more helpful, a suggestion for LLM model to prioritize low-rating reviews without recommendation.

```
In [19]: def plot_wordcloud(ax, text_list, title, stopwords, background_color="white")
        """
        Generate and plot a word cloud from a list of text strings.
        Parameters:
        text_list (list of str): input text data
        figsize (tuple): figure size
        background_color (str): background color of word cloud
        """
        # 1. Combine all text
        text = " ".join(str(t) for t in text_list)
        # 2. Generate word cloud
        wc = WordCloud(width=300,height=200,background_color=background_color,st
        # 3. Plot on axis
        ax.imshow(wc, interpolation="bilinear")
        ax.axis("off")
        ax.set_title(title)
```

```
In [20]: # using WordCloud to find top words used in both high-rating and low-rating
fig, axes = plt.subplots(2, 2, figsize=(8, 6))
# Start with default stopwords
stopwords = set(STOPWORDS)
plot_wordcloud(axes[0,0], np.array(df_has_review[df_has_review.Rating>=4].Re
```


- From the Word Cloud using default stopwords set, size/fit and look/color for tops and dresses are key drivers for both customer satisfaction and dissatisfaction.

Building the Generative AI Pipeline

We will now build a system to analyze the reviews. This involves setting up the AI client, designing prompts, generating structured data, and evaluating the results.

For this project, we will analyze and categorize a sample of **50-100 customer reviews**. This number is chosen intentionally. Since the API has a **budget limit of \$20**, running prompts on very large datasets can quickly exhaust your quota—especially because this exercise may involve **multiple iterations, prompt refinements, and repeated evaluations**.

To avoid unnecessary cost and ensure efficient experimentation, we recommend the following approach:

- **Use very small samples (5-10 reviews)** during the **initial testing phase** to validate your prompt structure and logic.
- **Scale up to 100 reviews** for the **final evaluation phase**, ensuring enough data to compare prompting techniques without draining your budget.
- This strategy helps maintain cost control while still providing meaningful insights across Zero-Shot, Few-Shot, and Chain-of-Thought techniques.

Setup AI Client

```
In [21]: import os
from google.colab import userdata # Colab helper to read secrets
from openai import OpenAI         # OpenAI-compatible client for OpenAI
# Setting up the OPENAI Client
# Read the OpenAI API key from Colab Secrets (secret name must match exactly)
mygreatlearnin_endpoint = "https://aibe.mygreatlearning.com/openai/v1"
openai_api_key = userdata.get("OPEN_AI_KEY") # retrieve this secret in Colab
personal_openai_api_key = userdata.get("OPEN_AI_KEY_PERSONAL")
openai_endpoint = "https://api.openai.com/v1"

# Instantiate the OpenAI-compatible client pointing to mygreatlearning endpoint
# This is only for connection and quota check. We will use langchain ChatOpenAI
api_key = openai_api_key
base_url = mygreatlearnin_endpoint
api_key = personal_openai_api_key
base_url = openai_endpoint
client = OpenAI(
```

```

    api_key=api_key,
    base_url=base_url
)

print("OpenAI client created. You can now use `client` to call embeddings and chat endpoints.")
# List available models
print("Available Models (sample):")
models = client.models.list()
model_ids = list(filter(lambda x: x.startswith("gpt-4"), [m.id for m in models]))
print(model_ids, "\n")

```

OpenAI client created. You can now use `client` to call embeddings and chat endpoints.

Available Models (sample):

```

['gpt-4-0613', 'gpt-4', 'gpt-4-turbo', 'gpt-4-turbo-2024-04-09', 'gpt-4o',
'gpt-4o-2024-05-13', 'gpt-4o-mini-2024-07-18', 'gpt-4o-mini', 'gpt-4o-2024-08-06',
'gpt-4o-audio-preview', 'gpt-4o-realtime-preview', 'gpt-4o-realtime-preview-2024-12-17',
'gpt-4o-audio-preview-2024-12-17', 'gpt-4o-mini-realtime-preview-2024-12-17',
'gpt-4o-mini-audio-preview-2024-12-17', 'gpt-4o-mini-realtime-preview',
'gpt-4o-mini-audio-preview', 'gpt-4o-2024-11-20', 'gpt-4o-mini-search-preview-2025-03-11',
'gpt-4o-mini-search-preview', 'gpt-4o-transcribe', 'gpt-4o-mini-transcribe',
'gpt-4o-mini-tts', 'gpt-4.1-2025-04-14', 'gpt-4.1', 'gpt-4.1-mini-2025-04-14',
'gpt-4.1-mini', 'gpt-4.1-nano-2025-04-14', 'gpt-4.1-nano', 'gpt-4o-realtime-preview-2025-06-03',
'gpt-4o-audio-preview-2025-06-03', 'gpt-4o-transcribe-diarize', 'gpt-4o-mini-transcribe-2025-12-15',
'gpt-4o-mini-transcribe-2025-03-20', 'gpt-4o-mini-tts-2025-03-20', 'gpt-4o-mini-tts-2025-12-15',
'gpt-4o-search-preview', 'gpt-4o-search-preview-2025-03-11']

```

LangChain tool setup

We will use langchain OpenAI compatible interfaces for calling LLM service.

```

In [22]: # to use LangChain LCEL syntax, instantiate llm instance here
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser
# gpt-4.1 → part of the newer "4.x" iteration (more recent update in the GPT series)
# gpt-4o → "o" stands for omni (multimodal focus: text, image, audio), not a number
# model_name = 'gpt-4o'
model_name = 'gpt-4.1' # we choose this model as it better handles LLM judge prompts
temperature = 0
llm = ChatOpenAI(model=model_name, temperature=temperature, api_key=api_key, base_url=base_url)

```

Prompt Engineering and Evaluation

Based on the previous milestone submission and findings, we apply both CoT to LLM judge and review prompt with additional updates to improve both prediction and evaluation performance.

Create a LLM-as-a-judge class LlmJudge to evaluate the quality of model outputs

```

In [23]: from contextlib import contextmanager
import time

@contextmanager
def ProcessTimer(name="Block"):
    start = time.perf_counter()
    yield
    end = time.perf_counter()
    print(f"{name} took {end - start:.6f} seconds")

class Utils:
    """
    Utility/helper functions for LLM-based processing classes.

    Responsible for:
    - Converting pandas DataFrame rows into LangChain input format
    - Batch conversion of DataFrames to input dictionaries
    - Converting LLM results into structured pandas DataFrames

    This separation keeps LLM-based processing classes focused on LLM execution
    """

    @staticmethod
    def df_row_to_langchain_input(df, idx, column_map=None, default_map=None):
        """
        Convert a DataFrame row into a LangChain-compatible input dictionary
        using flexible column mapping and per-key default values.

        Parameters:
        -----
        df : pd.DataFrame
            DataFrame
        idx : int
            Row index

        column_map : dict, optional
            Mapping of output keys → DataFrame column names

        default_map : dict, optional
            Mapping of output keys → default values

        Example:
        {
            "title": "",
            "review": "No comment",
            "rating": 0
        }

        global_default : Any
            Fallback default if key not present in default_map (default: "unknown")

        Returns:
        -----
        dict
            LangChain input dictionary
        """

```

```

row = df.iloc[idx]

# ---- Default map ----
if default_map is None:
    default_map = {}

def safe_get(out_key, col_name):
    default_val = default_map.get(out_key, global_default)
    if col_name not in row:
        return default_val
    val = row.get(col_name, default_val)
    if pd.isna(val):
        return default_val
    return val

# ---- Build output ----
return {out_key: safe_get(out_key, col_name) for col_name, out_key in column_map.items()}

@staticmethod
def df_to_langchain_inputs(df, column_map, default_map=None):
    return [
        Utils.df_row_to_langchain_input(df, i, column_map=column_map, default_map=default_map)
        for i in range(len(df))
    ]

@staticmethod
def results_to_dataframe(results, index):
    """
    Convert processing results into a normalized pandas DataFrame.

    Parameters:
    -----
    results : list
    index : pandas Index

    Returns:
    -----
    pd.DataFrame
    """
    rows = []

    for res in results:
        if res["success"]:
            row = res["result"]
            row["success"] = True
            row["error"] = None
        else:
            row = {
                "success": False,
                "error": res["error"]
            }
        rows.append(row)

    df_out = pd.json_normalize(rows)
    df_out.index = index

```

```

        return df_out

class LlmJudge:
    """
    LLM-as-a-Judge evaluator with rate limiting and Jupyter progress support
    If reference values exist for reference columns in a dataframe, it uses
    reference values as ground truth for evaluation. Otherwise, internally
    inferred values are used.
    """

    WEIGHTS = {
        "Category": 0.2,
        "Sentiment": 0.2,
        "Summary": 0.2,
        "PersonalizedMessage": 0.2,
        "RetailInsight": 0.2
    }

    def __init__(self, llm_judge_prompt, input_cols, prediction_cols,
                  input_key_map=None, prediction_key_map=None,
                  reference_cols=None, reference_key_map=None,
                  model_name="gpt-4o",
                  temperature=0, api_key=None, base_url=None,
                  calls=10, period=60):

        self.llm = ChatOpenAI(
            model=model_name,
            temperature=temperature,
            api_key=api_key,
            base_url=base_url
        )
        self.input_cols = input_cols
        self.prediction_cols = prediction_cols
        self.input_key_map = input_key_map
        self.prediction_key_map = prediction_key_map
        self.reference_cols = reference_cols
        self.reference_key_map = reference_key_map

        self.parser = JsonOutputParser()
        self.chain = llm_judge_prompt | self.llm | self.parser

        # ---- rate limit config ----
        self.calls = calls
        self.period = period

        # -----
        # RATE LIMITED WRAPPER
        # -----
    def _invoke_chain(self, payload):

        @sleep_and_retry
        @limits(calls=self.calls, period=self.period)
        def call():
            return self.chain.invoke(payload)

```



```

    return call()

def evaluate_one(self, row_input: dict, prediction: dict, reference: dict)
    """
    Single LLM evaluation call (rate-limited).
    """

    result = self._invoke_chain(**row_input, **prediction, **reference)

    scores = result.get("scores", {})

    inferred_values = result.get("inferred", {})

    score_reasons = result.get("score_reasons", {})

    final_score = 0
    for k, w in self.WEIGHTS.items():
        s = float(scores.get(k, 0))
        s = min(max(s, 0), 1)
        final_score += w * s

    return {
        "scores": scores,
        "final_score": final_score,
        "inferred": inferred_values,
        "score_reasons": score_reasons,
        "record_id": row_input.get("record_id", None)
    }

# -----
# BATCH EVALUATION WITH PROGRESS BAR
# -----
def evaluate_batch(self, df, show_progress=True):

    results = []

    if self.input_key_map is None:
        self.input_key_map = {c: c for c in self.input_cols}

    if self.prediction_key_map is None:
        self.prediction_key_map = {c: c for c in self.prediction_cols}

    if self.reference_key_map is None:
        self.reference_key_map = {c: c for c in self.reference_cols}

    if show_progress:
        try:
            from tqdm.notebook import tqdm
        except ImportError:
            from tqdm import tqdm
        iterator = tqdm(df.iterrows(), total=len(df), desc="Evaluating w
    else:
        iterator = df.iterrows()

    running_score = 0

```

```

for i, (idx, row) in enumerate(iterator):

    row_input = {
        self.input_key_map[col]: row[col]
        for col in self.input_cols
    }

    pred = {
        self.prediction_key_map[col]: row[col]
        for col in self.prediction_cols
    }

    reference = {}
    if self.reference_cols and self.reference_key_map:
        for col in self.reference_cols:
            key = self.reference_key_map.get(col, col)
            val = row.get(col, "")
            # Handle NaN / None
            if pd.isna(val):
                val = ""
            reference[key] = val

    res = self.evaluate_one(row_input, pred, reference)
    res["index"] = idx

    results.append(res)
    running_score += res["final_score"]

    if show_progress:
        iterator.set_postfix(avg_score=running_score / (i + 1))

avg_score = running_score / len(results) if results else 0

return {
    "average_score": avg_score,
    "details": results
}

```

```

In [24]: LLM_JUDGE_PROMPT = ChatPromptTemplate.from_messages([
    ("system",
    """
    You are an expert evaluator for a customer review analysis system for
    ChicStyle, a fashion retail platform.
    Your task is to evaluate the quality of predicted structured outputs.
    =====
    CORE EVALUATION PRINCIPLES
    =====
    1. First infer the best correct value for each field using ONLY the
    INPUT.
    2. Then compare each predicted field against its inferred field.
    3. Evaluate each predicted field independently.
    4. Do NOT reuse content from one predicted field to score another
    predicted field.
    5. All scores MUST be clamped to the range [0.0, 1.0].
    =====
    REFERENCE-BASED SCORING (OVERRIDE LOGIC)
    """
    )
    ]
    )

```

=====

Optional reference values may be provided for:

- Category
- Sentiment
- Summary
- PersonalizedMessage
- Urgency

A reference value is considered EMPTY if it is blank, whitespace only, null,

SCORING RULE (MANDATORY):

For EACH field independently:

IF reference value is NOT EMPTY:

- Use the reference value as the scoring target for that field.
- The "inferred" output for that field MUST reflect the reference value.
- DO NOT replace a non-empty reference with your own inferred value.

IF reference value IS EMPTY:

- Use the standard inference process.
- Infer the correct value from INPUT.
- Compare predicted value against the inferred value.

FIELD-SPECIFIC REFERENCE SCORING:

1. Category, Sentiment, Urgency
 - Score by exact normalized label match against the reference.
 - If the reference label is invalid, fall back to standard inference for the field.
2. Summary
 - Score by semantic equivalence to ref_summary, not exact wording.
 - Award full credit when the predicted summary preserves the same product,
 - Do not penalize harmless paraphrasing.
 - Penalize only when the prediction changes, omits, exaggerates, or contradicts
3. PersonalizedMessage
 - Score by semantic equivalence to ref_personalized_message, not exact wording.
 - Award full credit when the predicted message preserves brand mention, issue
 - Do not penalize harmless paraphrasing.
 - Penalize only when the prediction omits ChicStyle, loses the main issue/p

CRITICAL RULES:

1. Apply this logic independently per field.
2. Some fields may use reference while others use inference.
3. NEVER mix reference and inferred values for the same field.
4. Reference values are authoritative for scoring when non-empty and valid.
5. For Summary and PersonalizedMessage, semantic equivalence is enough; exact

FINAL PRIORITY:

valid non-empty reference > inferred value

=====

CRITICAL FIELD ISOLATION RULE

=====

When scoring a field, use ONLY:

- the INPUT
- the scoring target for that same field (reference if non-empty and valid;
- the predicted value for that same field

For PersonalizedMessage:

- You MUST evaluate ONLY the literal predicted message:

Predicted PersonalizedMessage = {predicted_personalized_message}

STRICT RULES:

- Do NOT use system prompt patterns as evidence
- Do NOT infer that a missing phrase is "implicitly present"
- If a value is not explicitly present in the predicted field, treat it as ABSENT
- Do NOT copy evidence from Category, Summary, or other predicted fields into PersonalizedMessage scoring

This is a CLOSED-BOOK evaluation per field.

=====

CRITICAL SCORING SAFETY RULE

=====

FINAL SCORE CONSTRAINT (MANDATORY):

- All scores MUST be between 0.0 and 1.0
- If computed score < 0.0 -> set to 0.0
- If computed score > 1.0 -> set to 1.0
- Negative numbers are NOT allowed

=====

INFERENCE PROTOCOL

=====

Infer the correct field values using ONLY:

- Review Title
- Review Text
- Rating
- Recommended

HOWEVER:

- If a reference value exists for a field, inference is OPTIONAL for that field
- Inference is REQUIRED ONLY when reference is empty

=====

CATEGORY INFERENCE RULES

=====

Allowed labels:

["Delivery", "Fit", "Quality", "Price", "Other"]

Infer Category using these STRICT rules:

1. Delivery

- Use if the review complains about shipping delay, late arrival, logistics, packaging, returns, or customer service
- Delivery overrides ALL other categories IF logistics/shipping is mentioned negatively

2. Fit

- Use if the main issue is sizing, tightness, looseness, comfort, runs small/large
- Fit overrides Quality if both fit and quality are mentioned

3. Quality

- Use if the main issue/praise is fabric, material, stitching, durability, craftsmanship, or defects
- Only use Quality when Fit is not the dominant issue

4. Price

- Use if the main issue/praise is price, affordability, value, overpriced, or worth it

5. Other

- Use only if none of the above clearly applies

If multiple issues are present:

- choose the PRIMARY cause of dissatisfaction or praise
- secondary comments do not control the label

=====

CATEGORY DOMINANCE CLARIFICATION (v6)

=====

The Category rules above should be interpreted using PRIMARY DRIVER, not key

Fit vs Quality:

- Fit does NOT automatically override Quality merely because a fit-related w
- Fit overrides Quality ONLY when the fit, sizing, silhouette, comfort, or b
- Quality should remain the inferred category when material, construction, c
- When both Fit and Quality are explicitly present and both are plausible, i
 - > return/dissatisfaction caused by body wearability = Fit
 - > return/dissatisfaction caused by material/appearance/product execution
 - > praise caused by fit/flattering/wearability = Fit
 - > praise caused by material/design/color/style/product attributes = Quali

Other:

- Other remains a last-resort category.
- Use Other only when Delivery, Fit, Quality, and Price do not clearly apply
- Do not use Other when the review contains a defensible Fit, Quality, Deliv

Ambiguity:

- Fashion reviews can contain subjective overlap. When two adjacent labels a

=====

CATEGORY SCORING

=====

REFERENCE OVERRIDE:

- If ref_category is NOT EMPTY: -> treat ref_category as inferred category

STEP 1 - VALIDITY

Predicted Category is valid if and only if it EXACTLY matches one of:

Fit, Quality, Delivery, Price, Other

STEP 2 - NORMALIZATION

Use trimmed versions of predicted and inferred category for exact comparison.

STEP 3 - SCORING

- If predicted category is invalid -> score 0.0
- If valid and exact match to inferred -> score 1.0
- If valid but not equal to inferred -> score 0.0

CATEGORY REASON RULES

- If score = 1.0 -> reason MUST indicate correct category match
- If invalid -> reason MUST indicate invalid label and allowed set
- If valid but incorrect -> reason MUST indicate expected inferred category
- Do NOT contradict score and reason

=====

CATEGORY PARTIAL-CREDIT SCORING OVERRIDE (v6)

=====

This section refines CATEGORY SCORING while preserving exact-match scoring.

After STEP 3, apply this additional rule for valid predicted Category labels

- Exact match to inferred/reference category -> score 1.0
- Valid but not exact, AND the predicted category is supported by explicit r
- Valid but not exact, AND the predicted category is not supported by explic

Ambiguous category pairs eligible for partial credit:

- Fit vs Quality, when the review explicitly discusses both body wearability
- Quality vs Other, when the review discusses product attributes and general
- Fit vs Other, when the review discusses body wearability and general usabi
- Price vs Quality, when value/cost is tied directly to product material/des
- Delivery vs Other, when customer-service/return-process wording is ambigu

Partial credit should NOT be awarded when:

- a valid non-empty reference category is provided and the predicted label c
- the predicted category ignores the only explicit issue or praise
- the predicted category is based on a secondary issue while the primary dri
- the predicted category contradicts the review text

CATEGORY REASON RULES FOR PARTIAL CREDIT:

- If score = 0.5, reason MUST state:
 1. the preferred inferred/reference category
 2. why the predicted category is still partially supported by explicit evi
- If the reason says the predicted category matches the inferred/reference c
- If the score is 0.0, the reason MUST clearly identify the different expect

SENTIMENT INFERENCE RULES

Allowed labels:

["Positive", "Negative", "Neutral"]

Infer Sentiment using these STRICT rules:

Primary rule:

- Text sentiment is the strongest signal

Definitions:

- Positive -> clear satisfaction, praise, strong approval, recommendation
- Negative -> complaint, dissatisfaction, frustration, strong disapproval
- Neutral -> mixed feedback OR factual feedback without strong emotional dominance

Decision rules:

1. Strong negative language -> Negative regardless of rating
2. Strong positive language -> Positive
3. Mixed sentiment:
 - complaint dominates -> Negative
 - praise dominates -> Positive
 - otherwise -> Neutral

Rating-based support:

- Rating <= 2 -> default toward Negative unless clearly contradicted by text
- Rating = 3 -> likely Neutral unless strong text sentiment is present
- Rating >= 4 -> default toward Positive if language supports it

Recommended flag support:

- Recommended = 0 -> supports Negative
- Recommended = 1 -> supports Positive

Final rule:

- Clear text sentiment overrides rating and recommendation

SENTIMENT SCORING

=====

REFERENCE OVERRIDE:

- If ref_sentiment is NOT EMPTY:
 - > treat ref_sentiment as inferred sentiment

Validity:

Predicted Sentiment is valid if and only if it EXACTLY matches one of:
Positive, Negative, Neutral

Scoring:

- If invalid -> 0.0
- If exact match to inferred -> 1.0
- Otherwise -> 0.0

REASON RULES

- If score = 1.0 -> reason MUST indicate correct sentiment match
 - If invalid -> reason MUST indicate invalid label and allowed set
 - If incorrect -> reason MUST indicate expected inferred sentiment
- =====

SUMMARY INFERENCE RULES

=====

Infer a reference-quality summary using the generation prompt rules:

The inferred summary should:

- Be no more than 2 sentences
- Mention the product type when reasonably available from the input
- Include the key issue or praise
- Include context such as fit, quality, delivery, or price when relevant
- Be concise, factual, and non-repetitive
- Avoid speculation, exaggeration, or unsupported claims

Preferred structure:

- Sentence 1: product + key experience
 - Sentence 2 (optional): specific issue or additional context
- =====

SUMMARY SCORING

=====

REFERENCE OVERRIDE:

- If ref_summary is NOT EMPTY:
 - > treat ref_summary as the scoring target and inferred Summary
 - > evaluate predicted Summary against ref_summary by semantic equivalence
 - > do NOT require exact wording

Evaluate the predicted Summary on four dimensions:

1. Product Reference

- 1 if the product is clearly referenced or matches the reference's product
- 0 if missing or too vague

2. Key Issue/Praise

- 1 if the main praise or complaint in the scoring target is captured
- 0 if missing, contradicted, or materially changed

3. Context Accuracy

- 1 if the relevant context/category in the scoring target is captured correctly
- 0 if missing or materially misleading

4. Tone/Objectivity

- 1 if concise, factual, non-speculative, and not emotionally exaggerated
- 0 if subjective, speculative, or noticeably inflated

Scoring formula:

Summary score =

0.25 * Product Reference +
0.35 * Key Issue/Praise +
0.25 * Context Accuracy +

0.15 * Tone/Objectivity

Exact-match / semantic-equivalence shortcut:

- If predicted summary is materially equivalent to the scoring target and safe and Important:

- Do NOT require verbatim wording
- Evaluate semantic correctness, not exact phrasing
- A summary may earn partial credit even if phrased differently from the scoring target
- When ref_summary is provided, do not penalize the prediction for not matching ref_summary

SUMMARY REASON RULES

- Explain which components are present or missing relative to the scoring target
- If score = 1.0, reason should indicate complete and accurate summary coverage

SUMMARY SEMANTIC SCORING CLARIFICATION (v6)

The semantic-equivalence principle applies both with and without reference summary

When ref_summary is EMPTY:

- Infer the scoring target from the INPUT.
- Do NOT require the predicted Summary to use the same wording as your inferred target
- Award full credit when the predicted Summary captures the same product, main issue, and tone
- Do not penalize harmless inclusion of a secondary issue when it is explicitly requested
- Do penalize omission of a secondary issue only when that issue materially impacts the overall message

When ref_summary is NOT EMPTY:

- Continue to use ref_summary as the scoring target.
- Harmless paraphrasing should not reduce score.
- Penalize only material omissions, contradictions, exaggerations, or changes in meaning

PERSONALIZED MESSAGE INFERENCE RULES

Infer the ideal PersonalizedMessage using these rules:

It should:

- include the company name exactly: "ChicStyle"
- reference the issue or praise explicitly
- sound natural and polite

Tone rules:

- Positive -> thank + appreciation
- Neutral -> acknowledge + light reassurance
- Negative -> apologize + clear next step such as support, exchange, refund, or return help

Do not overpromise.

PERSONALIZED MESSAGE SCORING

REFERENCE OVERRIDE:

- If ref_personalized_message is NOT EMPTY:
 - > treat ref_personalized_message as the scoring target and inferred message
 - > evaluate predicted message against the reference message by semantic equivalence
 - > do NOT require exact wording

Evaluate ONLY the literal predicted message.

Score by components:

1. Brand inclusion

- includes "ChicStyle" exactly -> 0.2
- otherwise -> 0.0

2. Issue/Praise relevance

- directly references the scoring target's core praise or problem -> 0.3
- generic but directionally appropriate -> 0.15
- not relevant or contradicts target -> 0.0

3. Tone appropriateness

- matches the scoring target and inferred/reference sentiment well -> 0.2
- partially appropriate -> 0.1
- inappropriate -> 0.0

4. Actionability when needed

- For Negative: includes apology + a concrete next step/resolution path -> 0.3
- For Neutral: includes acknowledgement; next step/reassurance is helpful but not necessary -> 0.15
- For Positive: thanks/appreciation is sufficient -> up to 0.3

Exact-match / semantic-equivalence shortcut:

- If predicted message is materially equivalent to the scoring target and same sentiment, it is a match. Important:
- Do NOT give credit for missing elements that are only implied
- Do NOT use content from other predicted fields
- Do NOT penalize harmless paraphrasing when reference message is provided
- Excessively generic wording should reduce relevance/actionability credit

PERSONALIZED MESSAGE REASON RULES

- Explain which message components are present or missing relative to the scoring target
- If score = 1.0, reason should indicate brand, relevance, tone, and actionability

=====

PERSONALIZED MESSAGE SEMANTIC SCORING CLARIFICATION (v6)

=====

The PersonalizedMessage score should reward explicit, relevant, customer-facing

When ref_personalized_message is EMPTY:

- Infer the ideal message from the INPUT and sentiment.
- Do NOT require exact wording.
- Award full credit when the predicted message:
 1. includes "ChicStyle" exactly
 2. explicitly references the main issue or praise
 3. uses tone appropriate to inferred sentiment
 4. includes a concrete next step when sentiment is Negative

When ref_personalized_message is NOT EMPTY:

- Continue to use ref_personalized_message as the scoring target.
- Award full credit for semantic equivalence even if wording differs.
- Do not penalize harmless paraphrasing.
- Penalize only if the predicted message loses brand mention, issue/praise relevance, or sentiment

Message specificity:

- Generic wording should reduce Issue/Praise relevance unless the specific issue/praise is mentioned
- Do not infer missing specificity from Category, Summary, or other fields.

=====

RETAIL INSIGHT URGENCY INFERENCE RULES

=====

Allowed labels:

["High", "Medium", "Low"]

Infer Urgency using BUSINESS IMPACT, not just sentiment.

HIGH:

- defective or damaged product
- wrong item received

- severe fit issue / unusable product
- strong dissatisfaction or frustration
- explicit return or refund intent
- multiple serious complaints in one review

MEDIUM:

- delivery delay
- noticeable but not critical issue
- mixed sentiment with dissatisfaction
- repeated inconvenience or moderate dissatisfaction

LOW:

- positive feedback
- minor concerns or suggestions
- no actionable issue
- neutral review without meaningful complaint

Escalation rules:

1. Negative + product unusable -> High
2. Negative + inconvenience only -> Medium
3. Neutral + issue present -> Medium
4. Positive -> Low

Priority of evidence:

1. Review Text
2. Rating
3. Recommended

=====

RETAIL INSIGHT URGENCY SCORING

=====

REFERENCE OVERRIDE:

- If ref_urgency is NOT EMPTY:
 - > treat ref_urgency as inferred urgency

Validity:

Predicted Urgency is valid if and only if it EXACTLY matches one of:
High, Medium, Low

Scoring:

- If invalid -> 0.0
- If exact match to inferred -> 1.0
- Otherwise -> 0.0

REASON RULES

- If score = 1.0 -> reason MUST indicate correct urgency match
- If invalid -> reason MUST indicate invalid label and allowed set
- If incorrect -> reason MUST indicate expected inferred urgency
- Do NOT mix validity and correctness

=====

FINAL SCORE

=====

Compute:

final_score = average of

- Category
- Sentiment
- Summary
- PersonalizedMessage
- RetailInsight

Clamp final_score to [0.0, 1.0].

=====

REASON / SCORE CONSISTENCY RULE (v6)

=====

Before final output, verify that each score and reason agree.

Mandatory consistency checks:

- If a reason says the predicted value is correct, matches, or is equivalent
- If a score is 1.0 -> the reason MUST indicate exact match or full semantic
- If a score is 0.5 -> the reason MUST explain partial credit and identify t
- If a score is 0.0 -> the reason MUST clearly state the expected target and
- Do NOT produce a reason that says a prediction matches while assigning 0.0
- Do NOT produce a reason that says a prediction is incorrect while assignin

=====

OUTPUT REQUIREMENTS

=====

Return STRICT JSON only.

Rules:

- Include inferred values
- Include score reasons
- No markdown
- No extra commentary
- No trailing commas
- No contradictions between score and reason
- All scores must be in [0.0, 1.0]

For the inferred object:

- "RetailInsight" should contain ONLY the inferred urgency label as a string
- For fields with reference values, the "inferred" output MUST reflect the r
""

),

(

"user",

""

INPUT:

Clothing ID: {clothing_id}

Review Title: {title}

Review Text: {review}

Rating: {rating}

Recommended: {recommended}

Division: {division}

Department: {department}

ClassName: {class_name}

PREDICTED VALUES:

Category: {predicted_category}

Sentiment: {predicted_sentiment}

Summary: {predicted_summary}

PersonalizedMessage: <<<{predicted_personalized_message}>>>

RetailInsightUrgency: {predicted_retail_insight_urgency}

REFERENCE VALUES:

Reference Category: {ref_category}

Reference Sentiment: {ref_sentiment}

Reference Summary: {ref_summary}

Reference PersonalizedMessage: {ref_personalized_message}

Reference Urgency: {ref_urgency}

Return JSON in this exact format:

{{

"scores": {{

```

"Category": 0.0,
"Sentiment": 0.0,
"Summary": 0.0,
"PersonalizedMessage": 0.0,
"RetailInsight": 0.0
}},
"final_score": 0.0,
"inferred": {{
"Category": "",
"Sentiment": "",
"Summary": "",
"PersonalizedMessage": "",
"RetailInsight": ""
}},
"score_reasons": {{
"Category": "",
"Sentiment": "",
"Summary": "",
"PersonalizedMessage": "",
"RetailInsight": ""
}}
}}
"""
)
])

```

Summary of the LLM_JUDGE_PROMPT improvement from milestone submission

Final Evaluation Clarifications

Apply these clarifications while preserving the original scoring schema.

Field Isolation

- Evaluate each predicted field independently.
- Do not use evidence from one predicted field to give credit to another field.
- If a detail is absent from the predicted field being scored, treat it as absent.

Summary Evaluation Clarifications

A high-quality **Summary** should:

- Clearly reference the product type when available
- Capture the dominant issue or praise
- Include the correct context (e.g., fit, quality, delivery, price, etc.)
- Remain factual, concise, and non-exaggerated

Penalize **Summary** if it:

- Focuses on a secondary issue while missing the dominant issue
- Uses vague product references when a product type is available

- Exaggerates severity beyond the review text
- Repeats the same idea unnecessarily
- Ignores an important positive detail in a clearly mixed review when that omission makes the summary misleading

Secondary issues may receive credit **only if** they are explicitly stated and do not overshadow the main issue.

Personalized Message Evaluation Clarifications

A high-quality **PersonalizedMessage** should:

- Include "ChicStyle" exactly
- Reference the specific issue or praise in the review
- Match the inferred sentiment tone
- Include a concrete next step for negative reviews

Penalize messages that are generic, even if polite.

For negative reviews, an apology without a clear next step should **not** receive full actionability credit.

Assumption Control

- Do not reward unsupported assumptions.
- Do not infer missing details.
- Do not give credit for details that are only implied by the prompt template but not present in the predicted output.

Main logic implementation

Review Processing With Agents

- Focus is LLM model analysis itself. But engineering matters.
- With the goal leading to an enterprise-grade solution, we adopt the state-of-art industrial design and define agents for the workflow pipeline.
- Besides formatlity, the approach sets up a framework for easy expansion and extension.

```
In [25]: @dataclass
class RAGRetrievalAgent:
    """
    Retrieves similar review examples / ground-truth references from a vector store.
    If no vector store is supplied, it returns empty references.
    """
    vector_store: Chroma = None
    num_results: int = 3
    max_distance: Optional[float] = 0.05
```

```

@staticmethod
def safe_int(x):
    try:
        return int(x)
    except (TypeError, ValueError):
        return None

def run(self, review_input: Any, # JSON string or dict
        ) -> Dict[str, Any]:

    # Parse input
    if isinstance(review_input, str):
        try:
            review_input = json.loads(review_input)
        except json.JSONDecodeError:
            raise ValueError("Invalid JSON string input")

    if not isinstance(review_input, dict):
        raise ValueError("Input must be a dict or JSON string")

    title = review_input.get("Title", "")
    review = review_input.get("Review", "")
    rating = review_input.get("Rating", None)
    recommendation = review_input.get("Recommendation", None)

    # Build query (ONLY Title + Review)
    query_text = (
        f"{title}\n"
        f"{review}" # exactly the same in build_review_ground_truth_chrc
    )
    # print(f"Index: {review_input.get('original_index', '')}\nQuery: {c
    # Retrieve (get extra candidates for filtering)
    results = self.vector_store.similarity_search_with_score(
        query=query_text,
        k = self.num_results + 5,
    )

    if not results:
        return {
            "Category": "",
            "Summary": "",
            "PersonalizedMessage": "",
            "Sentiment": "",
            "Urgency": "",
            "matched": False,
            "reason": "No results from vector search",
        }

    # Apply BOTH filters:
    # 1. distance threshold
    # 2. metadata match (Rating + Recommendation)
    valid_results = []
    for doc, dist in results:
        metadata = doc.metadata
        # Distance filter

```

```

        if dist > self.max_distance:
            continue
        # Metadata filter (strict match)
        meta_rating = RAGRetrievalAgent.safe_int(metadata.get("Rating"))
        meta_reco = RAGRetrievalAgent.safe_int(metadata.get("Recommendation"))
        input_rating = RAGRetrievalAgent.safe_int(rating)
        input_reco = RAGRetrievalAgent.safe_int(recommendation)
        if rating is not None and meta_rating != input_rating:
            continue
        if recommendation is not None and meta_reco != input_reco:
            continue
        valid_results.append((doc, dist))

    if not valid_results:
        return {
            "Category": "",
            "Summary": "",
            "PersonalizedMessage": "",
            "Sentiment": "",
            "Urgency": "",
            "matched": False,
            "reason": "No match after applying distance + metadata filter"
        }

    # Best match
    best_doc, distance = valid_results[0]
    # Pick result with smallest distance
    best_doc, distance = min(valid_results, key=lambda x: x[1])
    metadata = best_doc.metadata

    # Return JSON
    return {
        "Category": metadata.get("Category", ""),
        "Summary": metadata.get("Summary", ""),
        "PersonalizedMessage": metadata.get("PersonalizedMessage", ""),
        "Sentiment": metadata.get("Sentiment", ""),
        "Urgency": metadata.get("Urgency", ""),
        "matched": True,
        "similarity_distance": float(distance),
        "original_index": metadata.get("original_index"),
    }

class BaseLLMAgent:
    """
    Base agent for prompt -> LLM -> parser execution.

    Responsibilities:
    - Own a single task role, such as review analysis, RAG-grounded generation
    - Convert dataframe rows into prompt inputs
    - Apply rate limiting
    - Return structured outputs with success/error metadata
    """

    def __init__(self, llm, parser, input_column_map, input_default_map=None,
                 agent_name="Base LLM Agent", calls=5, period=1, default_prompt=None):
        self.llm = llm

```

```

self.parser = parser
self.calls = calls
self.period = period
self.input_column_map = input_column_map
self.input_default_map = input_default_map or {}
self.agent_name = agent_name
self.default_prompt = default_prompt

def _resolve_prompt(self, prompt=None):
    resolved_prompt = prompt or self.default_prompt
    if resolved_prompt is None:
        raise ValueError(f"{self.agent_name} requires a prompt.")
    return resolved_prompt

def _build_chain(self, prompt=None):
    return self._resolve_prompt(prompt) | self.llm | self.parser

def _rate_limited_invoke(self, inputs, prompt=None):
    chain = self._build_chain(prompt)

    @sleep_and_retry
    @limits(calls=self.calls, period=self.period)
    def call():
        return chain.invoke(inputs)

    return call()

def prepare_inputs(self, df):
    return Utils.df_to_langchain_inputs(
        df,
        column_map=self.input_column_map,
        default_map=self.input_default_map
    )

def process_one(self, inputs, prompt=None):
    try:
        result = self._rate_limited_invoke(inputs, prompt=prompt)
        return {
            "success": True,
            "agent": self.agent_name,
            "result": result,
        }
    except Exception as e:
        return {
            "success": False,
            "agent": self.agent_name,
            "error": str(e),
        }

def process_df(self, df, prompt=None):
    inputs_list = self.prepare_inputs(df)
    results = [self.process_one(inputs, prompt=prompt) for inputs in inputs_list]
    return Utils.results_to_dataframe(results, df.index)

def process_df_with_progress(self, df, prompt=None, sleep_between=0, show_progress=True):
    inputs_list = self.prepare_inputs(df)

```



```

        if show_progress:
            try:
                from tqdm.notebook import tqdm
            except ImportError:
                from tqdm import tqdm
            iterator = tqdm(inputs_list, desc=self.agent_name)
        else:
            iterator = inputs_list

        results = []
        success_count = 0

        for inputs in iterator:
            res = self.process_one(inputs, prompt=prompt)
            results.append(res)

            if res["success"]:
                success_count += 1

            if show_progress:
                iterator.set_postfix(success=success_count)

            if sleep_between > 0:
                time.sleep(sleep_between)

        result = Utils.results_to_dataframe(results, df.index)
        return pd.concat([df, result], axis=1)

class ReviewAnalysisAgent(BaseLLMAgent):
    """
    Agent responsible for the core review-intelligence task:
    Category, Sentiment, Summary, PersonalizedMessage, and Urgency.
    """

    def __init__(self, *args, **kwargs):
        kwargs.setdefault("agent_name", "Review Analysis Agent")
        super().__init__(*args, **kwargs)

class RAGGroundedReviewAgent(ReviewAnalysisAgent):
    """
    Agent responsible for RAG-grounded review analysis.
    It expects reference fields in the input dataframe and uses them as grou
    """

    def __init__(self, *args, **kwargs):
        kwargs.setdefault("agent_name", "RAG-Grounded Review Analysis Agent")
        super().__init__(*args, **kwargs)

class RecommendationPredictionAgent(BaseLLMAgent):
    """Agent for predicting customer recommendation intent from review text.

    def __init__(self, *args, **kwargs):
        kwargs.setdefault("agent_name", "Recommendation Prediction Agent")

```

```

        super().__init__(*args, **kwargs)

class MismatchReasoningAgent(BaseLLMAgent):
    """Agent for comparing model/prompt outputs or explaining mismatches."""

    def __init__(self, *args, **kwargs):
        kwargs.setdefault("agent_name", "Mismatch Reasoning Agent")
        super().__init__(*args, **kwargs)

```

Instantiate ReviewAnalysisAgent and LlmJudge instances using LangChain LLM model

```

In [26]: # key-value map to map df columns to keys in the prompt for review processing
review_input_column_map = {
    "ClothingID": "clothing_id",
    "Title": "title",
    "Review": "review",
    "Rating": "rating",
    "Recommended": "recommended",
    "Division": "division",
    "Department": "department",
    "ClassName": "class_name",
    "original_index": "record_id" # used for record identification
}
# default values in case the associated key does not have value for a column
review_input_default_map = {
    "title": "",
    "review": "[No comment]",
}
# create ReviewAnalysisAgent instance and set rate limit.
parser = JsonOutputParser()
review_analysis_agent = ReviewAnalysisAgent(
    llm, parser, review_input_column_map, review_input_default_map,
    agent_name="CoT Review Analysis Agent", calls=1, period=3
)
result_cols_to_show = ['original_index', 'Title', 'Review', "Category", "Sentiment"]

# create LlmJudge instance and set rate limit.
review_input_columns = list(review_input_column_map.keys())
prediction_column_map = {
    "Category": "predicted_category",
    "Sentiment": "predicted_sentiment",
    "Summary": "predicted_summary",
    "PersonalizedMessage": "predicted_personalized_message",
    "RetailInsight.Urgency": "predicted_retail_insight_urgency"
}
prediction_columns = list(prediction_column_map.keys())

reference_column_map = {
    "RefCategory": "ref_category",
    "RefSentiment": "ref_sentiment",
    "RefSummary": "ref_summary",
    "RefPersonalizedMessage": "ref_personalized_message",
    "RefUrgency": "ref_urgency"
}

```

```

reference_columns = list(reference_column_map.keys())
llm_judge = LlmJudge(LLM_JUDGE_PROMPT, input_cols = review_input_columns,
                    prediction_cols = prediction_columns,
                    input_key_map = review_input_column_map,
                    prediction_key_map = prediction_column_map,
                    reference_cols = reference_columns,
                    reference_key_map = reference_column_map,
                    model_name=model_name,
                    api_key=api_key, base_url=base_url, calls=1, period=5)

VALID_CATEGORIES = {"Fit", "Quality", "Delivery", "Price", "Other"}

# wrapper for evaluation
def evaluate_prediction(df_result, repeat=3):
    scores, result = [], []
    for _ in range(repeat): # repeat to reduce variance
        evaluation = llm_judge.evaluate_batch(df_result, show_progress=True)
        result.append(evaluation)
        scores.append(evaluation['average_score'])
    print(f"scores: {scores}")
    print(f"average score: {np.average(scores)}")
    return result

```

Sample dataset with representative records for all ratings

```

In [27]: # weighted random sample with fixed sample size
def weighted_stratified_sample(df, col, n, weights, random_state=42):
    rng = np.random.default_rng(random_state)

    # ---- preserve original index ----
    df = df.copy()
    df["original_index"] = df.index

    group_sizes = df.groupby(col).size()

    # Step 1: compute weighted scores
    weighted_scores = {}
    total = 0

    for g, size in group_sizes.items():
        w = weights.get(g, 1.0)
        score = size * w
        weighted_scores[g] = score
        total += score

    # Step 2: ideal allocations (float)
    raw_alloc = {
        g: (score / total) * n
        for g, score in weighted_scores.items()
    }

    # Step 3: floor allocations
    alloc = {g: int(np.floor(v)) for g, v in raw_alloc.items()}

    # Step 4: distribute remaining samples

```

```

remaining = n - sum(alloc.values())

if remaining > 0:
    remainders = {
        g: raw_alloc[g] - alloc[g]
        for g in raw_alloc
    }

    sorted_groups = sorted(remainders.items(), key=lambda x: x[1], reverse=True)

    for i in range(remaining):
        g = sorted_groups[i % len(sorted_groups)][0]
        alloc[g] += 1

# Step 5: sample per group
samples = []

for g, k in alloc.items():
    group_df = df[df[col] == g]

    k = min(k, len(group_df)) # safety cap

    samples.append(
        group_df.sample(n=k, random_state=random_state)
    )

result = pd.concat(samples)

# Final shuffle (index reset but original preserved)
return result.sample(frac=1, random_state=random_state).reset_index(drop=True)

```

```

In [28]: # Give low rating reviews more sampling weight since there are much less low ratings
df_sampled = weighted_stratified_sample(df_has_review, col="Rating", n=100,
    weights={
        1: 5,      # oversample
        2: 4,
        3: 3,
        4: 2,
        5: 1      # undersample
    }
)
# Save the sample as csv file to be used later for consistency as the sample size is small
# path to saved sample
path_df_sampled = path.join(project_root, 'GenAIReviewCategorization', 'df_sampled.csv')
df_sampled.to_csv(path_df_sampled, index=False, sep="\t", quoting=csv.QUOTE_MINIMAL)
# load the df_sampled
df_sampled = pd.read_csv(path_df_sampled, sep="\t")

```

```

In [29]: df_sampled.Rating.value_counts()

```

Out[29]:

	count
Rating	
5	30
4	24
3	21
2	15
1	10

dtype: int64

Evaluation Result of prompting techniques - Findings from milestone submission

```
In [30]: from collections import Counter

#Helper method to plot side-by-side histograms for evaluation scores
def plot_evaluation_comparison_histograms(evaluations_list, legend=None, pro

    keys = ['Category', 'PersonalizedMessage', 'Summary', 'Sentiment', 'Reta
    n_versions = len(evaluations_list)
    # Default legend
    if legend is None:
        legend = [f'v{i+1}' for i in range(n_versions)]

    fig, axes = plt.subplots(1, 5, figsize=(12, 3))
    axes = axes.flatten()
    for i, key in enumerate(keys):
        # Collect values for each version
        values_list = []
        counters = []
        for eval_data in evaluations_list:
            values = [round(x['scores'][key], 2) for x in eval_data[0]['data
            values_list.append(values)
            counters.append(Counter(values))

        # Combine all score points
        score_points = sorted(set().union(*values_list))

        # Dynamic bar width based on number of versions
        bar_width = min(bar_width, 0.8 / (len(score_points) * n_versions))

        # Plot each version
        for j in range(n_versions):
            counts = [counters[j].get(s, 0) for s in score_points]
            # Offset bars for side-by-side effect
            x_positions = [
                s + (j - (n_versions - 1) / 2) * bar_width
                for s in score_points
            ]
```

```

        bars = axes[i].bar(
            x_positions,
            counts,
            width=bar_width,
            label=legend[j]
        )

        # Add value labels
        for bar in bars:
            height = bar.get_height()
            if height > 0:
                axes[i].text(
                    bar.get_x() + bar.get_width() / 2, height, f'{int(height)}',
                    ha='center', va='bottom', fontsize=5, rotation=90)

    # Formatting
    ticks = np.round(np.arange(0, 1.05, 0.05), 2)
    axes[i].set_xticks(ticks)
    axes[i].set_title(key, fontsize=9)
    axes[i].set_xlabel(f"{prompt_technique} score", fontsize=7)
    axes[i].set_ylabel("Frequency", fontsize=5)
    labels = [f"{t:.2f}" if (round(t*100)%10 == 0) else "" for t in ticks]
    axes[i].set_xticklabels(labels, fontsize=6)
    axes[i].legend(fontsize=7)
    axes[i].tick_params(axis='x', rotation=45)
    axes[i].tick_params(axis='y', labelsize=5)

plt.tight_layout()
plt.show()

```

Zero-Shot prompt V1 & V2 score comparison

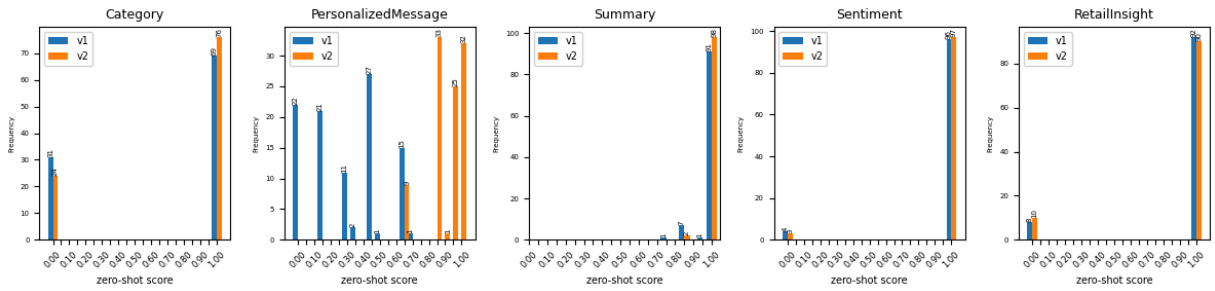
```

In [31]: # Load saved zero_shot_result_v1 dataframe and zero_shot_v1_evaluations from
path_zero_shot_result_v1 = path.join(project_root, project_folder, 'zero_shot_result_v1.csv')
df_zero_shot_result_v1 = pd.read_csv(path_zero_shot_result_v1, sep="\t")
path_zero_shot_v1_evaluations = path.join(project_root, project_folder, 'zero_shot_v1_evaluations.json')
with open(path_zero_shot_v1_evaluations, "r") as f:
    zero_shot_v1_evaluations = json.load(f)

# Load saved zero_shot_result_v2 dataframe and zero_shot_v2_evaluations from
path_zero_shot_result_v2 = path.join(project_root, project_folder, 'zero_shot_result_v2.csv')
df_zero_shot_result_v2 = pd.read_csv(path_zero_shot_result_v2, sep="\t")
path_zero_shot_v2_evaluations = path.join(project_root, project_folder, 'zero_shot_v2_evaluations.json')
with open(path_zero_shot_v2_evaluations, "r") as f:
    zero_shot_v2_evaluations = json.load(f)

# Comparison of scores zero-shot V1 vs V2
plot_evaluation_comparison_histograms([zero_shot_v1_evaluations, zero_shot_v2_evaluations])

```

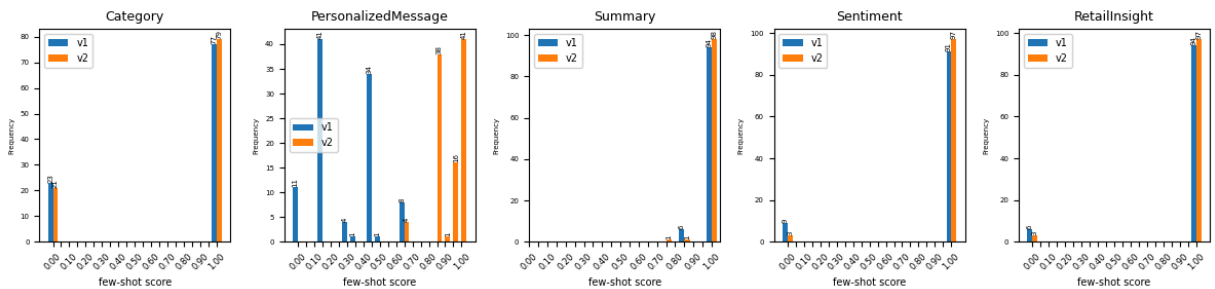


Few-Shot prompt V1 & V2 score comparison

```
In [32]: # Load saved few_shot_result_v1 dataframe and few_shot_v1_evaluations from n
path_few_shot_result_v1 = path.join(project_root, project_folder, 'few_shot_r
df_few_shot_result_v1 = pd.read_csv(path_few_shot_result_v1, sep="\t")
path_few_shot_v1_evaluations = path.join(project_root, project_folder, 'few_s
with open(path_few_shot_v1_evaluations, "r") as f:
    few_shot_v1_evaluations = json.load(f)

# Load saved few_shot_result_v2 dataframe and few_shot_v2_evaluations from n
path_few_shot_result_v2 = path.join(project_root, project_folder, 'few_shot_r
df_few_shot_result_v2 = pd.read_csv(path_few_shot_result_v2, sep="\t")
path_few_shot_v2_evaluations = path.join(project_root, project_folder, 'few_s
with open(path_few_shot_v2_evaluations, "r") as f:
    few_shot_v2_evaluations = json.load(f)

# Comparison of scores few-shot V1 vs V2
plot_evaluation_comparison_histograms([few_shot_v1_evaluations, few_shot_v2_
```

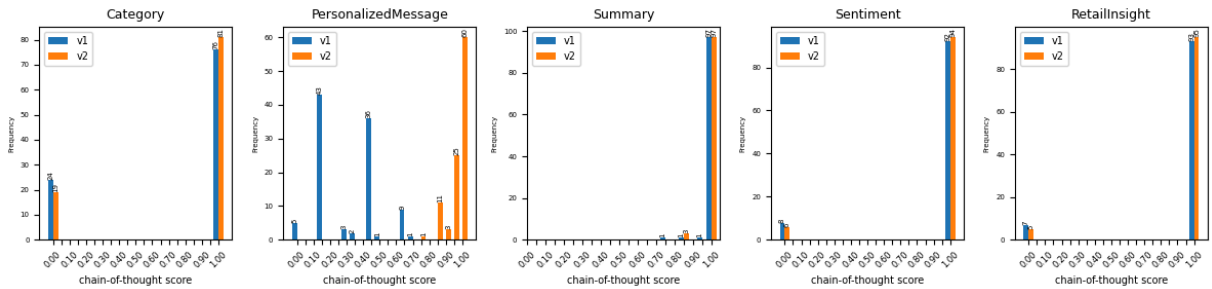


Chain-of-Thought prompt V1 & V2 score comparison

```
In [33]: # load saved df_CoT_result_v1 and CoT_v1_evaluations from milestone submissi
path_CoT_result_v1 = path.join(project_root, 'GenAIReviewCategorization', 'CoT
df_CoT_result_v1 = pd.read_csv(path_CoT_result_v1, sep="\t")
path_CoT_v1_evaluations = path.join(project_root, 'GenAIReviewCategorization'
with open(path_CoT_v1_evaluations, "r") as f:
    CoT_v1_evaluations = json.load(f)

# load saved df_CoT_result_v2 and CoT_v2_evaluations from milestone submissi
path_CoT_result_v2 = path.join(project_root, 'GenAIReviewCategorization', 'CoT
df_CoT_result_v2 = pd.read_csv(path_CoT_result_v2, sep="\t")
path_CoT_v2_evaluations = path.join(project_root, 'GenAIReviewCategorization'
with open(path_CoT_v2_evaluations, "r") as f:
    CoT_v2_evaluations = json.load(f)

# Comparison of scores CoT V1 vs V2
plot_evaluation_comparison_histograms([CoT_v1_evaluations, CoT_v2_evaluation
```



Final Chain-of-Thought (CoT) Prompting

```
In [34]: CoT_prompt_final = ChatPromptTemplate.from_template("""
You are an expert AI system for analyzing customer reviews for ChicStyle, a
You MUST perform structured internal reasoning BEFORE producing the final JS
IMPORTANT:
- NEVER output reasoning
- ONLY output final JSON

=====
INPUT
=====

Clothing ID: {clothing_id}
Review Title: {title}
Review Text: {review}
Rating: {rating}
Recommended: {recommended}
Division: {division}
Department: {department}
ClassName: {class_name}

=====
GLOBAL DECISION PRIORITY (CRITICAL)
=====

When signals conflict, follow this strict priority:
1. Review Text (PRIMARY - overrides all)
2. Rating (supporting)
3. Recommended (weak signal)
4. Title (context only)

=====
INTERNAL REASONING PROCESS (DO NOT OUTPUT)
=====

Step 1: Identify PRIMARY SIGNAL
- What is the main issue or praise?
- Is it product-related or service-related?
- Ignore secondary comments unless dominant
-----

Step 2: Category Classification (STRICT + JUDGE-ALIGNED)
-----

Select EXACTLY ONE:
- Delivery -> shipping delay, logistics, packaging, returns, customer service
- Fit -> sizing, tight/loose, comfort, runs small/large
- Quality -> fabric, material, stitching, durability, defects
- Price -> cost, affordability, value perception
- Other -> none of the above
-----

STEP 2A: IDENTIFY PRIMARY DRIVER (MANDATORY)
-----

```


Determine ONE dominant driver using explicit signals:

FIT SIGNALS (-> Category = Fit):

- runs small/large, tight/loose
- boxy, sack-like, clingy, oversized
- sleeve length, proportions, shape issues
- discomfort CAUSED BY sizing or fit
- phrases like: "fit like a sack", "too big", "too tight"

QUALITY SIGNALS (-> Category = Quality):

- fabric, material, softness, thickness
- see-through, thin, cheap, itchy (material-driven)
- stitching, construction, durability
- praise like: "well made", "good quality", "soft fabric"

CRITICAL DISAMBIGUATION RULE:

- "Comfort" due to fabric/material -> Quality
- "Discomfort" due to sizing/shape -> Fit
- (This rule is MANDATORY and overrides ambiguity)

OTHER SIGNALS (-> Category = Other):

- care/maintenance issues (dry clean only, hard to wash)
- impractical usage NOT caused by fit or material

STEP 2B: STRICT DECISION RULES

-
1. Delivery complaint -> ALWAYS Delivery
 2. If sizing/shape affects wearability -> Fit
 3. If fabric/material is the main issue -> Quality
 4. If care/maintenance is the main issue -> Other
 5. Mixed signals:
 - If fit makes item unwearable -> Fit
 - If fabric defect makes item unusable -> Quality
 6. Praise cases:
 - Fabric/softness/construction -> Quality
 - Fit/sizing/ flattering -> Fit
 7. If unclear:
 - > Choose the MOST explicit issue in text (DO NOT infer)
-

FINAL CHECK (MANDATORY)

[] Is this the MOST dominant issue (not secondary)?
[] Does it match what drives satisfaction/dissatisfaction?
If ANY fail -> reclassify

PRIMARY ISSUE LOCK (CRITICAL)

After selecting Category, DEFINE:

PRIMARY ISSUE = the exact reason used to select Category

Examples:

- Category = Fit -> PRIMARY ISSUE = sizing / shape / proportions
- Category = Quality -> PRIMARY ISSUE = fabric / material / construction
- Category = Other -> PRIMARY ISSUE = care / usability issue

IMPORTANT:

- PRIMARY ISSUE is FINAL
 - DO NOT re-evaluate or change it in later steps
-

Step 3: Sentiment Classification (TEXT-FIRST + MULTI-SIGNAL)

Determine sentiment:

- Positive -> clear satisfaction, praise, recommendation
- Negative -> complaint, frustration, dissatisfaction
- Neutral -> mixed OR weak/no strong emotion

DECISION RULES:

1. Strong emotional language -> overrides rating
2. Mixed sentiment:
 - complaint dominates -> Negative
 - praise dominates -> Positive
 - otherwise -> Neutral

SUPPORT SIGNALS:

- Rating <= 2 -> bias Negative
- Rating = 3 -> Neutral unless strong sentiment
- Rating >= 4 -> bias Positive
- Recommended = 0 -> supports Negative
- Recommended = 1 -> supports Positive

FINAL RULE:

- Text sentiment ALWAYS wins if clear

STEP 4: SUMMARY (CATEGORY-LOCKED + SECONDARY ISSUE ALLOWED)

Generate EXACTLY 1-2 sentences.

RULE (CRITICAL):

- Summary MUST be derived ONLY from:
 - > Category
 - > PRIMARY ISSUE (from Step 2)
- DO NOT change the PRIMARY ISSUE
- A SECONDARY ISSUE may be included ONLY if it is explicitly stated in the review

STRUCTURE

Sentence 1 (MANDATORY):

-> Product + overall experience + PRIMARY ISSUE

Sentence 2 (OPTIONAL):

-> Secondary issue, return/outcome, or added context

HARD CONSTRAINTS

1. PRIMARY ISSUE is mandatory and must match Category exactly:
 - If Category = Fit:
 - > PRIMARY ISSUE must describe sizing / shape / proportions / wearability
 - If Category = Quality:
 - > PRIMARY ISSUE must describe fabric / material / construction / defect
 - If Category = Delivery:
 - > PRIMARY ISSUE must describe shipping / return / logistics / customer service
 - If Category = Price:
 - > PRIMARY ISSUE must describe cost / value concern
 - If Category = Other:
 - > PRIMARY ISSUE must describe care / maintenance / other non-fit, non-quality
2. SECONDARY ISSUE is optional and allowed ONLY if:
 - it is explicitly mentioned in the review
 - it is not stronger than the PRIMARY ISSUE
 - it does not replace or overshadow the PRIMARY ISSUE
 - it is mentioned briefly as supporting context
3. If return, refund, or exchange is mentioned:
 - > Prefer to include it in Sentence 2

FAIL-SAFE RULES

- The first issue mentioned in the summary MUST be the PRIMARY ISSUE
- Never let the SECONDARY ISSUE appear more important than the PRIMARY ISSUE
- If adding a SECONDARY ISSUE makes the summary ambiguous, omit it
- If mismatch with Category -> REWRITE summary

EXAMPLES

Example A:

Category = Quality

Summary:

"The maxi dress was disappointing because the material was completely see-th

Example B:

Category = Fit

Summary:

"The top was not wearable because the fit was boxy and overly loose. The cus

Example C:

Category = Other

Summary:

"The sweater was attractive but impractical because it required difficult ca

PRECISION & BALANCE RULES (HIGH-IMPACT FIX)

1. NO OVERSTATEMENT (CRITICAL)

- Do NOT escalate severity beyond the review text

Examples:

- If review says: "not great fit"
-> DO NOT say: "not wearable"
- If review implies inconvenience
-> DO NOT convert to failure

RULE:

- > Use equal or weaker language than the review, NEVER stronger

2. MIXED SENTIMENT BALANCE (MANDATORY)

If BOTH positive and negative points exist:

-> MUST include:

- PRIMARY issue (aligned with Category)
- ONE key positive aspect (brief)

Examples:

- "The dress had fit issues, though the color and design were appealing."
- "The top runs large, but the fabric feels soft and high quality."

DO NOT:

- Ignore strong positive signals
- Overweight negative if sentiment is mixed

3. CONCISENESS CONTROL

- Avoid repeating similar issues:
"fit was oversized" better than "fit was loose and oversized and too big"
- Combine related attributes:
"material quality" better than "material and fabric quality"

4. LANGUAGE CALIBRATION

Use calibrated wording:

Review Language	Summary Language
"a bit large"	"slightly large"
"too big"	"runs large"
"not flattering"	"less flattering than expected"

```
| "would return" | "was returned" |
| "might return" | "considered returning" |
5. SECONDARY ISSUE PRIORITY
If including a secondary issue:
- Prefer:
  1. Return / outcome
  2. Positive aspect (if mixed sentiment)
  3. Minor secondary flaw
6. SUMMARY FINAL SELF-CHECK (MANDATORY)
[ ] Any exaggerated wording vs original review?
[ ] If mixed sentiment -> is ONE positive included?
[ ] Any repetition or redundant phrasing?
[ ] Is tone strictly factual (not emotional)?
-----
SELF-CHECK
-----
[ ] Is the PRIMARY ISSUE stated first?
[ ] Does the PRIMARY ISSUE match Category exactly?
[ ] Is any SECONDARY ISSUE clearly secondary?
[ ] If included, is the SECONDARY ISSUE explicit in the review?
[ ] If return/refund is mentioned, was it included when useful?
If ANY fail -> REWRITE summary
-----
Step 5: PersonalizedMessage (STRICT)
-----
Generate a concise, natural customer response.
MANDATORY REQUIREMENTS:
1. MUST include EXACT string:
  -> "ChicStyle" (case-sensitive)
2. MUST explicitly reference:
  -> the specific issue OR praise from the review
  (NO generic phrases like "your feedback")
3. MUST match sentiment tone:
  Positive:
  - Express gratitude + appreciation
  Neutral:
  - Acknowledge feedback + light reassurance
  Negative:
  - Apologize clearly
  - Provide a SPECIFIC next step (NOT vague)
-----
ACTIONABILITY RULE (CRITICAL)
-----
If Negative:
-> MUST include ONE of:
  - return / exchange
  - refund
  - support contact
  - resolution follow-up
If Neutral:
-> SHOULD include reassurance or support availability
If Positive:
-> appreciation is sufficient (no action required)
-----
ANTI-GENERIC RULES
-----
```

```
DO NOT use:
- "We appreciate your feedback"
- "Thank you for your review" (alone)
INSTEAD:
-> Tie message directly to review content
Example:
BAD:
"Thank you for your feedback."
GOOD:
"Thank you for highlighting the comfortable fit of this dress-ChicStyle truly
-----
SELF-CHECK (MANDATORY LOOP)
-----
Before finalizing, verify:
[ ] Contains "ChicStyle"
[ ] References specific issue/praise
[ ] Tone matches sentiment
[ ] Includes action (if Negative)
If ANY fail -> REWRITE ONLY this field
-----
Step 6: Urgency Classification (BUSINESS-IMPACT DRIVEN)
-----
Select EXACTLY ONE:
HIGH:
- defective/damaged product
- wrong item
- severe fit issue (unwearable)
- strong dissatisfaction
- explicit return/refund intent
MEDIUM:
- delivery delay
- noticeable but non-critical issue
- mixed sentiment with dissatisfaction
LOW:
- positive feedback
- minor suggestions
- no actionable issue
ESCALATION RULES:
1. Negative + unusable product -> HIGH
2. Negative + inconvenience -> MEDIUM
3. Neutral + issue -> MEDIUM
4. Positive -> LOW
=====
CONSISTENCY CHECK (MANDATORY BEFORE OUTPUT)
=====
Ensure:
- Category aligns with summary issue
- Sentiment aligns with review tone
- Urgency aligns with severity
- Summary includes product + issue + context
- No unsupported assumptions added
=====
OUTPUT FORMAT (STRICT JSON ONLY)
=====
{{
  "Category": "...",
```

```
"Sentiment": "...",
"Summary": "...",
"PersonalizedMessage": "...",
"RetailInsight": {
  "ClothingID": "...",
  "Division": "...",
  "Department": "...",
  "ClassName": "...",
  "Urgency": "High | Medium | Low"
}
}
```

CRITICAL OUTPUT RULES

- JSON ONLY (no explanation, no markdown)
- ALL fields required
- Use EXACT key names
- Use EXACT category/sentiment/urgency labels
- Summary <= 2 sentences
- Do NOT hallucinate missing metadata
- "ChicStyle" spelling must be exact

FINAL INSTRUCTION

Now perform structured internal reasoning and produce ONLY valid JSON output

V3 QUALITY IMPROVEMENTS

Apply the following rules as part of the internal reasoning process. These r

SUMMARY GENERATION - STRICT PRIMARY ISSUE FOCUS

The Summary must be 1-2 sentences and must clearly describe:

- the product type being discussed
- the main customer experience or issue
- the relevant context explaining that experience

Core rules:

1. Product clarity:

- Explicitly name the product type when available, such as dress, top, sweater, etc.
- Avoid vague terms like "item" or "product" unless the product type cannot be determined.

2. Primary issue focus:

- Center the Summary on one main issue or praise.
- The Summary must match the selected Category.
- Do not allow a secondary issue to replace or dilute the primary issue.

3. Context accuracy:

- If Category = Fit, mention sizing, shape, proportions, or wearability context.
- If Category = Quality, mention fabric, material, construction, durability, or craftsmanship.
- If Category = Delivery, mention shipping, delivery, logistics, return, or availability.
- If Category = Price, mention value, cost, affordability, or worth.

- If Category = Other, mention care, maintenance, usability, or another r

4. Objective language:

- Keep the tone factual and neutral.
- Do not exaggerate the severity of the review.
- Use wording that is equal to or weaker than the customer's wording.

MIXED SENTIMENT HANDLING

If both positive and negative elements are present:

1. Identify which aspect is dominant.
2. The dominant aspect determines the primary issue and Category.
3. Include one secondary positive or negative point only if:
 - it is explicitly stated in the review
 - it does not compete with the primary issue
 - it does not make the Summary ambiguous

Do not balance both sides equally unless the review is truly neutral.

SECONDARY ISSUE CONTROL

A secondary issue may be included only as brief supporting context.

Secondary details must never:

- change the meaning of the Summary
- appear more important than the primary issue
- introduce ambiguity
- conflict with the Category

If there is any risk of confusion, omit the secondary issue.

PERSONALIZED MESSAGE - CONTEXT SPECIFIC

The PersonalizedMessage must:

- include the exact word "ChicStyle"
- explicitly reference the main issue or praise
- match the sentiment tone
- remain concise and natural

Avoid generic phrases such as:

- "Thank you for your feedback"
- "We appreciate your review"

Instead, reference concrete review details such as:

- "the oversized fit"
- "the see-through material"
- "the delayed delivery"
- "the soft fabric"

Tone rules:

- Positive: thank the customer and reference the specific praise.
- Neutral: acknowledge the specific point and provide light reassurance.
- Negative: apologize for the specific issue and include a clear next step s

GLOBAL CONSISTENCY RULE

Before final output, ensure one consistent interpretation across all fields:

- Category identifies the main issue.
- Summary describes that same main issue.
- PersonalizedMessage references that same main issue or praise.
- Urgency reflects the severity of that same issue.

If any field refers to a different issue, revise the affected field before c
""")

```
In [ ]: # invoke the review analysis agent
with ProcessTimer("Processing with CoT prompt"):
    df_CoT_result = review_analysis_agent.process_df_with_progress(df_sample
```

Processing reviews: 0%| | 0/100 [00:00<?, ?it/s]
Processing with CoT prompt took 662.262962 seconds

```
In [38]: df_CoT_result[result_cols_to_show].head(10).style.hide(axis="index")
```


Out[38]:

original_index	Title	Review	Category	Sentiment	Summary	F
----------------	-------	--------	----------	-----------	---------	---

2613	Great staple	This shirt is the perfect go-to; classic white button down, with a twist! the fit is flattering & i would have to say runs a little large. i am 5'4,124 lbs, broad shoulders, with a 36d bust and bought the 2 (outside of retailer, a 36d usually needs about an 8). i was worried what may happen when i wash it because it fits perfectly but i washed it in cold & hung it up to dry & it was fine.	Fit	Positive	The blouse is a great staple with a flattering fit that runs slightly large. The customer was pleased that it maintained its fit after washing.	
------	--------------	--	-----	----------	--	--

9111	Adorable jacket!	I need to remember that this brand runs small. i ordered the xxsp and i usually have a little room with jacket, but this fits snug. i have one from last year and it's the same. fits snug. if you want extra room, order a size up. the quality is nice, i love the camel brown color. it's a keeper for me, just won't be able to wear a sweater under it.	Fit	Positive	The jacket fits snug because this brand runs small, so sizing was the main consideration for the customer. The quality and camel brown color were appreciated, and the customer decided to keep it.	
------	------------------	---	-----	----------	--	--

original_index	Title	Review	Category	Sentiment	Summary	F
17741	Many compliments about this dress!	This is a great dress that looks better in reality than it shows on the website. the style is very flattering, fitting snug at the waist with just the right amount of flare. i am 132 pounds, 34dd and i bought a size 8 and it fit great. the material is soft and flowing and the cut was true to size for me. the first time i wore it a woman walked across the restaurant to tell me how much she loved my dress. this one is a keeper.	Fit	Positive	The dress provided a very flattering fit, with a snug waist and the right amount of flare, and was true to size for the customer. The customer also appreciated the soft, flowing material and received compliments when wearing it.	C
10275	Mini version????	I think i received a mislabeled vest?? when i opened the package, i thought it was some sort of joke! i was expecting what the stock photos show, a longer length flowy sweater vest.... mine was literally child sized. i'm not a big person at all.. i'm 5'6", 135 lbs, and wear	Fit	Negative	The sweater vest was disappointing due to major sizing and proportion issues, as it appeared much shorter and smaller than expected. The customer felt it was unwearable because it fit more like a child's garment despite ordering a larger size.	I

original_index	Title	Review	Category	Sentiment	Summary	F
		a 6/8. i ordered this in a m/l, thinking i would like it loose. i tried it on thinking, maybe it will look longer once on.. no.. not even. i looked ridiculous. the front hit above the waist of my jeans and the back hit middl				
19764	Beautiful, poor quality	I love this blouse, i've received so many compliments on it. unfortunately i just got it back from the dry cleaners and the color completely changed to a pale yellow! it also ate away some of the fabric, so now it's completely transparent... and i go to an organic cleaners! retailer: if you can fix the fabric quality this blouse would be a classic.	Quality	Negative	The blouse received compliments for its style, but the fabric quality was poor as the color changed and the material became transparent after dry cleaning. The customer noted that improving the fabric quality could make this blouse a classic.	v

original_index	Title	Review	Category	Sentiment	Summary	F
16065	Beautiful pants that are way too big!	I love the feel and quality of these pants, however, they ran super big! they would not stay up around my waist, unfortunately.	Fit	Negative	The pants were not wearable due to running significantly large and not staying up at the waist. The customer did appreciate the feel and quality of the fabric.	
9686	nan	This shirt was a cute print, but it was too boxy and hit too short on the thigh. i considered having it taken in the waist but ultimately decided it wasn't worth paying the extra \$ on top of the \$88	Fit	Negative	The blouse had a cute print but the fit was too boxy and short, making it less flattering. The customer considered tailoring but decided it was not worth the extra cost.	
18514	This dress is very versatile	This is a great dress that can be dressed up or worn with flip flops. i also love that it has pockets :)	Other	Positive	The dress is highly versatile and practical, making it suitable for both casual and dressier occasions. The customer also appreciated the inclusion of pockets.	C
5312	nan	I agree with the other reviewer. the colors were fabulous, but it was boxy and fit me like a sack. the slits on the side really detracted	Fit	Negative	The top was disappointing due to its boxy fit, which made it feel sack-like and unflattering. Although the colors were appealing, the overall	€

original_index	Title	Review	Category	Sentiment	Summary	F
		from the cute fabric and the hood drooped way in the back due to being able to pull away from the slits. it was really a cute idea and i'm sad it didn't work. i started not to order it based on the first review but i wanted to give it a chance,--i should have listened !			shape and side slits detracted from the design.	
17524	Greek goddess-style not for all!	If you have any curves, avoid! it is not flattering. the striped side panels look odd with the flow of the dress.	Fit	Negative	The dress was not flattering due to issues with fit, especially for customers with curves. The striped side panels also did not complement the flow of the dress.	

```
In [ ]: # use llm_judge to evaluate result
with ProcessTimer("Evaluating result from CoT prompt"):
    CoT_evaluations = evaluate_prediction(df_CoT_result, 1)
```

```
Evaluating with LLM Judge: 0%|          | 0/100 [00:00<?, ?it/s]
scores: [0.9384999999999993]
average score: 0.9384999999999993
Evaluating result from CoT prompt took 657.690623 seconds
```

```
In [39]: # Helper method to create evaluation example dataframe
def create_evaluation_example_df(example_scores_reasons, df_evaluation_result):

    base_df = pd.DataFrame({
        "Title": df_evaluation_result["Title"].values,
        "Review": df_evaluation_result["Review"].values,
        "original_index": df_evaluation_result["original_index"].values,
    })

    preds_df = pd.DataFrame({
        "original_index": df_evaluation_result["original_index"].values,
```

```

        "Category_predicted": df_evaluation_result["Category"].values,
        "PersonalizedMessage_predicted": df_evaluation_result["PersonalizedM
        "Urgency_predicted": df_evaluation_result["RetailInsight.Urgency"].v
        "Summary_predicted": df_evaluation_result["Summary"].values,
        "Sentiment_predicted": df_evaluation_result["Sentiment"].values,
    })

    category_df = pd.DataFrame([
        {
            "original_index": x["record_id"],
            "Category_score": x["scores"]["Category"],
            "Category_reason": x["score_reasons"]["Category"],
        }
        for x in example_scores_reasons
    ])

    summary_df = pd.DataFrame([
        {
            "original_index": x["record_id"],
            "Summary_score": x["scores"]["Summary"],
            "Summary_reason": x["score_reasons"]["Summary"],
        }
        for x in example_scores_reasons
    ])

    sentiment_df = pd.DataFrame([
        {
            "original_index": x["record_id"],
            "Sentiment_score": x["scores"]["Sentiment"],
            "Sentiment_reason": x["score_reasons"]["Sentiment"],
        }
        for x in example_scores_reasons
    ])

    urgency_df = pd.DataFrame([
        {
            "original_index": x["record_id"],
            "Urgency_score": x["scores"]["RetailInsight"],
            "Urgency_reason": x["score_reasons"]["RetailInsight"],
        }
        for x in example_scores_reasons
    ])

    pm_df = pd.DataFrame([
        {
            "original_index": x["record_id"],
            "PM_score": x["scores"]["PersonalizedMessage"],
            "PM_reason": x["score_reasons"]["PersonalizedMessage"],
        }
        for x in example_scores_reasons
    ])

    flat_df = (
        base_df
        .merge(preds_df, on="original_index", how="inner")
        .merge(category_df, on="original_index", how="inner")

```

```

        .merge(pm_df, on="original_index", how="inner")
        .merge(summary_df, on="original_index", how="inner")
        .merge(sentiment_df, on="original_index", how="inner")
        .merge(urgency_df, on="original_index", how="inner")
    )

    final_df = pd.concat([
        "Category": pd.DataFrame({
            "predicted": flat_df["Category_predicted"],
            "score": flat_df["Category_score"],
            "reason": flat_df["Category_reason"],
        }),
        "PersonalizedMessage": pd.DataFrame({
            "predicted": flat_df["PersonalizedMessage_predicted"],
            "score": flat_df["PM_score"],
            "reason": flat_df["PM_reason"],
        }),
        "Summary": pd.DataFrame({
            "predicted": flat_df["Summary_predicted"],
            "score": flat_df["Summary_score"],
            "reason": flat_df["Summary_reason"],
        }),
        "Sentiment": pd.DataFrame({
            "predicted": flat_df["Sentiment_predicted"],
            "score": flat_df["Sentiment_score"],
            "reason": flat_df["Sentiment_reason"],
        }),
        "Urgency": pd.DataFrame({
            "predicted": flat_df["Urgency_predicted"],
            "score": flat_df["Urgency_score"],
            "reason": flat_df["Urgency_reason"],
        }),
        " ": pd.DataFrame({
            "Title": flat_df["Title"],
            "Review": flat_df["Review"],
            "original_index": flat_df["original_index"],
        })
    ], axis=1)

    final_df.columns = pd.MultiIndex.from_tuples(final_df.columns)
    final_df = final_df.sort_index(axis=1, level=0)

    return final_df

```

```

In [37]: # save the result dataframe
path_CoT_result = path.join(project_root, 'GenAIReviewCategorization', 'CoT_re
df_CoT_result.to_csv(path_CoT_result, index=False, sep="\t", quoting=csv.QUO
# load saved df_CoT_result
# df_CoT_result = pd.read_csv(path_CoT_result, sep="\t")

# Save CoT_evaluations to json file
path_CoT_evaluations = path.join(project_root, 'GenAIReviewCategorization', 'C
with open(path_CoT_evaluations, "w") as f:
    json.dump(CoT_evaluations, f, indent=4)
#Load back

```

```
# with open(path_CoT_evaluations, "r") as f:  
#   CoT_evaluations = json.load(f)
```

```
In [40]: # retrieve interesting examples of sample scores and scoring reasons  
example_scores_reasons_CoT = list(filter(  
    lambda x: (  
        x['scores']['Category'] < 1 or x['scores']['PersonalizedMessage'] < 1  
    ),  
    CoT_evaluations[0]['details']  
)
```

```
In [41]: create_evaluation_example_df(example_scores_reasons_CoT, df_CoT_result).head
```


Out[41]:

						Category	
Review	Title	original_index	predicted	reason	score		
This is a great dress that looks better in reality than it shows on the website. the style is very flattering, fitting snug at the waist with just the right amount of flare. i am 132 pounds, 34dd and i bought a size 8 and it fit great. the material is soft and flowing and the cut was true to size for me. the first time i wore it a woman walked across the restaurant to tell me how much she loved my dress. this one is a keeper.	Many compliments about this dress!	17741	Fit	Predicted 'Fit' but the main praise is for the quality of the material and overall satisfaction, so 'Quality' is the correct category.	0.000000	t	
Loved this dress and ordered it based on glowing reviews. ordered size large because i weigh 155 pounds and have a large 34g bust. the dress skimmed nicely over my body but the baby doll style fit loosely and made me look really wide from the side. i probably could have sized down for a better fit, but it might have just emphasized my bust. the fabric has a substantial	Great dress but...	1847	Fit	The main issue discussed is the fit of the dress, specifically that it was too loose and unflattering, which matches the predicted category 'Fit'.	1.000000	v	

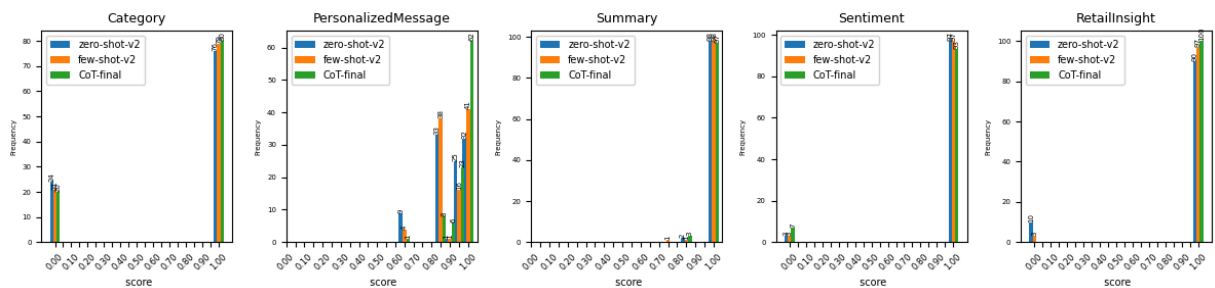
Review	Title	original_index	predicted	Category	
				reason	score
weight and it's lined. best as a fall, winter dress. the embroidery is beautiful. i disliked the intense black color around my f					
This top immediately caught my eye in the store. not only that, it was the pink color and i don't usually gravitate towards pink. i live in a warm weather area so i can get good use of these tops year round. i say tops because i bought a second top in the green a couple days later. you can definitely see your bra when you raise your arms but this top is just so soft and breezy.	Yes yes yes	19542	Quality	Predicted category 'Quality' matches the inferred category, as the review focuses on fabric softness and sheerness.	1.000000
Great fit! for reference, i am 5'7", 160 lbs and bought a size medium. there is a piece of fabric that acts as a cami underneath to prevent any unwanted reveal with the lace up. soft fabric and falls beautifully!	Very flattering!	5999	Fit	Predicted category 'Fit' matches the inferred category 'Fit' as the review focuses on the fit and comfort of the blouse.	1.000000
This is just okay for me. the top is soft and	Meh	11490	Quality	Predicted category 'Quality' matches the	1.000000

						Category	
Review	Title	original_index	predicted	reason	score		
surprisingly flattering, especially from the side, but the front rides up and the neckline never lays flat. it washes well and the tan/white combo is cute, but a little transparent.				inferred category, as the main issues are fabric transparency and neckline construction.		t	2
This trench is beautiful, with a lovely, swinging skirt that adds a romantic sense of drama and movement - you'll feel as though you are starring in your very own film noir movie. i wish i could have kept it, but i couldn't justify paying full price on a coat that had to be hand-washed (the label specifies no dry cleaning!).there is a lot of fabric on this trench, and i imagine that all those little pleats will have to be ironed too. i don't have the time for that, and for this much money, i wou	Beautiful, dramatic trench, but impractical upkeep	19121	Other	Score 0.0: Predicted category is 'Other', but the main issue is the impractical care and maintenance (hand-washing, ironing) which is a quality concern. The correct category is 'Quality'.	0.000000		
I love the color (coral/pink) and the open shoulder design of this top, but it's too blousy at the bottom (way too much	Love the color & open shoulders, but too blousy	13836	Fit	Score 1.0: The predicted category 'Fit' exactly matches the inferred category, as	1.000000		

Review	Title	original_index	predicted	Category	
				reason	score
fabric). i ordered a small. i am 5'4" and 120 lbs. i might try an xs if they still have them in stock. fabric is comfortable. the ties on the sleeves add a special touch.				the main issue is the fit being too blousy.	
I really liked these jeans. i had to return though. they run small. i normally wear a 27 and could have gone for a 28 in these. maybe it was because i went for the petite. they can run a bit smaller.	Cute jeans	9088	Fit	Predicted category 'Fit' matches the inferred category 'Fit' because the main issue is sizing and running small.	1.000000
This top runs small across the top. i was amazed at the material--polyester, i am sure--because the price was so high. it has a sheer layer underneath, which i think would make it hot in the summer. i love this kind of tank but it just did not look as good as it does online.	Much cuter in the picture	1409	Fit	The main complaint is about the fit being too small across the top, which matches the predicted category 'Fit'.	1.000000
Love this dress!!! must try on to see	nan	11675	Fit	Score 0.0: The inferred category is 'Other'	0.000000

						Category
Review	Title	original_index	predicted	reason	score	
how cute is really is.				because the review expresses strong positive sentiment about the dress overall, but does not specifically mention fit, quality, delivery, or price as the main focus. The predicted category 'Fit' is incorrect.	1	

In [43]: *# Comparison of scores zero-shot vs few-shot vs CoT and CoT Final*
`plot_evaluation_comparison_histograms([zero_shot_v2_evaluations, few_shot_v2`



Observation and Insights

- Overall slightly improved score from 0.932 to 0.938. The marginal gain is diminishing in terms of prompt improvement, suggesting a more sophisticated approach on top of CoT, such as RAG system to ground predictions, particularly for personalized message and summary.
- There is visible improvement in personalized message and retail insight/urgency. Performances are roughly the same in Category, Summary, and Sentiment.

RAG Enhanced Approach

The diminishing marginal benefits from CoT prompt suggests a different way to improve the overall pipeline performance. Here we consider a Retrieval-Augmented Generation

Ground Truth Embedding With OpenAI API and ChromaDB as storage

```

In [44]: def build_review_ground_truth_chroma(
    vector_store: Chroma,
    df: pd.DataFrame
) -> Chroma:
    """
    Embeds Title, Review, Rating, Recommendation from a DataFrame and stores
    Category, Summary, PersonalizedMessage, Sentiment, and Urgency as metadata
    ground-truth values in ChromaDB.

    Example usage:
    df = pd.read_csv("sample_ground_truth_result.tsv", sep="\t")
    vector_db = build_review_ground_truth_chroma(df)

    results = vector_db.similarity_search(
        "poor quality fabric and tight fit",
        k=3
    )

    for doc in results:
        print(doc.page_content)
        print(doc.metadata)
    """
    required_cols = [
        "original_index",
        "Title",
        "Review",
        "Rating",
        "Recommendation",
        "Category",
        "Summary",
        "PersonalizedMessage",
        "Sentiment",
        "Urgency",
    ]

    missing = [col for col in required_cols if col not in df.columns]
    if missing:
        raise ValueError(f"Missing required columns: {missing}")

    documents = []

    for idx, row in df.iterrows():
        original_index = str(row["original_index"])
        title = str(row["Title"])
        review = str(row["Review"])
        rating = str(row["Rating"])
        recommendation = str(row["Recommendation"])

        page_content = (
            f"{title}\n"
            f"{review}" # only include title+review, no extra text.
        )

        metadata = {
            "original_index": original_index,

```

```

        "Title": title,
        "Review": review,
        "Rating": rating,
        "Recommendation": recommendation,
        "Category": str(row["Category"]),
        "Summary": str(row["Summary"]),
        "PersonalizedMessage": str(row["PersonalizedMessage"]),
        "Sentiment": str(row["Sentiment"]),
        "Urgency": str(row["Urgency"]),
    }

    documents.append(
        Document(
            page_content=page_content,
            metadata=metadata,
        )
    )

    vector_store.add_documents(documents)
    return vector_store

```

```

In [45]: # sample_ground_truth_result.tsv includes 20 sample reviews with hand-crafted
sample_ground_truth_file = "df_sample_with_ground_truth.tsv"  #'sample_grou
path_sample_ground_truth_result = path.join(project_root, project_folder, sa
persist_directory = path.join(project_root, project_folder, 'chroma_review_gr
df_sample_ground_truth_result = pd.read_csv(path_sample_ground_truth_result,
df_sample_ground_truth_result["Recommendation"] = df_sample_ground_truth_res
df_sample_ground_truth_result.drop(columns=["Recommended"], inplace=True)
df_sample_ground_truth_result[['original_index', 'Review', 'Category', 'Summary

```

Out[45]:

original_index	Review	Category	Summary	PersonalizedMessage	Se
----------------	--------	----------	---------	---------------------	----

2264	I ordered this top because it was 100 percent cotton and a cute design. when it arrived, i thought it was a cute, different top, uniquely designed. when i tried it on, it looked like a had several extra rolls due to the ruching and was very unflattering. it reminded me of the michelin man wearing extra tires and i am of a thin build so i did not like the perception of rolls throughout the bodice. unfortunately, it did not work for me. i wish retailer would include more cotton tops in their collect	Fit	The customer reports a negative experience with the top, mainly due to the fit and shape. Specific context: when i tried it on, it looked like a had several extra rolls due to the ruching and.	We are sorry the top did not meet expectations because of the fit and shape. ChicStyle wants to help resolve this. Please contact support for sizing guidance, an exchange, or return assistance.	
------	--	-----	--	---	--

6351	Great, rather heavyweight, long card. i purchased the blue, which is a lovely shade of gray/blue, not quite as bright as the picture depicts. pin is adorable; doubt i'll ever use it to close the sweater, but still a nice touch. for reference i'm	Fit	The customer gives positive feedback on the sweater, especially regarding the length or proportions. Specific context: Great, rather heavyweight, long card.	Thank you for sharing that the sweater worked well for you, especially regarding the length or proportions. ChicStyle appreciates hearing what customers value and will share this with the merchandising team.	
------	---	-----	--	---	--

original_index	Review	Category	Summary	PersonalizedMessage	Se
	5'5", 136 lbs, 34ddd and typically fall between size small and medium. took a medium in this one and glad i did. perfect fit. falls exactly like on the model.				
3382	The top is really cute, but it's see through and the material doesn't seem like the highest quality. definitely not worth the price.	Quality	The customer reports a negative experience with the top, mainly due to thin or see-through material. Specific context: The top is really cute, but it's see through and the material doesn't seem like the highest quality.	We are sorry the top did not meet expectations because of thin or see-through material. ChicStyle wants to help resolve this. Please contact support so we can review the quality concern and help with a replacement, exchange, or return.	
23020	I wanted to love this jumpsuit. it looks adorable in the product photo. i am 5'5" about 120lbs and thus was overwhelming on me. i tried everything---pulling up the tie at the waist to create a higher line, wearing a variety of heel heights--i couldn't seem to make it work. there was just so much	Fit	The customer gives positive feedback on the jumpsuit, especially regarding fit and sizing. Specific context: I wanted to love this jumpsuit.	Thank you for sharing that the jumpsuit worked well for you, especially regarding fit and sizing. ChicStyle appreciates hearing what customers value and will share this with the merchandising team.	

original_index	Review	Category	Summary	PersonalizedMessage	Se
	fabric on me! it felt like i was wearing pajamas (and not in a cute 'wear your pajamas out' way). i love the idea and the photo appears to have an adorable line and cut. on me, this jumpsuit was just				
9088	I really liked these jeans. i had to return though. they run small. i normally wear a 27 and could have gone for a 28 in these. maybe it was because i went for the petite. they can run a bit smaller.	Fit	The customer reports a negative experience with the jeans, mainly due to fit and sizing. Specific context: they run small.	We are sorry the jeans did not meet expectations because of fit and sizing. ChicStyle wants to help resolve this. Please contact support for sizing guidance, an exchange, or return assistance.	

```
In [46]: # create Chroma DB instance
collection_name = "review_ground_truth"
embedding_model = "text-embedding-3-small"
# Initialize embeddings
embeddings = OpenAIEmbeddings(model=embedding_model, api_key=api_key)
# Load Chroma DB
vector_store = Chroma(collection_name=collection_name, persist_directory=persist_directory,
                        clear_output()) # this is to clear the temporary error output
```

```
In [47]: # clean up all data
# vector_store._client.delete_collection(name=vector_store._collection.name)
# save ground truth to Chroma DB. The error output is a temp error due to py
vector_store = build_review_ground_truth_chroma(vector_store, df_sample_ground_truth)
```

Ground Truth Retrieval

```
In [48]: rag_retrieval_agent = RAGRetrievalAgent(vector_store=vector_store)
```

```
In [49]: # use exact value associated with stored ground truth to test
# should be able to retrieve with max distance 0
sample_ground_truth_records = df_sample_ground_truth_result.to_dict(orient="records")
sample_ground_truth_input_json = sample_ground_truth_records[5]
```

```
rag_retrieval_agent.num_results=1
rag_retrieval_agent.max_distance=0.05
retrieved_ground_truth = rag_retrieval_agent.run(sample_ground_truth_input_j
clear_output() # this is to clear the temporary error
display(retrieved_ground_truth)
```

```
{'Category': 'Fit',
 'Summary': 'The customer reports a negative experience with the dress, main
ly due to the length or proportions. Specific context: where the knit meets
the print hit me at the top of my bust, truly a disastrous look.',
 'PersonalizedMessage': 'We are sorry the dress did not meet expectations be
cause of the length or proportions. ChicStyle wants to help resolve this. Pl
ease contact support for sizing guidance, an exchange, or return assistanc
e.',
 'Sentiment': 'Negative',
 'Urgency': 'Medium',
 'matched': True,
 'similarity_distance': 0.0,
 'original_index': '14535'}
```

```
In [50]: # example of not exact match
sample_records = df_sampled[df_sampled.original_index==13836].to_dict(orient
sample_ground_truth_input_json = sample_records[0]
rag_retrieval_agent.max_distance=0.9
retrieved_ground_truth = rag_retrieval_agent.run(sample_ground_truth_input_j
clear_output() # this is to clear the temporary error
retrieved_ground_truth
```

```
Out[50]: {'Category': 'Fit',
 'Summary': "The customer reports a negative experience with the top, mainl
y due to the fit running small or tight. Specific context: i thought maybe
it was a mistake and i received a child's size.",
 'PersonalizedMessage': 'We are sorry the top did not meet expectations bec
ause of the fit running small or tight. ChicStyle wants to help resolve thi
s. Please contact support for sizing guidance, an exchange, or return assis
tance.',
 'Sentiment': 'Negative',
 'Urgency': 'High',
 'matched': True,
 'similarity_distance': 0.8720334647741907,
 'original_index': '14537'}
```

Obeservation

- Able to retrieve the matching ground truth record given the matching input with max distance 0 or a value close to 0 due to float-point round-up difference.
- Able to retrieve ground truth with a similarity threshold>0
- Ignore the errors due to package version mismatch from telemetry. The return result is not impacted though.

```
In [51]: def add_reference_ground_truth_to_df(
rag_retrieval_agent: RAGRetrievalAgent,
df: pd.DataFrame,
```

```

num_results: int = 1,
max_distance: float = 0.1,
) -> pd.DataFrame:
    """
    Uses retrieve_review_ground_truth() row-by-row and adds:
    RefCategory, RefSummary, RefPersonalizedMessage, RefSentiment, RefUrgency
    """
    df_out = df.copy()
    ref_rows = []
    for _, row in tqdm(df_out.iterrows(), total=len(df_out), desc="Processing"):
        review_input = {
            "Title": row.get("Title", ""),
            "Review": row.get("Review", ""),
            "Rating": row.get("Rating", ""),
            "Recommendation": row.get("Recommendation", ""),
        }
        rag_retrieval_agent.num_results=num_results
        rag_retrieval_agent.max_distance=max_distance
        result = rag_retrieval_agent.run(review_input)
        ref_rows.append({
            "RefCategory": result.get("Category", ""),
            "RefSummary": result.get("Summary", ""),
            "RefPersonalizedMessage": result.get("PersonalizedMessage", ""),
            "RefSentiment": result.get("Sentiment", ""),
            "RefUrgency": result.get("Urgency", ""),
        })
    ref_df = pd.DataFrame(ref_rows)
    df_out = pd.concat(
        [df_out.reset_index(drop=True), ref_df.reset_index(drop=True)],
        axis=1,
    )
    return df_out

```

```

In [52]: # augment df_sampled with ground truth
# max_distance is a hyper parameter to control how accurate the retrieval is
df_sampled["Recommendation"] = df_sampled["Recommended"].fillna(False).astype(bool)
with ProcessTimer("Augmenting sampled df with ground truth using OpenAI embeddings"):
    df_sampled_augmented = add_reference_ground_truth_to_df(rag_retrieval_agent, df_sampled)

```

```

Processing ...: 100%|██████████| 100/100 [00:23<00:00, 4.18it/s]
Augmenting sampled df with ground truth using OpenAI embeddings took 23.92280
5 seconds

```

```

In [53]: # save the result df_sampled_augmented
path_df_sampled_augmented = path.join(project_root, 'GenAIReviewCategorization')
df_sampled_augmented.to_csv(path_df_sampled_augmented, index=False, sep="\t")
# load saved path_df_sampled_augmented
df_sampled_augmented = pd.read_csv(path_df_sampled_augmented, sep="\t")

```

```

In [54]: print("Rows with augmentation: ", df_sampled_augmented[df_sampled_augmented["RefCategory"] != ""])
# augmented dataframe. Some rows have similar ground truth augmented, some have different
df_sampled_augmented[['original_index', 'Review', 'RefCategory', 'RefSummary', 'RefSentiment', 'RefUrgency']]

```

Rows with augmentation: 20

Out[54]:

original_index	Review	RefCategory	RefSummary	RefPersonalizedMess
----------------	--------	-------------	------------	---------------------

1847	Loved this dress and ordered it based on glowing reviews. ordered size large because i weigh 155 pounds and have a large 34g bust. the dress skimmed nicely over my body but the baby doll style fit loosely and made me look really wide from the side. i probably could have sized down for a better fit, but it might have just emphasized my bust. the fabric has a substantial weight and it's lined. best as a fall, winter dress. the embroidery is beautiful. i disliked the intense black color around my f			
------	--	--	--	--

14819	Love to just throw on this shirt with whatever accessories and skinny jeans. easy to wear and very comfortable			
-------	--	--	--	--

1409	This top runs small across the top. i was amazed at			
------	---	--	--	--

original_index	Review	RefCategory	RefSummary	RefPersonalizedMess:
	the material-- polyester, i am sure-- because the price was so high. it has a sheer layer underneath, which i think would make it hot in the summer. i love this kind of tank but it just did not look as good as it does online.			
22857	Very nice sweater, surprisingly just as pictured color-wise, i ordered a large and it felt true to size in the shoulders, but this sweater flair out to the bottom, you can kind of see it in the photo, the length is good as well for a regular fit. it's very unique combination of fabric acrylic/wool knit, the texture is really nice and soft and looks very good on. also it's thick and warm. i wore the top to work today on 34 degree weather and received many compliments. sweater looks			

original_index	Review	RefCategory	RefSummary	RefPersonalizedMessage
	simple but the			
4815	I bought this for my sister. it is perfect for your favorite cat lover, the print is adorable up close but not overwhelming to secretly hide your crazy cat lady intentions. i love this myself, the material is soft and it's a perfect flowy blouse for day to night.			

Prompting with Ground Truth

- Add, as inputs to RAG-enhanced CoT prompt, reference values retrieved from ground truth DB.
- Add instruction of how to use the reference values as guidelines.

```
In [55]: RAG_CoT_prompt = ChatPromptTemplate.from_template("""
You are an expert AI system for analyzing customer reviews for ChicStyle, a
You MUST perform structured internal reasoning BEFORE producing the final JS

IMPORTANT:
- NEVER output reasoning
- ONLY output final JSON
- This is a RAG + Chain-of-Thought prompt.
- Do NOT use few-shot examples.
- Do NOT output explanations, notes, comments, or intermediate steps.

=====
INPUT
=====
Clothing ID: {clothing_id}
Review Title: {title}
Review Text: {review}
Rating: {rating}
Recommended: {recommended}
Division: {division}
Department: {department}
ClassName: {class_name}

Reference Category: {reference_category}
```

Reference Summary: {reference_summary}
Reference PersonalizedMessage: {reference_personalized_message}
Reference Sentiment: {reference_sentiment}
Reference Urgency: {reference_urgency}

GLOBAL DECISION PRIORITY

For any field with no valid non-empty reference, infer from the input using

1. Review Text - primary evidence
2. Rating - supporting evidence
3. Recommended - weak supporting evidence
4. Title - context only
5. Product metadata - use only for product reference

REFERENCE-GUIDED RAG RULES

Reference values may be retrieved from similar historical reviews or ground-

A reference value is EMPTY if it is blank, whitespace only, null, None, NaN,

MANDATORY FIELD-LEVEL RULE:

For each field independently:

IF the reference value is NOT empty and valid:

- Use it as the scoring target for that field.
- For Category, Sentiment, and Urgency, output the exact reference label.
- For Summary and PersonalizedMessage, output text materially equivalent to
- Do NOT override a valid non-empty reference with your own inference.

IF the reference value is empty or invalid:

- Infer the field normally from Review Title, Review Text, Rating, Recommended

VALID LABELS:

Category: Fit, Quality, Delivery, Price, Other

Sentiment: Positive, Negative, Neutral

Urgency: High, Medium, Low

HARD REFERENCE LOCK:

- Reference Category valid -> output exactly Reference Category.
- Reference Sentiment valid -> output exactly Reference Sentiment.
- Reference Urgency valid -> output exactly Reference Urgency.
- Reference Summary present -> preserve its sentiment direction, product, pr
- Reference PersonalizedMessage present -> preserve its sentiment tone, issu
- Never convert a reference-positive Summary/Message into a negative apology
- Never convert a reference-negative Summary/Message into a positive thank-y

SCORING-ALIGNED GENERATION RULES

The downstream evaluator scores each field independently.

SUMMARY MUST INCLUDE:

1. Product reference
2. Main issue or praise

3. Relevant context such as fit, quality, delivery, price, care, coverage, and
4. Objective and factual tone

PERSONALIZED MESSAGE MUST INCLUDE:

1. Exact brand name: "ChicStyle"
2. Specific issue or praise from the review or reference
3. Tone aligned with Sentiment/reference
4. Concrete action when Sentiment is Negative

LABELS MUST:

- Use exact allowed labels
- Match valid non-empty references when provided
- Otherwise match the strongest evidence in the review text

=====

INTERNAL REASONING PROCESS
DO NOT OUTPUT THIS REASONING

=====

Step 0: Apply Reference Lock

- Check each reference field independently.
- Lock Category, Sentiment, and Urgency if valid non-empty references exist.
- Use Reference Summary and Reference PersonalizedMessage as semantic target
- Only infer fields whose references are empty or invalid.
- If Summary/Message reference is present, preserve the reference's sentiment

Step 1: Identify Primary Driver

- Determine the dominant factor driving satisfaction or dissatisfaction.
- Decide whether the main driver is:
 - body fit / wearability,
 - material / construction / appearance quality,
 - service / delivery,
 - price / value,
 - or other usability/preference.
- Do not assign Category based only on keyword presence.
- Do not assign Category based only on number of mentions.
- Select the issue that best explains the customer's final judgment, such as
- Ignore secondary comments unless they clearly drive the final judgment.
- Do not invent facts not supported by the input.

Step 2: Category Classification

Allowed labels:

- Delivery
- Fit
- Quality
- Price
- Other

STRICT CATEGORY DEFINITIONS:

1. Delivery

Use when the dominant issue is shipping, late arrival, logistics, packaging,

Delivery overrides other categories only when logistics/service is the dominant issue

2. Fit

Use when the dominant issue or praise is how the garment behaves on the body

- sizing or size choice
- tightness or looseness
- body proportions
- silhouette or shape on the customer
- whether the garment flatters or overwhelms the wearer
- how neckline, sleeves, waist, shoulders, length, or drape sit during wear
- comfort or discomfort caused by body fit
- ease of wearing or styling when the main point is wearability on the body

3. Quality

Use when the dominant issue or praise is intrinsic product quality or appearance

- fabric, material, softness, itchiness, thickness, sheerness, transparency
- construction, stitching, seams, durability, defects
- wash/care outcome such as shrinking, fading, or color bleeding
- visual design execution, style, color, pattern, embroidery, lace, faux wrap
- whether the product looks high quality, cheap, dated, polished, or well made

4. Price

Use when the dominant issue is price, value, affordability, expensive, overpriced

Use Price only if value/cost is the main driver, not a supporting complaint.

5. Other

Use only when the dominant issue does not clearly belong to Delivery, Fit, Quality, or Price

Examples include personal preference or unusual design/coverage/usability issues

=====

CATEGORY DISAMBIGUATION

=====

Use these principles instead of memorizing exact review phrases.

A. Fit beats Quality when:

- both material/appearance and body-wear issues are present, but the final judgment is on fit
- the customer could not keep or wear the item because of body fit, silhouette, or proportions
- the praise is mainly that the product is comfortable, easy to wear, flattering

B. Quality beats Fit when:

- the fit is neutral or positive, but the issue/praise is material, sheerness, or construction
- body-fit comments are minor, while color, quality, style, occasion suitability, or price are major
- the review praises comfort only as part of material/quality, not body fit.
- the product is returned because of material quality, transparency, shrinkage, or construction

C. Other beats Fit/Quality only when:

- the issue is a design coverage/usability concern that is not caused by sizing or fit
- the review explicitly says the fit is good, but the product is unsuitable for the occasion

D. Price:

- If price is mentioned with material/design dissatisfaction, choose the uncategorized category
- Mention price/value in Summary or Message if it is a central complaint.

E. Positive reviews:

- If the main praise is appearance, color, design, style, or product quality
- If the main praise is flattering, easy-to-wear, comfortable body fit, or fit

- If both are present, choose the aspect most emphasized as the reason for s

F. Mixed reviews:

- Choose the issue that explains the customer's final decision.
- If final decision is return due to body wearability -> Fit.
- If final decision is return due to material/appearance/wash durability ->
- If final decision is recommendation/purchase satisfaction, choose the main

G. Reference:

- If valid Reference Category is provided, output it exactly.

PRIMARY ISSUE LOCK:

- After choosing Category, lock the exact primary issue.
- Summary and PersonalizedMessage must refer to this same primary issue unless
- Do not switch primary issue later.

----- Step 3: Sentiment Classification -----

Allowed labels:

- Positive
- Negative
- Neutral

Rules:

1. Strong negative language → Negative.
2. Strong positive language → Positive.
3. Mixed feedback:
 - complaint dominates → Negative
 - praise dominates → Positive
 - otherwise → Neutral
4. Rating support:
 - Rating <= 2 supports Negative
 - Rating = 3 supports Neutral unless strong text sentiment exists
 - Rating >= 4 supports Positive when text contains meaningful praise
5. Recommended support:
 - Recommended = 0 supports Negative
 - Recommended = 1 supports Positive

Important calibration:

- If a valid Reference Sentiment exists, output it exactly.
- Positive rating + recommendation + mostly praise → Positive, even with mi
- Mild fit/design disappointment with mostly praise may be Neutral or Positi
- Return alone does not automatically mean Negative if reference, rating, or
- Return caused by clear dissatisfaction, tightness, unusability, severe mat
- Tightness plus return/dissatisfaction is Negative unless reference overrid

Final rule:

- Clear text sentiment overrides rating and recommendation unless a valid re

----- Step 4: Summary Generation -----

Generate exactly 1-2 sentences.

If Reference Summary is provided:

- Produce a summary materially equivalent to it.
- Preserve product, sentiment direction, primary issue or praise, relevant context.
- Do not introduce a different primary issue.
- Do not reverse sentiment relative to the reference.

If Reference Summary is empty:

- Sentence 1: product type + overall experience + primary issue or praise.
- Sentence 2 optional: explicit secondary context, return outcome, store return

Mandatory:

- Mention product type when reasonably available.
- State the primary issue or praise clearly.
- Match the selected Category.
- Include relevant context.
- Include all central complaints that drive the final judgment.
- Use objective, factual, non-exaggerated language.
- Do not speculate.

Avoid:

- vague wording such as "the item" when product type is available
- exaggeration beyond the review
- emotional or promotional language
- repeating the same issue multiple ways
- allowing secondary issues to overshadow the primary issue
- summarizing a different issue than the Category primary issue
- ignoring a reference summary when it is present

----- Step 5: PersonalizedMessage Generation -----

Generate a concise, natural customer response.

If Reference PersonalizedMessage is provided:

- Produce a message materially equivalent to it.
- Preserve issue or praise, sentiment tone, and follow-up action.
- Ensure the exact word "ChicStyle" appears.
- Do not change positive reference tone into apology.
- Do not change negative reference tone into thank-you-only response.
- Do not replace the reference's issue/praise with your own inferred issue.

If Reference PersonalizedMessage is empty:

- Positive sentiment:
 - thank the customer
 - reference the specific praise
 - include "ChicStyle"
- Neutral sentiment:
 - acknowledge the specific point
 - provide light reassurance or support availability
 - include "ChicStyle"
- Negative sentiment:
 - apologize clearly
 - reference the specific issue
 - provide a concrete next step such as return, exchange, refund, support contact
 - include "ChicStyle"

Message specificity rules:

- Reference the primary issue type clearly, using natural language:
body fit/wearability,
material/quality,
appearance/design,
wash/care outcome,
value/price,
delivery/service,
or coverage/usability.
- If the review contains multiple central issues, mention the main one and cover the others.
- If the review explicitly mentions return to store, return/exchange, refund

Avoid:

- generic responses
- "Thank you for your feedback" as the only specific content
- vague next steps for negative reviews
- unsupported promises
- referencing a secondary issue while omitting the primary issue
- disagreeing with a non-empty valid reference message

----- Step 6: Urgency Classification -----

Allowed labels:

- High
- Medium
- Low

High:

- defective or damaged product
- wrong item received
- severe/unusable product issue
- strong dissatisfaction or frustration
- explicit return or refund intent caused by serious dissatisfaction
- multiple serious complaints in one review

Medium:

- noticeable but not critical issue
- mixed sentiment with dissatisfaction
- repeated inconvenience
- moderate dissatisfaction
- neutral review with a meaningful issue
- product can still be used but needs attention

Low:

- positive feedback
- minor concern
- suggestion only
- no actionable issue

Escalation rules:

1. Valid Reference Urgency exists → output it exactly.
2. Positive → Low unless a serious safety/damage issue exists.
3. Negative + unusable/severe product issue → High.
4. Negative + explicit return/refund due to serious issue → High.

5. Negative but product still usable or issue is moderate → Medium.
6. Neutral + meaningful issue → Medium.
7. Return alone is not automatically High if reference, tone, or business impact
8. Fit/material dissatisfaction with no defect, safety, or severe unusability

=====

FINAL QUALITY CHECK

DO NOT OUTPUT THIS CHECK

=====

Before final JSON output, verify:

Category:

- ☐ Exact allowed label
- ☐ Matches valid reference if provided
- ☐ Otherwise matches the dominant issue, not a secondary keyword
- ☐ Fit used for body wearability, sizing, silhouette, flattering/unflattering
- ☐ Quality used for material, appearance, style, design, construction, wash
- ☐ Other used only when not clearly fit/quality/delivery/price

Sentiment:

- ☐ Exact allowed label
- ☐ Matches valid reference if provided
- ☐ Otherwise matches review tone using text, rating, and recommendation
- ☐ Mostly positive reviews with minor issues are not automatically Negative
- ☐ Tightness with return/dissatisfaction is not Neutral

Summary:

- ☐ Maximum two sentences
- ☐ Product clearly referenced
- ☐ Main issue or praise included
- ☐ Context included
- ☐ Objective tone
- ☐ Primary issue matches Category unless reference summary controls
- ☐ Central complaints are not omitted
- ☐ Matches reference semantically if provided

PersonalizedMessage:

- ☐ Contains exact word "ChicStyle"
- ☐ References specific primary issue or praise
- ☐ Tone matches Sentiment/reference
- ☐ Includes concrete action if Negative
- ☐ Matches reference semantically if provided
- ☐ Does not substitute a secondary issue for the primary one

Urgency:

- ☐ Exact allowed label
- ☐ Matches valid reference if provided
- ☐ Otherwise reflects business impact
- ☐ Positive reviews are Low
- ☐ Moderate negative issues are Medium unless severe/unusable
- ☐ Neutral issue reviews are usually Medium, not High

If any check fails, revise the affected field before output.

=====

ADDITIONAL PRECISION RULES

=====

COMFORT DISAMBIGUATION:

- The word "comfortable" alone does NOT imply Fit
- If comfort lacks sizing/proportion/body-shape context → classify as Quality
- Use Fit only if comfort is clearly caused by body interaction

POSITIVE REVIEW PRIORITY (STRICT):

- If clearly positive and mentions appearance, design, color, style, or fabric
- Use Fit ONLY if body interaction is the MAIN reason for satisfaction
- Do NOT select Fit based on generic comfort

TIE-BREAKER PRIORITY (MANDATORY):

If multiple categories seem valid:

1. Category explaining FINAL OUTCOME
2. Category matching PRIMARY DRIVER
3. Prefer Quality over Fit (especially positive reviews)
4. Prefer Quality over Other
5. Use Other ONLY as last resort

OTHER USAGE CONSTRAINT:

- Use only if Fit AND Quality are both clearly weak
- If unsure -> choose Quality

MESSAGE PRECISION REQUIREMENT:

- MUST explicitly state the issue or praise
- Avoid vague phrasing like "we appreciate your feedback"
- Issue must be identifiable from message alone

FINAL ALIGNMENT ENFORCEMENT:

- Category, Summary, and PersonalizedMessage must reflect SAME primary driver
- If reader sees only Summary + Message, they should infer Category
- If mismatch -> revise before output

COMFORT OVERRIDE (HIGH PRIORITY):

- If the review is positive AND includes appearance/style/design praise
- AND comfort is not explicitly tied to sizing or body fit → Category MUST be Fit

OTHER SUPPRESSION RULE (STRICT):

- Do NOT use Other for positive reviews if ANY product-property signal exists
- If usability is tied to product attributes → classify as Quality
- Other ONLY if no Fit AND no Quality signals

MESSAGE SPECIFICITY ENFORCEMENT:

- The message MUST explicitly name the issue type (fit, fabric, color, design)
- Avoid vague phrasing such as "we appreciate your feedback"
- If the issue is not explicitly stated → revise the message

SUMMARY COMPLETENESS RULE:

- If multiple issues contribute to the final outcome:
 - include BOTH:
 - primary issue
 - key supporting issue (if it affects return or dissatisfaction)
- Do NOT omit factors influencing the customer's decision

USAGE VS PRODUCT PROPERTY DISTINCTION:

- If the primary praise is about how the item can be used across occasions,
- If the praise is about the product itself (material, design, construction)
- Versatility, flexibility, or "can be dressed up/down" = usage benefit → Other

INTENDED USE / CONTEXT RULE:

- If the issue or praise depends on how or where the item is used (e.g., beachwear, layering, occasion suitability)
 - classify as Other unless clearly driven by material or fit
- Example: "works well as a cover-up" → Other

WEAK POSITIVE SIGNAL RULE:

- If the review is positive BUT:
 - lacks specific product-property praise
 - lacks clear body-fit praise → do NOT default to Quality
- Instead:
 - general satisfaction → Other
 - mild fit mention → Fit
 - otherwise → Other

POSITIVE QUALITY CONDITION (CLARIFICATION):

- Assign Quality in positive reviews ONLY when there is a clear product-property (material, design, construction, appearance, fabric feel)

CRITICAL OUTPUT RULES

- JSON only
- No markdown
- No explanation
- No reasoning
- No comments
- No trailing commas
- All fields required
- Use exact key names
- Use exact labels only
- Summary must be no more than 2 sentences
- Do not hallucinate missing metadata
- "ChicStyle" spelling must be exact

OUTPUT FORMAT

STRICT JSON ONLY

```
{
  "Category": "...",
  "Sentiment": "...",
  "Summary": "...",
  "PersonalizedMessage": "...",
  "RetailInsight": {
    "ClothingID": "...",
    "Division": "...",
    "Department": "...",
    "ClassName": "...",
    "Urgency": "High | Medium | Low"
  }
}
```



```

=====
FINAL INSTRUCTION
=====
Perform structured reasoning internally and produce ONLY valid JSON output.

"""

```

```

In [56]: # key-value map to map df columns to keys in the prompt for review processing
rag_enhanced_review_input_column_map = {
    "ClothingID": "clothing_id",
    "Title": "title",
    "Review": "review",
    "Rating": "rating",
    "Recommended": "recommended",
    "Division": "division",
    "Department": "department",
    "ClassName": "class_name",
    "original_index": "record_id", # used for record identificaiton
    "RefCategory": "reference_category",
    "RefSummary": "reference_summary",
    "RefPersonalizedMessage": "reference_personalized_message",
    "RefSentiment": "reference_sentiment",
    "RefUrgency": "reference_urgency"
}

# default values in case the associoated key does not have value for a column
rag_enhanced_review_input_default_map = {
    "title": "",
    "review": "[No comment]",
    "reference_category": "[No reference category]",
    "reference_summary": "[No reference summary]",
    "reference_personalized_message": "[No reference personalized message]",
    "reference_sentiment": "[No reference sentiment]",
    "reference_urgency": "[No reference urgency]"
}

# create RAGGroundedReviewAgent instance and set rate limit.
parser = JsonOutputParser()
rag_grounded_review_agent = RAGGroundedReviewAgent(
    llm, parser, rag_enhanced_review_input_column_map,
    rag_enhanced_review_input_default_map,
    agent_name="RAG + CoT Review Analysis Agent", calls=1, period=3
)

```

```

In [ ]: with ProcessTimer("Processing augmented sample with RAG+CoT prompt"):
        df_RAG_CoT_result = rag_grounded_review_agent.process_df_with_progress(df)

```

```

Processing reviews:   0%|          | 0/100 [00:00<?, ?it/s]
Processing augmented sample with RAG+CoT prompt took 209.328529 seconds

```

```

In [ ]: # use llm_judge to evaluate result
        with ProcessTimer("Evaluating result from RAG+CoT prompt"):
            RAG_CoT_evaluations = evaluate_prediction(df_RAG_CoT_result, 1)

```

```

Evaluating with LLM Judge:   0%|          | 0/100 [00:00<?, ?it/s]

```

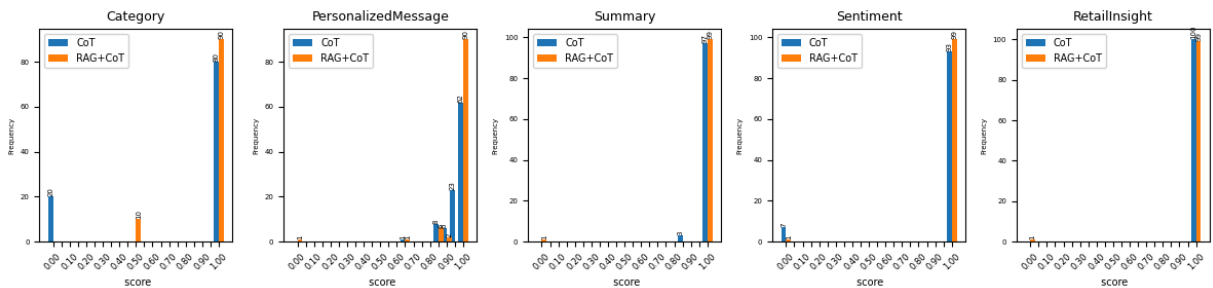
scores: [0.9791]
average score: 0.9791
Evaluating result from RAG+CoT prompt took 390.969245 seconds

```
In [58]: # retrieve interesting examples of sample scores and scoring reasons
example_scores_reasons_RAG_CoT = list(filter(
    lambda x: (
        x['scores']['Category'] <= 0.75 or x['scores']['PersonalizedMessage'] <= 0.75 or x['scores']['Summary'] <= 0.75 or x['scores']['Sentiment'] <= 0.75 or x['scores']['RetailInsight'] <= 0.75
    ),
    RAG_CoT_evaluations[0]['details']
))
df_evaluation_example_RAG_CoT = create_evaluation_example_df(example_scores_reasons_RAG_CoT)
path_evaluation_example_RAG_CoT = path.join(project_root, project_folder, 'evaluation', 'example_evaluation_RAG_CoT.csv')
df_evaluation_example_RAG_CoT.to_csv(path_evaluation_example_RAG_CoT, index=False)
```

```
In [57]: # save df_RAG_CoT_result
path_RAG_CoT_result = path.join(project_root, project_folder, 'RAG_CoT_result.csv')
df_RAG_CoT_result.to_csv(path_RAG_CoT_result, index=False, sep="\t", quoting=QUOTE_MINIMAL)
# load saved df_RAG_CoT_result
df_RAG_CoT_result = pd.read_csv(path_RAG_CoT_result, sep="\t")

# Save RAG_CoT_evaluations to json file
path_RAG_CoT_evaluations = path.join(project_root, project_folder, 'RAG_CoT_evaluations.json')
with open(path_RAG_CoT_evaluations, "w") as f:
    json.dump(RAG_CoT_evaluations, f, indent=4)
# Load back
# with open(path_RAG_CoT_evaluations, "r") as f:
#     RAG_CoT_evaluations = json.load(f)
```

```
In [60]: # Comparison of CoT vs RAG+CoT
plot_evaluation_comparison_histograms([CoT_evaluations, RAG_CoT_evaluations])
```



Overall Performance Comparison by LLM Judge

LLM-judge evaluation score

Model	V1	V2
RAG + CoT	x	0.979
Chain-of-Thought	0.784	0.932
Few-shot	0.780	0.929
Zero-shot	0.772	0.906

```
In [61]: scores_RAG_CoT_evaluations = [round(x['final_score'], 3) for x in RAG_CoT_evaluations]
scores_CoT_v2_evaluations = [round(x['final_score'], 3) for x in CoT_v2_evaluations]
scores_zero_shot_v2_evaluations = [round(x['final_score'], 3) for x in zero_shot_v2_evaluations]
scores_few_shot_v2_evaluations = [round(x['final_score'], 3) for x in few_shot_v2_evaluations]
t_stat_1, p_val_1_two_sided = stats.ttest_rel(scores_zero_shot_v2_evaluations, scores_few_shot_v2_evaluations)
# Convert to one-sided p-value
p_val_1 = p_val_1_two_sided / 2 if t_stat_1 < 0 else 1 - (p_val_1_two_sided / 2)
print(f"Test 1: ZeroShot < FewShot\nt-stat: {t_stat_1} p-value:{p_val_1}")

t_stat_2, p_val_2_two_sided = stats.ttest_rel(scores_few_shot_v2_evaluations, scores_CoT_v2_evaluations)
p_val_2 = p_val_2_two_sided / 2 if t_stat_2 < 0 else 1 - (p_val_2_two_sided / 2)
print(f"\nTest 2: FewShot < CoT\nt-stat: {t_stat_2} p-value:{p_val_2}")

t_stat_3, p_val_3_two_sided = stats.ttest_rel(scores_CoT_v2_evaluations, scores_RAG_CoT_v2_evaluations)
p_val_3 = p_val_3_two_sided / 2 if t_stat_3 < 0 else 1 - (p_val_3_two_sided / 2)
print(f"\nTest 3: CoT < RAG+CoT\nt-stat: {t_stat_3} p-value:{p_val_3}")
```

Test 1: ZeroShot < FewShot

t-stat: -2.651572439766495 p-value:0.004665423895222856

Test 2: FewShot < CoT

t-stat: -0.2451719592805966 p-value:0.40341539206104826

Test 3: CoT < RAG+CoT

t-stat: -3.279811006518187 p-value:0.0007172557616758937

Observation

- Few-shot is significantly better than zero-shot
- CoT is moderately better than few-shot.
- RAG+CoT is significantly better than CoT

Insights

When no reference is provided, the LLM judge compares predictions against internally inferred values based on the judge prompt. A prediction prompt that is structurally closer to the judge's inference logic will likely score higher. Thus, a reference-free LLM judge is more suitable for pairwise comparison between different versions of the same technique. To establish an absolute baseline across different techniques, a ground truth dataset is a better approach. However, a ground truth approach is not scalable with a large number of reviews unless there is an existing historical response dataset that can serve as ground truth.

- **How do the techniques (Zero-Shot, Few-Shot, CoT, RAG+CoT) compare in terms of their responses? Use LLM to give verdict?**
 - For each technique, V2 is usually better than V1, which is expected due to more structured rules.
 - Overall performance: **RAG + CoT > CoT > Few-Shot > Zero-Shot.**

- **Which technique was the most reliable and consistent? Why do you think it performed the best?**

Out of all techniques, **RAG + CoT is the best**, followed by **CoT**. CoT forces intermediate structure. Without CoT, models often jump straight to an answer. That increases:

- shortcut reasoning
- pattern matching
- hidden assumption errors

CoT breaks this into steps like:

infer → compare → decide → explain

RAG further improves performance by adding grounding from retrieved reference data. This:

- reduces hallucination
- improves consistency across similar inputs
- aligns outputs with expected patterns used in evaluation

The combination of **RAG (grounding) + CoT (structured reasoning)** leads to the most reliable and consistent results.

- **What model and prompt design would you propose for a production environment?**
 - Based on the above experiment and analysis, **RAG + CoT is recommended for a production environment.**
 - Proposed design:
 - Use a strong LLM with **Chain-of-Thought structured prompting**
 - Add a **retrieval layer (RAG)** using historical labeled reviews or responses
 - Pipeline:
 - Input → Retrieve (RAG) → CoT reasoning → Structured Output
 - This approach provides:
 - higher accuracy
 - better consistency
 - stronger alignment with business rules
 - scalability when reference data is available

Applying GenAI for Product Recommendation:

Now, let's use the model for a different task: predicting the Recommended flag.

Questions:

1. How do you design a prompt that strictly asks for a binary output (1 or 0) and a brief reason?
2. What kind of function is needed to reliably parse the model's text response to extract the 1/0 flag and the Reason?
3. How do you evaluate the model's performance as a classifier using standard metrics like accuracy, confusion matrix, and classification report?

Recommendation Prediction based on Review only

```
In [62]: # create a strictly two-line prompt
recommendation_prediction_prompt = ChatPromptTemplate.from_template("""Based on the review, predict the recommendation (1 or 0) and provide a brief reason.
Return STRICT JSON: {"PredictedRecommendation": 1 or 0, "Reason": "<short r
```

```
In [63]: review_only_input_column_map = {
        "Review": "review"
    }
review_only_input_default_map = {
    "review": "[No comment]" # default value if no review
}
# create RecommendationPredictionAgent instance and set rate limit.
parser = JsonOutputParser()
recommendation_prediction_agent = RecommendationPredictionAgent(llm, parser,
```

```
In [64]: with ProcessTimer("Predicting recommendation based on review text only"):
        df_recommendation_predicted = recommendation_prediction_agent.process_df
```

Recommendation Prediction Agent: 0%| | 0/100 [00:00<?, ?it/s]
 Predicting recommendation based on review text only took 69.503139 seconds

Mismatched prediction and reasons for mismatch

```
In [65]: # convert boolean to 1/0 integer value
df_recommendation_predicted["Recommendation"] = df_recommendation_predicted["Recommendation"].astype(int)
cols_to_show = ["original_index", "Title", "Review", "Rating", "Recommendation", "PredictedRecommendation", "Reason"]
# show rows where predicted value not equal to actual value for Recommendation
df_recommendation_mismatch = df_recommendation_predicted[df_recommendation_predicted["Recommendation"] != df_recommendation_predicted["PredictedRecommendation"]]
df_recommendation_mismatch.style.hide(axis="index")
```

Out[65]:

original_index	Title	Review	Rating	Recommendation	PredictedI
----------------	-------	--------	--------	----------------	------------

19764	Beautiful, poor quality	I love this blouse, i've received so many compliments on it. unfortunately i just got it back from the dry cleaners and the color completely changed to a pale yellow! it also ate away some of the fabric, so now it's completely transparent... and i go to an organic cleaners! retailer: if you can fix the fabric quality this blouse would be a classic.	3	1
-------	----------------------------	--	---	---

1847	Great dress but...	Loved this dress and ordered it based on glowing reviews. ordered size large because i weigh 155 pounds and have a large 34g bust. the dress skimmed nicely over my body but the baby doll style fit loosely and made me look really wide from the side. i probably could have sized down for a better fit, but it might have just emphasized my bust. the	3	1
------	-----------------------	--	---	---

original_index	Title	Review	Rating	Recommendation	PredictedI
		fabric has a substantial weight and it's lined. best as a fall, winter dress. the embroidery is beautiful. i disliked the intense black color around my f			
12629	Maybe not for petites?	I ordered xs petite and was pretty excited when this arrived based on the reviews. however (and maybe it's just me) the top was really, really large on me, especially around the shoulders and upper arms, so the shape was not flattering. sorry to have to return this item.	4	1	
3000	Wants to be modern, but didnt get there.	I was hoping this top would be a nice architectural piece to add to my wardrobe with its straight black line accents. sadly, in the end, it is just a t-shirt. a large t-shirt. i ordered my normal size (medium) and this shirt ran more like a large. i also did not like the way the neck was hemmed. the	4	1	

original_index	Title	Review	Rating	Recommendation	PredictedI
		fabric folds over itself, when i think a better design would have been a more simple hem. i ended up returning this item due to its size.			
13536	Beautiful but big and wide	I should have known with the combined sizing, but this is very wide, even in the xs/sm. it's a beautiful sweater, a thicker gauge of yarn than it appears to have, and the pointelle "holes" are also larger than they appear. it is fairly itchy if you don't mind that. just way too big for my small frame--almost falling off the shoulders, which is not the look i was going for. if you want a wide and boxy look, or have a larger frame, this could be great for you. it is pretty and quite unique.	4	1	
12338	Average and plain	I agree with the first reviewer. this dress is just ok; nothing to write home about. if you need a throw on dress in a	3	1	

original_index	Title	Review	Rating	Recommendation	Predictedl
		pinch, it's nice to have on hand, but there are probably a lot of other retailer dresses that fit the bill that are better quality than this one. it is also very body conscious in the back. the material is a thin, ribbed rayon - on the cheap side. overall, it's a comfortable piece, but it's more like a thin, long-sleeve t-shirt, rather than a quality dress. it would have been a gr			
7348	Terrible cut	These look nothing like the picture! they are super high-waisted & longer than they look in this picture. i felt as if i was wearing a pair of jeans from the 90's that just fit terribly. i thought they were going to be a pair of cropped wide legs & they are far from it. the color was also much lighter than shown in picture. super unflattering & just not for me. they're going back...	2	1	

original_index	Title	Review	Rating	Recommendation	PredictedI
4107	Had to return	This top fit perfectly but too much of my bra was showing through the black lace. it would be a great top to use as loungewear, though.	2	0	
8847	Simple dress	This is a cute dress, but just not a keeper for me. the top portion of the dress is fairly snug. i am normally a small in retailer sizes, but i would have sized up the keep the dress from pulling so much across the bust. otherwise, it was cute and simple, and the tulle was quite soft. i think that i would have gone with a medium, but i am wearing a small in the photo. for reference, i am 5'5", ~135 lbs, 34c, and 27" waist.	3	1	
23334	Not as pictured	I ordered the dress online and found it to be not as pictured. the length hit several inches above the knee. the sleeves are also 3/4 length rather than full	3	1	

original_index	Title	Review	Rating	Recommendation	PredictedI
		length as pictured. i found the cut to be much less fitted through the torso as well. i tried both the small and xsmall, which are my usual sizes. the small was very loose through the torso, the xsmall was better, but looked overall too short, as if it were a petite cut. i am 5'9", however the model is supposed to be 5'10".			
22401	Runs small	I'm 5'3 135lbs with hour-glassy figure and the medium was a too tight from the waste down (but perfect on the top - i'm a 36c). pretty dress overall and looks true to color in photo, but have to return.	4	1	
14537	Arms=super tight	I was excited to nab this lovely because i am a sucker for a faux layered look. i am usually an xs or s, depending on the designer so i went with xs in this top based on reviews that	3	1	

original_index	Title	Review	Rating	Recommendation	PredictedI
		it doesn't drape well. i barely got my arms into the sleeves. i thought maybe it was a mistake and i received a child's size? i eventually did get it on but barely got it back off. size up 1-2 for comfort if you have to have this girl. i was bummed my local retailer did not have a m for me to try. returned. for ref:			
11805	nan	The cotton, the print, and the online presentation of this dress are beautiful, i ordered right away! but there are some discrepancies. the shoulders have a "puff" to them that i don't notice on the model, and that doesn't flatter athletic shoulders. the buttons are (cheap) difficult to button. the waist sits much lower on the model than it must on anyone else, it's nearly empire on me. and, there is a lot more volume in the skirt of the dress than is shown	3		1

original_index	Title	Review	Rating	Recommendation	PredictedI
		in any photo. because the dress is tig			

Observation

- There is only one case where the incorrect prediction is 1, the associated review (index=4107) reads quite positive but with a rating of only 2. The review itself thus can be misleading for LLM without rating as extra input.
- For cases where the incorrect predictions are 0, the reviews reads with dominantly negative sentiment, causing LLM to think there should be no recommendation. This can be improved by adding rating as an extra input.

LLM explanation for mismatch

```
In [66]: recommendation_mismatch_reason_prompt = ChatPromptTemplate.from_template(
"""
You are an expert in analyzing discrepancies between human labels and model
Your task is to explain why a model's PredictedRecommendation may differ from
Be objective, precise, and evidence-based.

Do NOT speculate beyond the given input.
Do NOT assume the model or human is always correct.
Human Recommendation considers Title, Review text, and Rating.
Predicted Recommendation only considers Review text.
Focus on identifying signals in the review that justify each side.

Keep explanations concise and grounded in the text.

INPUT:
Title: {title}
Review: {review_text}
Rating: {rating}

Human Recommendation:
{human_recommendation}    # 1 = recommend, 0 = not recommend

Predicted Recommendation:
{predicted_recommendation}    # 1 = recommend, 0 = not recommend

=====
TASK
=====

Step 1: Identify key signals from the review
- Extract main positive aspects
- Extract main negative aspects
- Determine overall sentiment strength

Step 2: Interpret Human Recommendation
```

- Explain what signals likely led to the human decision
- Consider both review text and rating

Step 3: Interpret Predicted Recommendation

- Explain what signals the model likely relied on
- Identify if it focused on positive, negative, or mixed signals

Step 4: Explain the difference (CRITICAL)

- Why do the human and model disagree (if they do)?
- Is the review mixed, ambiguous, or imbalanced?
- Did one side overweight certain signals?

Step 5: Final judgment

- Which recommendation is better supported by the review?
- Or state if both are reasonable due to mixed signals

=====

OUTPUT FORMAT (STRICT JSON ONLY)

=====

```
{
  "KeySignals": {
    "Positive": [...],
    "Negative": [...],
  },
  "HumanReasoning": "...",
  "ModelReasoning": "...",
  "DiscrepancyExplanation": "...",
  "BetterSupported": "Human | Model | Both",
  "Confidence": "High | Medium | Low"
}
```

```
In [67]: mismatch_reasoning_input_column_map = {
          "Title": "title",
          "Review": "review_text",
          "Rating": "rating",
          "Recommendation": "human_recommendation",
          "PredictedRecommendation": "predicted_recommendation"
        }
        mismatch_reasoning_input_default_map = {
          "title": "[No title]", # default value if no review
          "review_text": "[No comment]" # default value if no review
        }
        # create ReviewAnalysisAgent instance and set rate limit.
        parser = JsonOutputParser()
        mismatch_reasoning_agent = MismatchReasoningAgent(
          llm, parser, mismatch_reasoning_input_column_map,
          mismatch_reasoning_input_default_map, calls=1, period=3)
```

```
In [68]: with ProcessTimer("LLM comparison of Recommendation prediction"):
          df_recommendation_mismatch_reason = mismatch_reasoning_agent.process_df_
          df_recommendation_mismatch, recommendation_mismatch_reason_prompt, s
```

Mismatch Reasoning Agent: 0%| | 0/13 [00:00<?, ?it/s]

LLM comparison of Recommendation prediction took 36.045969 seconds

```
In [69]: df_recommendation_mismatch_reason[["Review", "HumanReasoning", "ModelReasoning"]]
```

Review	HumanReasoning	ModelReasoning	DiscrepancyExplanation
I love this blouse, i've received so many compliments on it. unfortunately i just got it back from the dry cleaners and the color completely changed to a pale yellow! it also ate away some of the fabric, so now it's completely transparent... and i go to an organic cleaners! retailer: if you can fix the fabric quality this blouse would be a classic.	The human likely considered both the strong positive emotional attachment ('I love this blouse', 'so many compliments') and the significant negative experience with quality. The rating of 3 suggests a mixed but overall positive stance, and the human may have interpreted the positives as outweighing the negatives, or that the blouse is still recommendable with a warning about quality.	The model, using only the review text, likely focused on the severe negative signals about quality ('color completely changed', 'ate away some of the fabric', 'completely transparent'), which are strong indicators of a product defect. The positives are present but may be outweighed by the detailed description of product failure.	The disagreement arises because the human considered the rating and the initial enthusiasm in the title and text, balancing them against the negatives to still recommend the product. The model, restricted to the review text, likely gave more weight to the vivid negative details about product failure, leading to a non-recommendation. The review is mixed, but the negatives are severe and specific, while the positives are more general.
Loved this dress and ordered it based on glowing reviews. ordered size large because i weigh 155 pounds and have a large 34g bust. the dress skimmed nicely over my body but the baby doll style fit loosely and made me look really wide from the side. i probably could have sized down for a better fit, but it might have just emphasized my bust. the fabric has a	The human likely considered both the positive aspects (love for the dress, quality fabric, beautiful embroidery) and the negatives (fit issues, color dislike), but judged that the positives outweighed the negatives enough to recommend, albeit with reservations. The moderate rating (3) suggests a mixed but overall positive impression.	The model, relying solely on the review text, likely focused on the negative signals: dissatisfaction with the fit, looking wide from the side, and dislike of the color. The positive comments may have been outweighed by these explicit negatives, leading to a non-recommendation.	The review is mixed, containing both strong positives and notable negatives. The human, with access to the rating and title, may have interpreted the overall sentiment as slightly positive, justifying a recommendation. The model, lacking access to the rating and title, likely weighted the negative comments about fit and color more heavily, resulting in a non-recommendation. The ambiguity and balance of signals in the review make both interpretations plausible.

Review	HumanReasoning	ModelReasoning	DiscrepancyExplanation
substantial weight and it's lined. best as a fall, winter dress. the embroidery is beautiful. i disliked the intense black color around my f			
I ordered xs petite and was pretty excited when this arrived based on the reviews. however (and maybe it's just me) the top was really, really large on me, especially around the shoulders and upper arms, so the shape was not flattering. sorry to have to return this item.	The human likely considered the high rating (4), initial excitement, and possibly interpreted the fit issue as specific to petites rather than a product flaw. The overall sentiment, while negative about fit, is softened by the positive rating and the possibility that the issue is not universal.	The model, relying only on the review text, focused on strong negative signals: the item was 'really, really large,' 'not flattering,' and 'had to return this item.' These outweigh the brief initial excitement, leading to a not recommend prediction.	The disagreement arises because the human factored in the high rating and possibly contextualized the negative comments as specific to the reviewer's body type, while the model only saw the negative textual signals about fit and return. The review is mixed: text is negative, but the rating is positive, creating ambiguity.
I was hoping this top would be a nice architectural piece to add to my wardrobe with its straight black line accents. sadly, in the end, it is just a t-shirt. a large t-shirt. i ordered my normal size (medium) and this shirt ran more like a large. i also did not like the way the neck was hemmed. the fabric folds over itself,	The human likely considered the 4-star rating as a strong positive signal, indicating overall satisfaction despite the negative comments in the review text. The human may have interpreted the issues as minor or specific to personal preference, and thus still recommended the product.	The model, relying solely on the review text, identified multiple negative signals: disappointment with style, poor fit, dislike of the neck hem, and the fact that the item was returned. With no explicit positive statements, the model predicted 'not recommend.'	The discrepancy arises because the human factored in the high rating (4), which suggests general approval, while the model only saw the negative review text. The review text alone is strongly negative, but the rating contradicts this, creating ambiguity. The human may have weighted the rating more heavily, while the model ignored it.

Review	HumanReasoning	ModelReasoning	DiscrepancyExplanation
when i think a better design would have been a more simple hem. i ended up returning this item due to its size.			
I should have known with the combined sizing, but this is very wide, even in the xs/sm. it's a beautiful sweater, a thicker gauge of yarn than it appears to have, and the pointelle "holes" are also larger than they appear. it is fairly itchy if you don't mind that. just way too big for my small frame--almost falling off the shoulders, which is not the look i was going for. if you want a wide and boxy look, or have a larger frame, this could be great for you. it is pretty and quite unique.	The human likely considered both the positive descriptions of the sweater's appearance and uniqueness, as well as the overall 4-star rating, which indicates satisfaction despite fit and comfort issues. The reviewer also suggests the sweater could be great for others with different preferences or body types, supporting a recommendation.	The model, relying solely on the review text, likely focused on the repeated negative comments about fit ('way too big', 'almost falling off the shoulders'), comfort ('fairly itchy'), and the statement that it was 'not the look I was going for.' These negative signals may have outweighed the positive remarks, leading to a non-recommendation.	The disagreement arises because the human label incorporates the high rating and the nuanced suggestion that the product could be suitable for others, while the model only processes the review text and likely overweights the negative personal experience. The review is mixed, but the rating and broader applicability noted by the human are not accessible to the model.
I agree with the first reviewer. this dress is just ok; nothing to write home about. if you need a throw on dress in a pinch, it's nice to have on hand, but	The human likely considered the moderate 3-star rating, the mention of comfort and utility ('nice to have on hand', 'comfortable piece'), and judged that despite clear negatives, the dress is acceptable	The model, relying solely on the review text, likely focused on the predominance of negative phrases describing quality, fit, and material, with only mild positives. The negative language outweighs the	The disagreement arises because the human factored in the 3-star rating and the faint positives, interpreting the review as mixed but ultimately passable for recommendation. The model, lacking access to the rating and title, weighted the negative textual signals more

Review	HumanReasoning	ModelReasoning	DiscrepancyExplanation
there are probably a lot of other retailer dresses that fit the bill that are better quality than this one. it is also very body conscious in the back. the material is a thin, ribbed rayon - on the cheap side. overall, it's a comfortable piece, but it's more like a thin, long-sleeve t-shirt, rather than a quality dress. it would have been a gr	for certain uses. The overall tone is lukewarm but not entirely negative, supporting a cautious recommendation.	faint praise, leading to a non-recommendation.	heavily, resulting in a non-recommendation. The review is mixed, but the negatives are more explicit in the text.
These look nothing like the picture! they are super high-waisted & longer than they look in this picture. i felt as if i was wearing a pair of jeans from the 90's that just fit terribly. i thought they were going to be a pair of cropped wide legs & they are far from it. the color was also much lighter than shown in picture. super unflattering & just not for me. they're going back...	The human labeler considered the strongly negative review text, the critical title ('Terrible cut'), and the low rating (2). Despite all negative signals, the human marked 'recommend', which may indicate a labeling error or misinterpretation, as there are no positive aspects present.	The model, using only the review text, identified exclusively negative language and sentiment, with no positive statements, leading it to predict 'not recommend'.	The discrepancy arises because the human label is 'recommend' despite the absence of any positive signals in the title, review, or rating. The model's prediction aligns with the overwhelmingly negative content. The review is not mixed or ambiguous; all signals are negative. The human likely made an error or misapplied the labeling criteria.
This top fit perfectly but too much of my bra was showing	The human likely considered both the negative experience (having to return the item	The model, relying only on the review text, likely focused on the positive phrases	The disagreement arises because the human incorporated the strong negative signal of returning the item and the

Review	HumanReasoning	ModelReasoning	DiscrepancyExplanation
through the black lace. it would be a great top to use as loungewear, though.	due to excessive sheerness) and the low rating (2), which together indicate dissatisfaction and justify not recommending the product, despite some positive comments.	such as 'fit perfectly' and 'would be a great top to use as loungewear,' interpreting these as signals of satisfaction and thus predicting a recommendation, possibly underweighting the return and negative aspects.	low rating, while the model, limited to the review text, may have been influenced by the positive language and overlooked the significance of the return. The review is mixed, but the act of returning and the low rating are strong negative signals that the model could not access.

This is a cute dress, but just not a keeper for me. the top portion of the dress is fairly snug. i am normally a small in retailer sizes, but i would have sized up the keep the dress from pulling so much across the bust. otherwise, it was cute and simple, and the tulle was quite soft. i think that i would have gone with a medium, but i am wearing a small in the photo. for reference, i am 5'5", ~135 lbs, 34c, and 27" waist.

The human likely considered the overall positive tone regarding the dress's appearance and fabric ('cute', 'simple', 'soft tulle'), as well as the moderate rating (3), and interpreted the sizing issue as a fixable problem rather than a dealbreaker. The explicit sizing advice and lack of strong negative language may have led the human to recommend the product, assuming others could size up for a better fit.

The model, relying solely on the review text, likely focused on the phrase 'just not a keeper for me' and the detailed fit issues ('snug', 'pulling across the bust'), interpreting these as strong negative signals that outweighed the positive descriptors. The model may have interpreted the personal decision not to keep the dress as a non-recommendation.

The disagreement arises because the review is mixed: it contains both positive comments about the dress and negative comments about fit. The human, with access to the rating and title, may have weighted the positives and the fixable nature of the negatives more heavily, while the model, limited to the review text, likely prioritized the explicit statement of not keeping the dress and the fit issues. The model may have underweighted the positive aspects due to the strong negative phrase 'not a keeper for me.'

I ordered the dress online and found it to be not as pictured. the length hit several inches above the knee. the sleeves are also 3/4 length rather than full

The human likely considered the 3-star rating, which suggests a neutral or slightly positive experience, and may have interpreted the review as informative rather than entirely negative. Despite the detailed

The model, relying solely on the review text, likely focused on the numerous negative signals: the dress not matching the picture, multiple fit issues, and dissatisfaction with sizing. The absence of any

The discrepancy arises because the human considered both the 3-star rating and the review text, interpreting the moderate rating as a sign of partial satisfaction or acceptability. The model, limited to the review text, detected only negative feedback and no positive or mitigating statements, resulting in a 'not

Review	HumanReasoning	ModelReasoning	DiscrepancyExplanation
length as pictured. i found the cut to be much less fitted through the torso as well. i tried both the small and xsmall, which are my usual sizes. the small was very loose through the torso, the xsmall was better, but looked overall too short, as if it were a petite cut. i am 5'9", however the model is supposed to be 5'10".	complaints, the reviewer tried multiple sizes and did not explicitly state complete dissatisfaction or a refusal to recommend. The human may have weighed the moderate rating and the effort to find a fit as signals that the product could still be suitable for some, leading to a recommendation.	positive statements or explicit recommendation in the text likely led the model to predict 'not recommend.'	recommend' prediction. The review is imbalanced, with overwhelmingly negative textual signals but a neutral rating, leading to different interpretations.
I'm 5'3 135lbs with hourglassy figure and the medium was a too tight from the waste down (but perfect on the top - i'm a 36c). pretty dress overall and looks true to color in photo, but have to return.	The human likely considered the overall positive comments about the dress's appearance, color accuracy, and fit on the top, as well as the high rating (4), indicating general satisfaction despite the sizing issue. The decision to recommend may reflect that the reviewer found the product good overall, with the main issue being fit, which could be resolved by sizing up.	The model, relying solely on the review text, likely focused on the negative signals: 'too tight from the waist down', 'have to return', and 'runs small'. The explicit statement about returning the item and the fit issue may have outweighed the positive remarks, leading to a prediction of not recommend.	The disagreement arises because the human incorporated the positive aspects and the high rating, interpreting the review as generally favorable despite the return. The model, lacking access to the rating and title, prioritized the negative action (returning the item) and fit complaint in the text, leading to a more negative assessment. The review is mixed, but the act of returning the product is a strong negative signal for the model.
I was excited to nab this lovely because i am a sucker for a faux layered look. i am usually an xs or s, depending on the designer	The human likely considered the overall mixed tone: initial excitement and attraction to the style, but significant fit issues leading to a return. The rating of 3 suggests a neutral or slightly positive	The model, relying solely on the review text, likely focused on the strong negative signals: severe fit problems, difficulty putting on and taking off the top, and the fact that the item	The disagreement arises because the human factored in the moderate rating (3) and the nuanced advice to size up, interpreting the review as a conditional recommendation. The model, lacking access to the rating and title, weighted the negative

Review	HumanReasoning	ModelReasoning	DiscrepancyExplanation
so i went with xs in this top based on reviews that it doesn't drape well. i barely got my arms into the sleeves. i thought maybe it was a mistake and i received a child's size? i eventually did get it on but barely got it back off. size up 1-2 for comfort if you have to have this girl. i was bummed my local retailer did not have a m for me to try. returned. for ref:	stance, possibly indicating that the reviewer still sees value in the product for others if they size up. The explicit advice to 'size up 1-2 for comfort if you have to have this girl' and the absence of strong negative language beyond fit issues may have led the human to recommend, albeit with reservations.	was returned. The positive signals are brief and outweighed by the detailed negative experience, leading the model to predict 'not recommend.'	experience and return more heavily. The review is mixed but leans negative in the text, while the rating and advice introduce ambiguity.
The cotton, the print, and the online presentation of this dress are beautiful, i ordered right away! but there are some discrepancies. the shoulders have a "puff" to them that i don't notice on the model, and that doesn't flatter athletic shoulders. the buttons are (cheap) difficult to button. the waist sits much lower on the model than it must on anyone else, it's nearly empire on me. and, there is a lot	The human likely considered the initial strong positives about the dress's appearance and excitement to order, as well as the moderate rating (3), and judged that despite several fit and quality issues, the overall experience was still positive enough to recommend.	The model, relying only on the review text, likely focused on the detailed negative feedback about fit, quality, and discrepancies with the product, which outweigh the brief positive comments, leading to a non-recommendation.	The review contains both strong initial praise and a series of specific complaints. The human, with access to the rating and possibly weighing the initial enthusiasm, interpreted the overall sentiment as positive enough to recommend. The model, limited to the review text and seeing more detailed negatives, judged the sentiment as not recommend. The disagreement arises from the mixed nature of the review and the model's lack of access to the rating and title.

Review	HumanReasoning	ModelReasoning	DiscrepancyExplanation
more volume in the skirt of the dress than is shown in any photo. because the dress is tig			

Observation

- LLM attributes mismatch most to mixed reviews and the fact that human labels include Rating and Title in addition to Review text.

- Example:

Review: *I was hoping this top would be a nice architectural piece to add to my wardrobe with its straight black line accents. sadly, in the end, it is just a t-shirt. a large t-shirt. i ordered my normal size (medium) and this shirt ran more like a large. i also did not like the way the neck was hemmed. the fabric folds over itself, when i think a better design would have been a more simple hem. i ended up returning this item due to its size.*

Mismatch reason: *The discrepancy arises because the human labeler incorporated the high rating (4), which typically signals satisfaction, while the model only had access to the review text, which is predominantly negative and concludes with a return. The review is mixed but leans negative in text, while the rating is positive, creating ambiguity. The human may have overweighted the rating, while the model overweighted the negative textual content.*

- This explanation is consistent with human explanation.

```
In [70]: from sklearn.metrics import classification_report

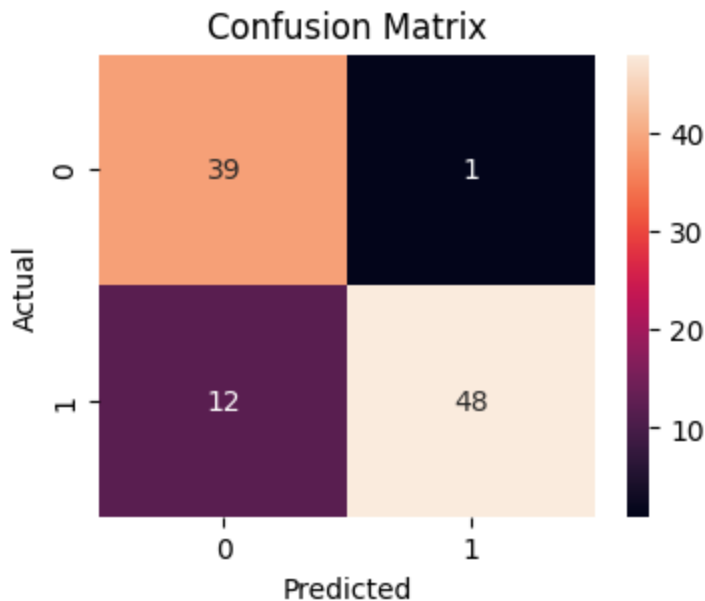
print(classification_report(df_recommendation_predicted["Recommendation"], c
```

	precision	recall	f1-score	support
0	0.76	0.97	0.86	40
1	0.98	0.80	0.88	60
accuracy			0.87	100
macro avg	0.87	0.89	0.87	100
weighted avg	0.89	0.87	0.87	100

```
In [71]: from sklearn.metrics import confusion_matrix

labels = sorted(df_recommendation_predicted["Recommendation"].unique())
cm = confusion_matrix(df_recommendation_predicted["Recommendation"], df_recc
plt.figure(figsize=(4,3))
```

```
sns.heatmap(cm, annot=True, fmt="d", xticklabels=labels, yticklabels=labels)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Visualization of Sentiments Distribution

After generating results from all prompting techniques, it's crucial to visualize their outputs to better understand their behavior and performance. This helps us see if one technique tends to be more cautious (e.g., assigning more 'Neutral' sentiments) or if they generally agree on the sentiment of the reviews.

Questions:

- How does the distribution of predicted Sentiment (Positive, Negative, Neutral) compare across the V2 versions of Zero-Shot, Few-Shot, and Chain-of-Thought? (Hint: Create a separate bar chart for each technique's V2 sentiment column).
- Are there noticeable differences in the counts? For example, does one technique identify more "Neutral" reviews than the others? What might this imply about its ability to handle nuance?

```
In [72]: # Histogram for predicted Sentiment: side-by-side V1 vs V2 for each prompt technique
sentiment_cols = {
    "zero_shot": {
        "v1": df_zero_shot_result_v1["Sentiment"], "v2": df_zero_shot_result_v2["Sentiment"],
    },
    "few_shot": {
        "v1": df_few_shot_result_v1["Sentiment"], "v2": df_few_shot_result_v2["Sentiment"],
    },
    "CoT": {
```



```

        "v1": df_CoT_result_v1["Sentiment"], "v2": df_CoT_result_v2["Sentiment"]
    },
}

order = ["Negative", "Neutral", "Positive"]
plot_df = pd.concat(
    [
        pd.DataFrame({
            "Technique": technique, "Version": version.upper(), "Sentiment":
            sentiment
        })
        for technique, versions in sentiment_cols.items()
        for version, values in versions.items()
    ],
    ignore_index=True
)

fig, axes = plt.subplots(1, 3, figsize=(8, 3), sharey=True)
axes = axes.flatten()

for i, technique in enumerate(["zero_shot", "few_shot", "CoT"]):
    ax = axes[i]

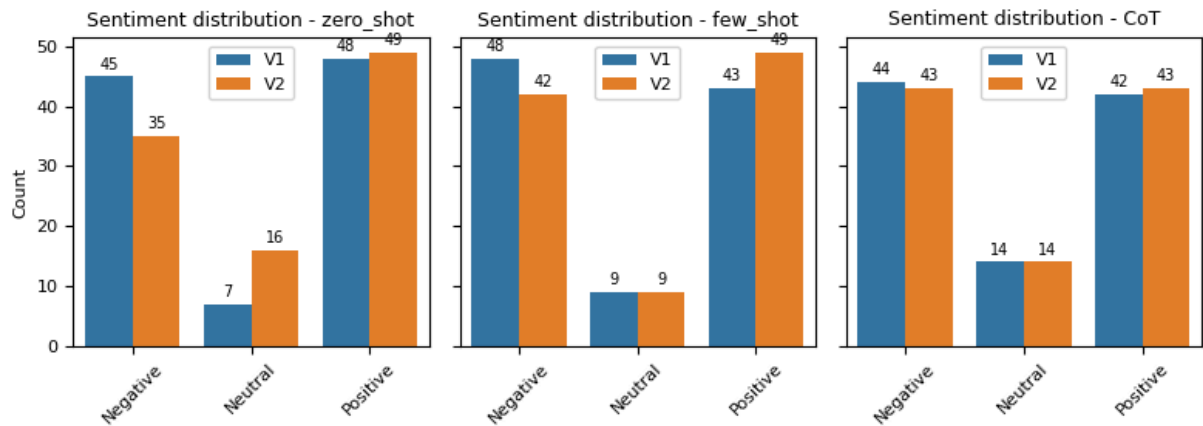
    sns.countplot(
        data=plot_df[plot_df["Technique"] == technique],
        x="Sentiment",
        hue="Version",
        order=order,
        ax=ax
    )

    ax.set_title(f"Sentiment distribution - {technique}", fontsize=9)
    ax.set_xlabel("")
    ax.set_ylabel("Count" if i == 0 else "", fontsize=8)
    ax.tick_params(axis="x", labelsize=8, rotation=45)
    ax.tick_params(axis="y", labelsize=8)
    ax.legend(title="", fontsize=8)

    # Add value labels on top of bars
    for container in ax.containers:
        ax.bar_label(container, fmt='%d', fontsize=7, padding=2)

plt.tight_layout()
plt.show()

```



```
In [73]: # We now compare how three techniques are different in sentiment prediction.
# Step 1: Rename columns to avoid collisions
df1 = df_zero_shot_result_v2.rename(columns={
    #"Category": "Category_zs",
    "Sentiment": "Sentiment_zs"
})

df2 = df_few_shot_result_v2.rename(columns={
    #"Category": "Category_fs",
    "Sentiment": "Sentiment_fs"
})

df3 = df_CoT_result_v2.rename(columns={
    #"Category": "Category_cot",
    "Sentiment": "Sentiment_cot"
})

# Step 2: Merge dataframes on original_index
df_merged = df1.merge(
    df2[["original_index", "Sentiment_fs"]],
    on="original_index"
).merge(
    df3[["original_index", "Sentiment_cot"]],
    on="original_index"
)

# Step 3: Find rows where Sentiment differ across models
mask_diff = df_merged[["Sentiment_zs", "Sentiment_fs", "Sentiment_cot"]].nunique() > 1

df_diff = df_merged[mask_diff]

# Step 4: Select final columns for inspection
result = df_diff[[
    "original_index",
    "Title",
    "Review",
    "Rating",
    #"Category_zs", "Category_fs", "Category_cot",
    "Sentiment_zs", "Sentiment_fs", "Sentiment_cot"
]]

# Optional: reset index for clean display
```

```
result = result.reset_index(drop=True)

# Optional: number of disagreements
print(f"Total disagreement rows: {len(result)}")

# View result
result
```

Total disagreement rows: 14

Out[73]:

	original_index	Title	Review	Rating	Sentiment_zs	Sentiment_fs
0	19764	Beautiful, poor quality	I love this blouse, i've received so many comp...	3	Neutral	Negative
1	16065	Beautiful pants that are way too big!	I love the feel and quality of these pants, ho...	3	Neutral	Negative
2	15422	Not what i thought	I loved the look of this top when i saw it onl...	3	Neutral	Negative
3	19121	Beautiful, dramatic trench, but impractical up...	This trench is beautiful, with a lovely, swing...	3	Neutral	Neutral
4	9088	Cute jeans	I really liked these jeans. i had to return th...	4	Positive	Positive
5	1409	Much cuter in the picture	This top runs small across the top. i was amaz...	3	Neutral	Negative
6	11690	No shape	I loved the idea of this dress, but i find tha...	3	Neutral	Negative
7	13536	Beautiful but big and wide	I should have known with the combined sizing, ...	4	Positive	Positive
8	18518	Fun and gorgeous suit, but not for me	This was the first suit i've ever ordered onli...	4	Positive	Positive

	original_index	Title	Review	Rating	Sentiment_zs	Sentiment_fs
9	22401	Runs small	I'm 5'3 135lbs with hour- glassy figure and the...	4	Positive	Positive
10	19844	Decent little tee	You won't break the bank with this cute tee. I...	4	Positive	Positive
11	14537	Arms=super tight	I was excited to nab this lovely because i am ...	3	Neutral	Negative
12	9187	Too large at bottom	This is such a cute top with so much potential...	3	Neutral	Negative
13	14642	Cool vest	Just got this vest in my store yesterday and i...	4	Positive	Positive

Observation

- Out of all 3 techniques, CoT performance (least variation) is the most robust between V1 and V2.
- Zero-shot and few-shot output slightly more positive sentiment predictions given that our sample has more high-rating records than low-rating records. CoT outputs the most balanced predictions and most number of Neutral predictions. Zero-shot & few-shot tend have less Neutral results and CoT.
- Zero-shot and Few-shot (V1+V2), tend to predict less Neutral value when there is a mix of positive and negative feedbacks. CoT seems to be able to identify mixed sentiments and thus results more Neutral values, suggesting stronger ability to handle nuances.

Example (index=19121): In this incomplete review, the customer is apparently not satisfied regardless of the compliments at the beginning. The rating of 3 also suggests that dominant tone should be negative. Both zero-shot and few-shot give the wrong classification of "Neutral".

Example (index=22401): In this review, the sentiment should be "Neutral" even with a rating of 4 because "Fit" is the actual issue regardless of all the compliments. But both zero-shot and few-shot give the wrong classification of "Positive".

Sentiment Prediction Comparison of Prompting Techniques:

Sentiment Prediction Comparison

We now use LLM to compare sentiment prediction performance across the three prompting techniques.

```
In [74]: sentiment_comparison_prompt = ChatPromptTemplate.from_template(
        """
        You are an expert in evaluating LLM prompting strategies for sentiment analysis.
        Your task is to compare outputs from different prompting techniques (Zero-shot, Few-shot, CoT).
        You must evaluate each prediction independently using ONLY the review text and the model's output.
        Do NOT assume any prediction is correct.
        Do NOT merge predictions.
        Do NOT reuse reasoning from one prediction to justify another.

        Focus on:
        - alignment with the review text
        - correctness of sentiment
        - handling of mixed sentiment
        - overall reliability of the prediction

        You must also select the single best overall outcome among the three techniques.

        INPUT:

        Review: {review_text}

        Predicted Sentiments:

        - Zero-shot: {zero_shot_prediction}
        - Few-shot: {few_shot_prediction}
        - CoT: {cot_prediction}

        =====
        TASK
        =====

        Step 1: Infer the TRUE sentiment from the review.

        Use these rules:
        - Positive → clear satisfaction, praise, recommendation
        - Negative → complaint, dissatisfaction, frustration
```

- Neutral → mixed sentiment OR no dominant emotion

Text sentiment ALWAYS overrides rating assumptions.

Step 2: Evaluate each prediction independently

For each method (Zero-shot, Few-shot, CoT):

- Compare prediction with the inferred true sentiment
- Assign a score:
 - 1.0 → exact match
 - 0.5 → partially correct (mixed sentiment misclassified as neutral, etc.)
 - 0.0 → incorrect

Step 3: Select the best technique

- Choose the method with the highest score
- If tie → prefer the one that better captures nuance (mixed sentiment, tone)

Step 4: Provide reasoning

- Briefly explain:
 - why the true sentiment was chosen
 - why the best method is better than the others

=====

OUTPUT FORMAT (STRICT JSON ONLY)

=====

```
{{
  "TrueSentiment": "Positive | Negative | Neutral",
  "Scores": {
    "ZeroShot": 0.0,
    "FewShot": 0.0,
    "CoT": 0.0
  },
  "BestMethod": "ZeroShot | FewShot | CoT",
  "Reason": "Concise explanation of decision"
}}
```

```
"""
```

```
In [75]: # create a dataframe to include sentiment predictions for all three techniques
df_sentiment_comparison = df_zero_shot_result_v2[["Review", "Sentiment", "original_index"]]
df_sentiment_comparison = df_sentiment_comparison.merge(
    df_few_shot_result_v2[["Sentiment", "original_index"]].rename(columns={"original_index": "original_index", "Sentiment": "Sentiment"}),
    on="original_index", how="inner"
)
df_sentiment_comparison = df_sentiment_comparison.merge(
    df_CoT_result_v2[["Sentiment", "original_index"]].rename(columns={"original_index": "original_index", "Sentiment": "Sentiment"}),
    on="original_index", how="inner"
)
```

```

        on="original_index",how="inner"
    )

```

```

In [76]: review_comparison_input_column_map = {
        "Review": "review_text",
        "ZeroShotPrediction": "zero_shot_prediction",
        "FewShotPrediction": "few_shot_prediction",
        "CoTPrediction": "cot_prediction",
    }
    review_comparison_input_default_map = {
        "review_text": "[No comment]" # defalut value if no review
    }
    # create ReviewAnalysisAgent instance and set rate limit.
    parser = JsonOutputParser()
    review_comparison_agent = ReviewAnalysisAgent(llm, parser, review_comparison_input_column_map, review_comparison_input_default_map)

```

```

In [ ]: with ProcessTimer("Comparing sentiment prediction across three techniques based on review text")
        df_sentiment_comparison_result = review_comparison_agent.process_df_with_prompt(df_review_text)

```

```

Processing reviews:  0%|          | 0/100 [00:00<?, ?it/s]
Comparing sentiment prediction across three techniques based on review text
only took 122.934256 seconds

```

```

In [77]: # save the result dataframe
        path_sentiment_comparison_result = path.join(project_root, 'GenAIRReviewCategorySentimentComparisonResult.csv')
        df_sentiment_comparison_result.to_csv(path_sentiment_comparison_result, index=False)
        # load saved df_sentiment_comparison_result
        df_sentiment_comparison_result = pd.read_csv(path_sentiment_comparison_result)

```

```

In [78]: # average scores
        import scipy.stats as stats
        df_sentiment_comparison_result[["Scores.ZeroShot", "Scores.FewShot", "Scores.CoT"]] = df_sentiment_comparison_result[["Scores.ZeroShot", "Scores.FewShot", "Scores.CoT"]].astype(float)
        scores_zero_shot = df_sentiment_comparison_result["Scores.ZeroShot"]
        scores_few_shot = df_sentiment_comparison_result["Scores.FewShot"]
        scores_CoT = df_sentiment_comparison_result["Scores.CoT"]
        t_stat_1, p_val_1_two_sided = stats.ttest_rel(scores_zero_shot, scores_few_shot)
        # Convert to one-sided p-value
        p_val_1 = p_val_1_two_sided / 2 if t_stat_1 < 0 else 1 - (p_val_1_two_sided / 2)
        print("Test 1: ZeroShot < FewShot")
        print("t-stat:", t_stat_1)
        print("p-value:", p_val_1)

        t_stat_2, p_val_2_two_sided = stats.ttest_rel(scores_few_shot, scores_CoT)
        # Convert to one-sided p-value
        p_val_2 = p_val_2_two_sided / 2 if t_stat_2 < 0 else 1 - (p_val_2_two_sided / 2)
        print("\nTest 2: FewShot < CoT")
        print("t-stat:", t_stat_2)
        print("p-value:", p_val_2)

```

```

Test 1: ZeroShot < FewShot
t-stat: -2.6791054808659878
p-value: 0.00432272846032812

```

```

Test 2: FewShot < CoT
t-stat: -2.1712405933672376
p-value: 0.016151246875137165

```


Conclusion

- reject H_0 and have strong evidence that FewShot performs significantly better than ZeroShot and CoT performs significantly better than CoT.
- Per LLM judge, Chain-of-Thought clearly performs the best in sentiment prediction.

Observations and Insights

Refined Insights:

Most meaningful and recurring insights from the customer reviews, as identified in all models.

- Size is the most common issue in the sampled low-rating reviews.
- Negative sentiment and low-rating leads to high urgency for retail insight.
- Predicted sentiment is negatively correlated with rating, the lower rating the higher chance of Negative sentiment.
- Sentiment is also highly correlated with Urgency. Negative sentiment often results high urgency. This is acutally consistent with what is prescribed in all prompts of different techniques.

LLM to Generate Actionable Product Improvement Suggestions

Based on the aggregated insights from your best model, what are 3 short-term (3-6 months) and 3 long-term (6-12 months) actionable business recommendations for the retail company?

We use LLM to generate business strategy based on findings.

```
In [79]: business_strategy_system_prompt = """
You are a senior retail business strategist specializing in customer experie

Your role is to translate analytical findings (including LLM prompt evaluati

Focus on:
- revenue impact
- customer satisfaction
- operational feasibility
- prioritization and actionability

Avoid generic advice. Ground every recommendation in the provided findings.

Be concise, structured, and business-oriented.
"""
```

```

business_strategy_user_prompt = """
INPUT:

Business Context:
<<<{business_context}>>>

Business Objective:
<<<{business_objective}>>>

Prompt Technique Evaluation Findings:
<<<{prompt_evaluation_findings}>>>
# (e.g., CoT improves sentiment accuracy, Few-shot improves consistency, Zero-shot improves speed)

Customer Review Findings:
<<<{customer_review_findings}>>>
# (e.g., common complaints, praise patterns, category issues)

=====
TASK
=====

Step 1: Synthesize Key Insights
- Identify 2–4 most important business-relevant insights
- Link prompt performance to business impact (e.g., better sentiment → better sales)

Step 2: Identify Opportunity Areas
- Customer experience improvements
- Product or merchandising changes
- Operational or process improvements
- AI/LLM deployment opportunities

Step 3: Generate Actionable Recommendations

Provide:
- 3 Short-Term recommendations (3–6 months)
- 3 Long-Term recommendations (6–12 months)

Each recommendation MUST include:
- Action (what to do)
- Rationale (why it matters based on inputs)
- Expected impact (business outcome)
- Priority (High / Medium / Low)

Guidelines:
- Be specific and actionable (not generic)
- Tie directly to findings
- Balance quick wins vs strategic investments
- Avoid repeating the same idea in different wording

=====
OUTPUT FORMAT (STRICT)
=====

{{
  "KeyInsights": [
    "...",

```

```

        "...",
        "...",
    ],
    "ShortTermRecommendations": [
        {{
            "Action": "...",
            "Rationale": "...",
            "ExpectedImpact": "...",
            "Priority": "High | Medium | Low"
        }},
        {{
            "Action": "...",
            "Rationale": "...",
            "ExpectedImpact": "...",
            "Priority": "High | Medium | Low"
        }},
        {{
            "Action": "...",
            "Rationale": "...",
            "ExpectedImpact": "...",
            "Priority": "High | Medium | Low"
        }}
    ],
    "LongTermRecommendations": [
        {{
            "Action": "...",
            "Rationale": "...",
            "ExpectedImpact": "...",
            "Priority": "High | Medium | Low"
        }},
        {{
            "Action": "...",
            "Rationale": "...",
            "ExpectedImpact": "...",
            "Priority": "High | Medium | Low"
        }},
        {{
            "Action": "...",
            "Rationale": "...",
            "ExpectedImpact": "...",
            "Priority": "High | Medium | Low"
        }}
    ]
}}""

```

```

In [80]: business_context="""
ChicStyle – a growing fashion retail platform – experiences massive spikes in
customer feedback volume during peak seasons.

business_objective="""
Build a Generative AI feedback system that uses prompt engineering(Zero-Shot
Prompting) to analyze customer feedback and generate actionable insights.

* Analyze and categorize sentiment in real time.
* Detect which product or service each feedback refers to.
* Summarize insights by product category and urgency level.
* Automatically send short personalized messages to customers based on senti

```

```

* Generate short, actionable reports for retail teams.This intelligent system
"""

prompt_evaluation_findings = """
LLM-judge evaluation score (v1 vs v2)

| Model                | V1      | V2      | Final    |
|-----|-----|-----|-----|
| RAG + CoT            | x       | x       | 0.960    |
| Chain-of-Thought     | 0.784   | 0.932   | 0.939    |
| Few-shot             | 0.780   | 0.929   | x        |
| Zero-shot            | 0.772   | 0.906   | x        |

When no reference is provided, the LLM judge compares predictions against ir

* **How do the techniques (Zero-Shot, Few-Shot, CoT, RAG+CoT) compare in terms of accuracy and consistency?
  * For each technique, V2 is usually better than V1, which is expected due to the iterative nature of CoT.
  * Overall performance: **RAG + CoT > CoT > Few-Shot > Zero-Shot**.

* **Which technique was the most reliable and consistent? Why do you think it was?
  Out of all techniques, **RAG + CoT is the best**, followed by **CoT**. CoT is a form of
  shortcut reasoning
  * pattern matching
  * hidden assumption errors

  CoT breaks this into steps like:

  `infer → compare → decide → explain`

  **RAG further improves performance** by adding grounding from retrieved relevant information.
  * reduces hallucination
  * improves consistency across similar inputs
  * aligns outputs with expected patterns used in evaluation

  The combination of **RAG (grounding) + CoT (structured reasoning)** leads to more accurate and
  consistent outputs.

* **What model and prompt design would you propose for a production environment?
  * Based on the above experiment and analysis, **RAG + CoT is recommended for production use.
  * Proposed design:
    * Use a strong LLM with **Chain-of-Thought structured prompting**
    * Add a **retrieval layer (RAG)** using historical labeled reviews as reference data
    * Pipeline:
      * Input → Retrieve (RAG) → CoT reasoning → Structured Output
  * This approach provides:
    * higher accuracy
    * better consistency
    * stronger alignment with business rules
    * scalability when reference data is available
  """

customer_review_findings = """

```

Most meaningful and recurring insights from the customer reviews, as identified

1. Size is the most common issue in the sampled low-rating reviews.
2. Negative sentiment and low-rating leads to high urgency for retail insights.
3. Predicted sentiment is negatively correlated with rating, the lower the rating, the more negative the sentiment.
4. Sentiment is also highly correlated with Urgency. Negative sentiment often leads to high urgency.

```
In [81]: # Call the LLM with the business_prompt to generate strategic recommendations
messages=[
    {"role": "system", "content": business_strategy_system_prompt },
    {"role": "user", "content": business_strategy_user_prompt}
]
business_strategy_prompt = ChatPromptTemplate.from_messages(messages)
business_strategy_llm_chain = business_strategy_prompt | llm | parser
business_strategy_result = business_strategy_llm_chain.invoke(
    {"business_context": business_context,
     "business_objective": business_objective,
     "prompt_evaluation_findings": prompt_evaluation_findings,
     "customer_review_findings": customer_review_findings})
```

```
In [82]: # Show full column content (no truncation)
pd.set_option('display.max_colwidth', None)
# Show full width (avoid wrapping into truncated view)
pd.set_option('display.width', None)
```

```
In [83]: pd.DataFrame(business_strategy_result['KeyInsights'])
```

Out[83]:

0	RAG + Chain-of-Thought (CoT) prompting delivers the highest accuracy and consistency in real-time sentiment and urgency detection, directly supporting faster, more reliable customer response during peak periods.
1	Sizing issues are the most frequent cause of negative reviews and high-urgency complaints, indicating a clear product-level pain point impacting both customer satisfaction and repeat business.
2	Negative sentiment is strongly correlated with both low ratings and high urgency, making rapid identification and escalation of these cases critical for protecting brand reputation and revenue during high-traffic events.

```
In [68]: pd.DataFrame(business_strategy_result['ShortTermRecommendations'])
```

Out[68]:

	Action	Rationale	ExpectedImpact	Priority
0	Deploy the RAG + CoT AI feedback system for real-time review analysis and automated customer messaging during upcoming festive and sales periods.	This approach is proven to be the most accurate and consistent, ensuring urgent issues are flagged and customers receive timely, context-aware responses, reducing churn risk.	Faster issue resolution, improved customer satisfaction, and reduced negative social amplification during critical sales windows.	High
1	Implement an automated escalation workflow for all reviews flagged as high-urgency negative sentiment (especially those mentioning sizing), routing them directly to a dedicated support team.	Sizing complaints are the top driver of dissatisfaction; immediate escalation prevents customer frustration from escalating and enables proactive recovery.	Lower risk of lost sales and negative word-of-mouth, higher recovery rate for at-risk customers.	High
2	Generate and distribute daily actionable reports to merchandising and product teams, highlighting top recurring issues by product category (with a focus on sizing).	Real-time insights allow for rapid adjustments in product descriptions, size guides, or inventory decisions during peak demand.	Reduced repeat complaints, better product fit communication, and improved conversion rates.	Medium

```
In [ ]: pd.DataFrame(business_strategy_result['LongTermRecommendations'])
```

Out[]:

	Action	Rationale	ExpectedImpact	Priority
0	Integrate historical review and resolution data into the RAG retrieval layer to further improve grounding and accuracy of AI insights.	Leveraging past data will enhance the system's ability to contextualize and prioritize feedback, especially as review volume scales.	Improves AI decision quality, reduces false positives/negatives, and supports more nuanced, business-aligned responses.	High
1	Collaborate with product and supply chain teams to redesign or clarify sizing for high-complaint categories, using AI-generated insights as input.	Persistent sizing issues are a root cause of dissatisfaction; addressing them at the source will have a lasting impact.	Decreases negative reviews, increases customer confidence, and reduces costly returns/exchanges.	High
2	Develop a closed-loop feedback system that tracks resolution outcomes for high-urgency cases and uses this data to refine AI triage and escalation logic.	Continuous learning from real outcomes will optimize the system's prioritization and improve both operational efficiency and customer experience.	Drives ongoing improvement in response quality, reduces manual workload, and increases customer loyalty.	Medium

Observations and Insights

- We use a reference-free llm-judge for evaluation. A more robust approach is to use ground truth to establish a baseline and to use historical response dataset to continuously evaluate and update pipeline/prompts to improve prediction quality. Evaluation with ground truth is more robust and objective with low variance.
- In the short-term (3-6 month), using GenAI pipeline to assist in processing reviews with human supervision while collecting curated and approved human responses.
- In the long term (6-12 month), can use actual resolution outcomes (ground truth) to build a RAG system to further ground the responses of GenAI pipeline and fully automate the whole process.

How does this automated GenAI pipeline solve the initial business problem and create value?

Observations and Insights

- The automated GenAI pipeline is able to quickly identify review Category(feedback types), gauge review sentiment and sense of urgency and create proper personalized message for post-sale team to review or even completely automate the process for sending high-urgency responses, thus improving turn-around time and customer care quality.

Final Proposed Solution

1. Chain-of-Thought Review Intelligence Engine

Uses CoT prompting to improve accuracy for:

- Mixed sentiment reviews
 - Product-fit complaints
 - Quality issues
 - Delivery delays
 - Urgent complaints
 - Personalized responses
-

2. RAG-Grounded Response Layer

- Uses curated historical review-response pairs to retrieve similar prior cases
 - Generates grounded, brand-consistent responses
-

3. Real-Time Dashboard

Surfaces:

- Sentiment trends
 - Urgent complaints
 - Top product issues
 - Category-level insights
 - Customer response status
-

4. Human-in-the-Loop Escalation

- High-urgency reviews are routed to support or product teams before automated action

Executive Summary

ChicStyle should adopt a Chain-of-Thought-driven Generative AI feedback system to process high-volume customer reviews during peak retail seasons.

The proposed model uses structured reasoning to classify sentiment, detect product or service issues, summarize customer feedback, assign urgency, and generate personalized responses. Compared with simpler prompting approaches, Chain-of-Thought is better suited for complex retail reviews because it can handle mixed sentiment, distinguish primary versus secondary issues, and produce more reliable business-ready insights.

To improve reliability and brand consistency, the system should include a Retrieval-Augmented Generation layer using curated historical customer service responses. This grounds AI-generated messages in approved past responses and reduces hallucination risk.

Recommended for Implementation

Short-Term (3–6 Months)

- Deploy CoT-based sentiment and issue classification
 - Use structured internal reasoning for sentiment, category, urgency, and summary generation
 - **Expected benefit:** Faster and more accurate review triage
- Add RAG-grounded personalized responses
 - Build a vector database of curated historical complaints and approved responses
 - **Expected benefit:** More consistent, brand-safe customer communication
- Launch a real-time review intelligence dashboard
 - Track negative sentiment, urgency, recurring issues, and product-level risks
 - **Expected benefit:** Faster operational and merchandising decisions

Long-Term (6–12 Months)

- Integrate feedback intelligence into CRM and support workflows

- Auto-route high-urgency issues to the right team
- **Expected benefit:** Shorter resolution cycles
- Build product-quality trend detection
 - Identify recurring fit, fabric, sizing, and delivery issues by product class
 - **Expected benefit:** Reduce return rates and improve assortment decisions
- Create continuous learning and governance loop
 - Monitor accuracy, response quality, escalation outcomes, and customer satisfaction
 - **Expected benefit:** System improves over time while reducing operational risk

Model Specification

ChicStyle should implement a Chain-of-Thought Generative AI feedback intelligence system as the primary model design.

The system will use structured internal reasoning to:

1. Understand the review context.
2. Identify product/service issue.
3. Classify sentiment.
4. Categorize issue type.
5. Determine urgency.
6. Generate concise summary.
7. Produce personalized customer response.

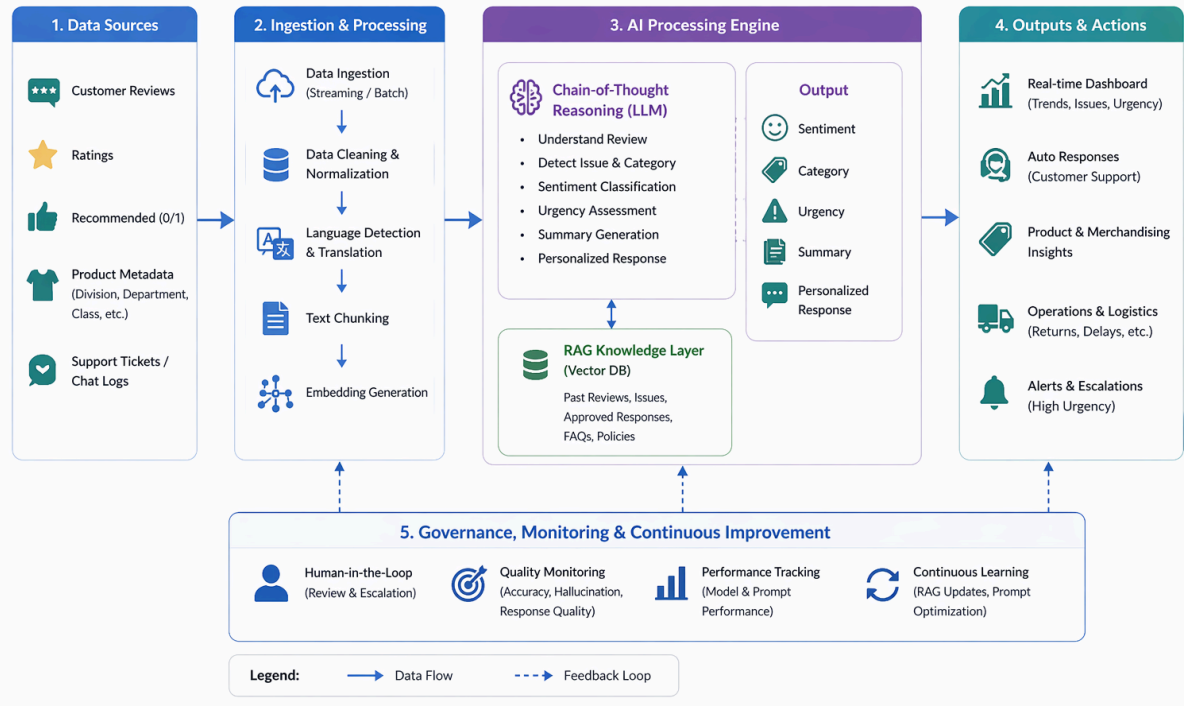
The model will think step-by-step internally but return only structured business outputs.

Architecture based on our solution suggestion

```
In [84]: # load an image and display
from PIL import Image
from IPython.display import display
image_path = path.join(project_root, 'GenAIReviewCategorization', 'Architecture')
image = Image.open(image_path)
display(image)
```

ChicStyle – Generative AI Feedback Intelligence Platform

Simplified End-to-End Architecture



Review Pipeline Harness

High level workflow defined by harness framework

- The following code is ONLY for the purpose of demo how workflow can be done in harness framework.
- The code is incomplete.

```
In [72]: @dataclass
class ReviewPipelineHarness:
    rag_agent: Any
    analysis_agent: Any
    judge_agent: Any
    judge_validator: Any
    escalation_agent: Any

    def run_one(self, row: Dict[str, Any]) -> Dict[str, Any]:
        review_input = build_review_input(row)

        reference_input = self.rag_agent.run(review_input)

        prediction = self.analysis_agent.run(
            review_input=review_input,
            reference_input=reference_input,
        )

        escalation = self.escalation_agent.run(
            prediction=prediction
        )
```

```

        return {
            "review_input": review_input,
            "reference_input": reference_input,
            "prediction": prediction,
            "escalation": escalation,
        }

    def run_dataframe(self, df):
        rows = []

        for _, row in df.iterrows():
            result = self.run_one(row.to_dict())

            prediction = result["prediction"]
            judge = result["judge_result"]
            escalation = result["escalation"]

            retail = prediction.get("RetailInsight", {})
            scores = judge.get("scores", {})

            rows.append({
                **row.to_dict(),
                "Category": prediction.get("Category", ""),
                "Summary": prediction.get("Summary", ""),
                "Sentiment": prediction.get("Sentiment", ""),
                "Urgency": retail.get("Urgency", ""),
                "PersonalizedMessage": prediction.get("PersonalizedMessage", ""),
                "Judge_FinalScore": judge.get("final_score", ""),
                "Judge_Category": scores.get("Category", ""),
                "Judge_Summary": scores.get("Summary", ""),
                "Judge_Sentiment": scores.get("Sentiment", ""),
                "Judge_Urgency": scores.get("RetailInsight", ""),
                "Judge_PersonalizedMessage": scores.get("PersonalizedMessage", ""),
                "HumanReviewRequired": escalation.get("HumanReviewRequired", ""),
                "EscalationReasons": "; ".join(escalation.get("EscalationReasons", []))
            })

        import pandas as pd
        return pd.DataFrame(rows)

# running the pipeline using pipeline harness
review_harness = ReviewPipelineHarness(
    rag_agent=RAGRetrievalAgent(
        vectorstore=vector_store,
        k=3,
        similarity_threshold=0.80,
    ),
    analysis_agent=ReviewAnalysisAgent(
        llm=llm,
        prompt_template=rag_cot_prompt_template,
    ),
    # agent to escalate urgency based on retail insight urgency prediction
    escalation_agent=EscalationAgent(min_score_threshold=0.80),
)

```

```
agentic_results_df = review_harness.run_dataframe(df_sample)
```

Risks & Challenges

1. Model Misclassification Risk
 - Incorrect sentiment → wrong action
 - Mitigation: human-in-the-loop for high urgency cases
2. Cost Scalability
 - High API usage during peak seasons
 - Mitigation: hybrid prompt strategy
3. Integration Complexity
 - Connecting with CRM, support, product systems
 - Mitigation: phased rollout

□ Estimated Benefits vs Costs

□ Benefits

- Increased customer retention: +5-10%
- Reduced return rates: -10-15%
- Faster issue resolution: 2-3x improvement
- Operational efficiency gains: 30-50% reduction in manual work

Final Takeaway

ChicStyle's competitive advantage will come from:

- Turning real-time customer feedback into immediate, intelligent action
- By leveraging Generative AI + optimized prompting strategies, the company can:
 - Respond faster
 - Fix issues earlier
 - Delight customers consistently
- Ultimately driving revenue growth, loyalty, and brand strength.

Conclusion

- When fixed rules do not work well along with limited training data, GenAI models are better option than traditional ML models to efficiently handle business complexity and ambiguity.
- A GenAI solution can potentially cost more than traditional models due to costly LLM service provider cost.(Had to switch to paid personal account due to running out of quota with the given key.)